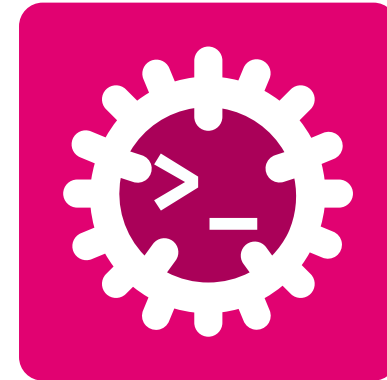


Advanced Programming

Full Course Slides



Silvan Zahno

University of Applied Sciences Western Switzerland - HES-SO Valais Wallis
Systems Engineering
Infotronics
2026-08-07 - v0.1.0



Contents

Introduction	2
Imperative Programming	42
Object-Oriented Programming	155
Design Patterns	194
Concurrency	217
Debug / Testing / Documentation	268
Glossary & Bibliography	297



Introduction

The Good, The Bad and The Ugly



Resp.	No	Abrév.	Modules Module Cours Vorlesungen	Credits	Heures par semestre Stunden pro Semester					
					1	2	3	4	5	6
anx	I3.1	EmS1	Embedded Systems 1	11			165			
sap	I3.11	EmP	Embedded Programming				75			
anx	I3.12	ElNB	Basic Electronic Circuits				90			
zas	I3.2	SWD1	Software Development 1	7			105			
zas	I3.21	AProG	Advanced Programming				75			
zas	I3.22	ProGA	Applied Programming				30			
mei	I4.1	OT1	Operational Technology 1	6				150		
mei	I4.11	CNet	Communication Networks					75		
anx	I4.12	ElNC	Circuit Design					75		
zas	I4.2	SWD2	Software Development 2	4				75		
zas	I4.21	MDSW	Model-Driven Software Engineering					30		
cfr	I4.22	AML	Applied Machine Learning					45		
mei	I5.1	EmS2	Embedded Systems 2	10					180	
anx	I5.11	AEC	Advanced Electronic Circuits						90	
mei	I5.12	ESE	Embedded Systems Engineering						90	
sti	I5.2	OT2	Operational Technology 2	10					150	
day	I5.21	RSD	Reliable Software Development						90	
sti	I5.22	IIoT	Industrial Internet of Things						60	
mei	I6.1	EmS3	Embedded Systems 3	6						160
gua	I6.11	SoC	System-on-Chip							80
mei	I6.12	RTOS	Real-Time OS							80
anx	I6.2	Data	Data	4						40
anx	I6.21	DAq	Data Acquisition							40
nir	I6.22	SProc	Signal Processing							(30)

Infotonics



Resp.	No	Abrév.	Modules Module Cours Vorlesungen	Credits	Heures par semestre Stunden pro Semester					
					1	2	3	4	5	6
anx	I3.1	EmS1	Infotronics	11			165			
sap	I3.11	EmP					75			
anx	I3.12	ElNB					90			
zas	I3.2	SWD1		7			105			
zas	I3.21	AProg					75			
zas	I3.22	ProgA					30			
mei	I4.1	OT1		6				150		
mei	I4.11	CNet						75		
anx	I4.12	ElnC						75		
zas	I4.2	SWD2		4				75		
zas	I4.21	MDSW						30		
cfr	I4.22	AML						45		
mei	I5.1	EmS2		10					180	
anx	I5.11	AEC							90	
mei	I5.12	ESE							90	
sti	I5.2	OT2		10					150	
day	I5.21	RSD							90	
sti	I5.22	IIoT							60	
mei	I6.1	EmS3		6						160
gua	I6.11	SoC								80
mei	I6.12	RTOS								80
anx	I6.2	Data		4						40
anx	I6.21	DAq								40
nir	I6.22	SProc								(30)



Advanced Programming (AProg)

Develop the ability to design, implement, test, and maintain robust software systems by applying modern programming paradigms, software design principles, concurrency, and software quality practices.

Applied Programming (ProgA)

Apply software engineering methods to develop a complete software system from analysis and design to implementation, testing, and delivery.



Course Structure

- 6h/week for week 1-7, 8h/week for week 8-15
- Project last 4 weeks @ 8h/week

Grading

Nbr	Course	Weighting				
		Exam	Labo/Exercises	Project	Semesterexam	Course
I3.21	Advanced Programming	-	1	-	2	2
I3.22	Applied Programming	-	-	1	-	1

Expectations

- Attend and actively participate in lectures and labs
- Complete exercises, labs, and project milestones
- Work independently outside class hours
- **Read and Write** a lot of code



Evaluation Criteria for Labs, Exercises and Project

Git workflow - required for all deliverables:

```
# Work and commit regularly during the session
git add .
git commit -m "feat: exercise 3 - temperature
converter"

# Mark completion with a tag after each series /
labo
git tag exercises-03
git push && git push --tags
```

- Commits show **progress** and **effort**
- Tags mark **submission points** - missing tag = not submitted
- Commit history is part of the evaluation

Exercises

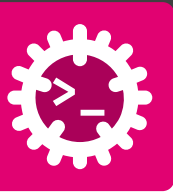
- Number of exercises **completed**
- Number of **unit tests** passing
- Code quality and structure

Labos

- One labo chosen **at random** for detailed review
- Code quality and structure
- Comments and naming

Project

- Exhaustive README.md
- Clean, documented code
- Working features
- Short written report

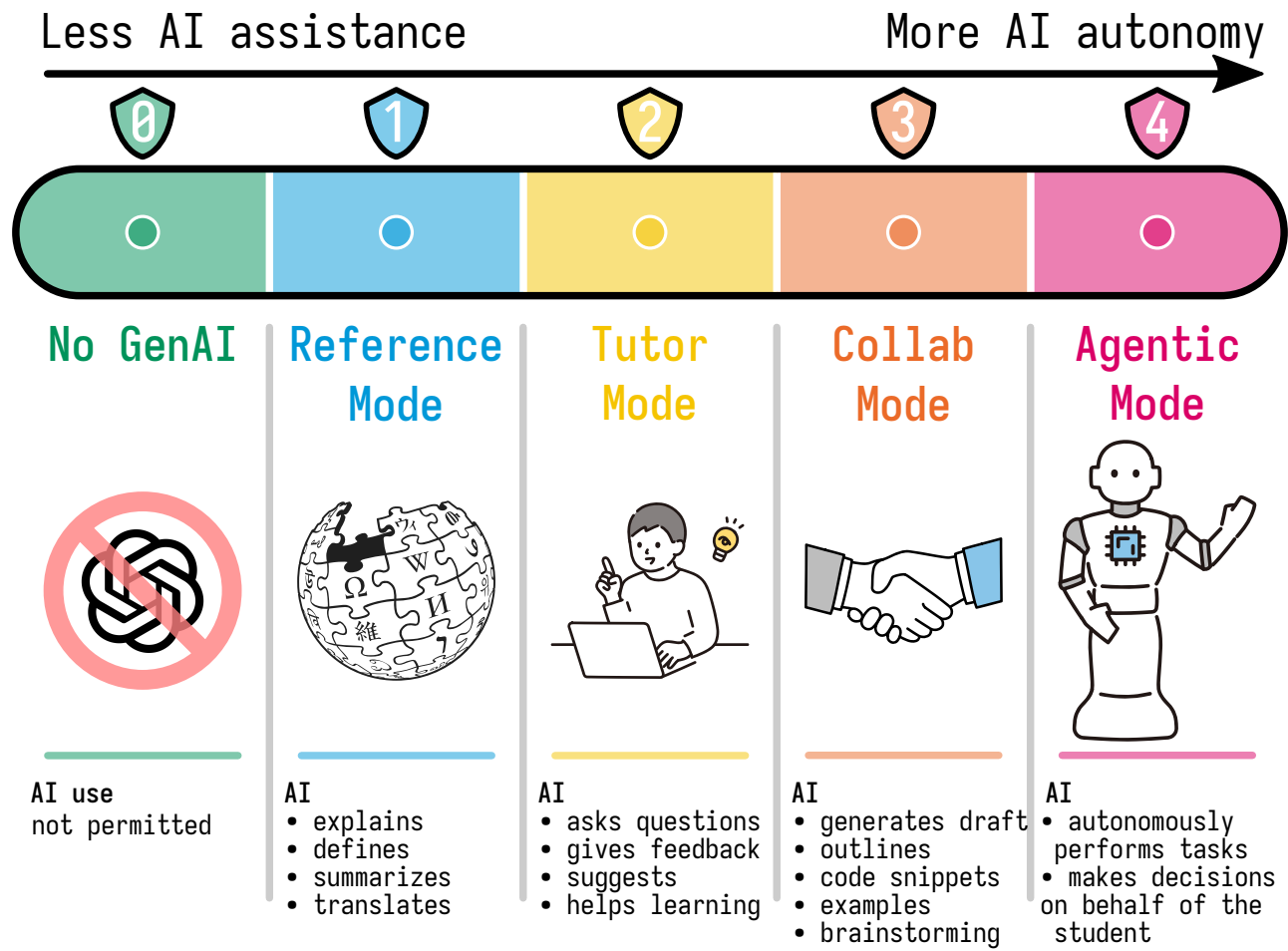
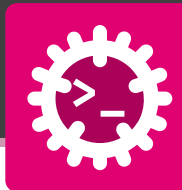


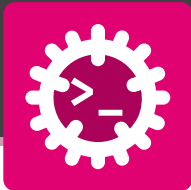
Conventional Commits

Format: `<type>(<scope>): <description>`

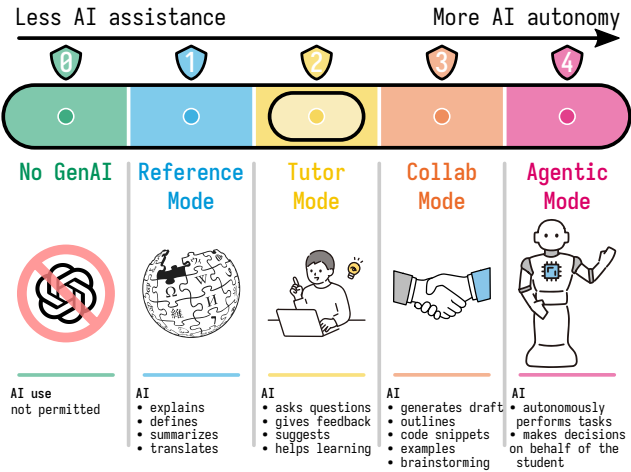
```
feat(ex02): implement temperature converter
fix(ex03): correct overflow in binary inspector
test(ex04): add unit tests for distance converter
refactor(labo1): extract knight move validation
docs(project): write README with setup instructions
chore(project): add justfile build recipe
```

Type	When to use
feat	new feature or behaviour
fix	bug fix
docs	documentation only
style	formatting, no logic change
refactor	improve code without changing behaviour
test	add or update tests
chore	build, CI, tooling
perf	performance improvement
revert	undo a previous commit

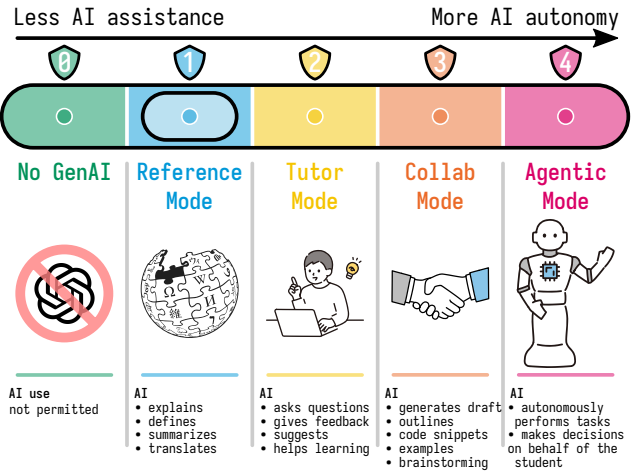




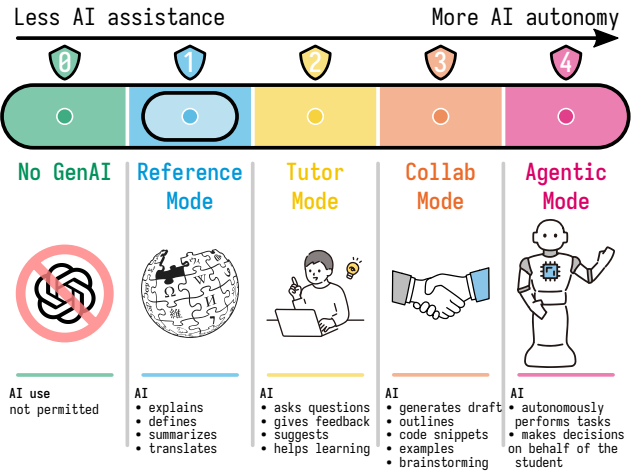
Course



Exercises & Labo



Project





Dedicated **AI prompt** for each level are provided (Course / Exercises & Labo / Project).

1. **Use the given prompt** - it is **mandatory** for **all** AI interactions in this course.
2. **Log every interaction** in LOG.md in your **Labs repo**.
3. **AI helps you learn** - it does **not** do the work for you.
4. **Ask for explanations** - not hints or finished solutions.



Exercises, labs and project are **automatically checked** for AI-generated content.

Any detected AI usage that does **not follow these guidelines** will be **penalized**.



Zahno Silvan (ZaS)

🏢 ENG.23.N312

✉ silvan.zahno@hevs.ch

☎ +41 58 606 88 07

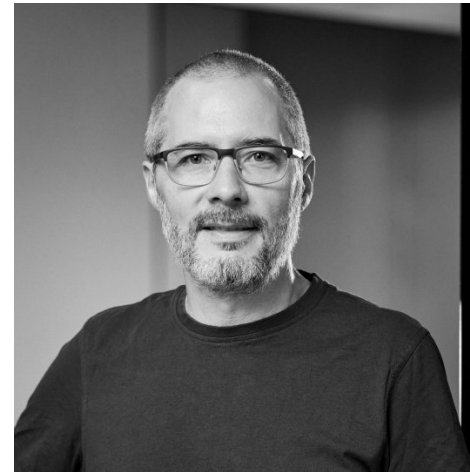


Roudy Dagher (DaY)

🏢 ENG.23.N309

✉ roudy.dagher@hevs.ch

☎ +41 58 606 95 33

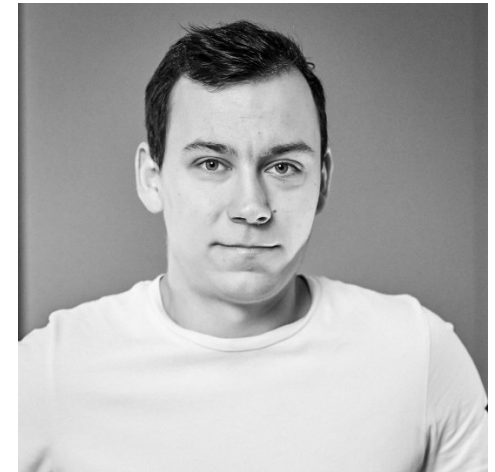


Thomas Sterren (STi)

🏢 ENG.23.N306

✉ thomas.sterren@hevs.ch

☎ +41 79 578 87 25



Rémy Borgeat (BoY)

🏢 ENG.23.N313

✉ remy.borgeat@hevs.ch

☎ +41 79 578 92 20



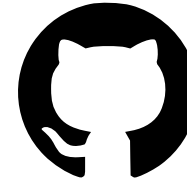
Teams



Name: [SYND-SwD](#)

Code: [efjvezh](#)

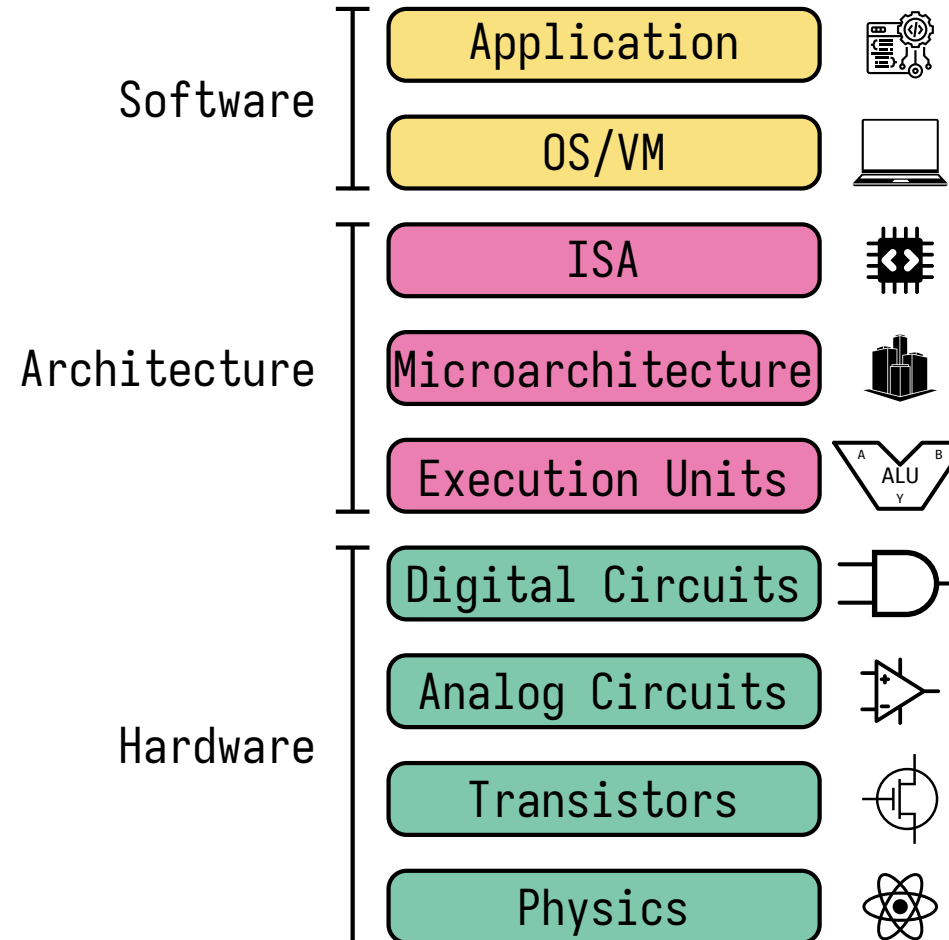
Github



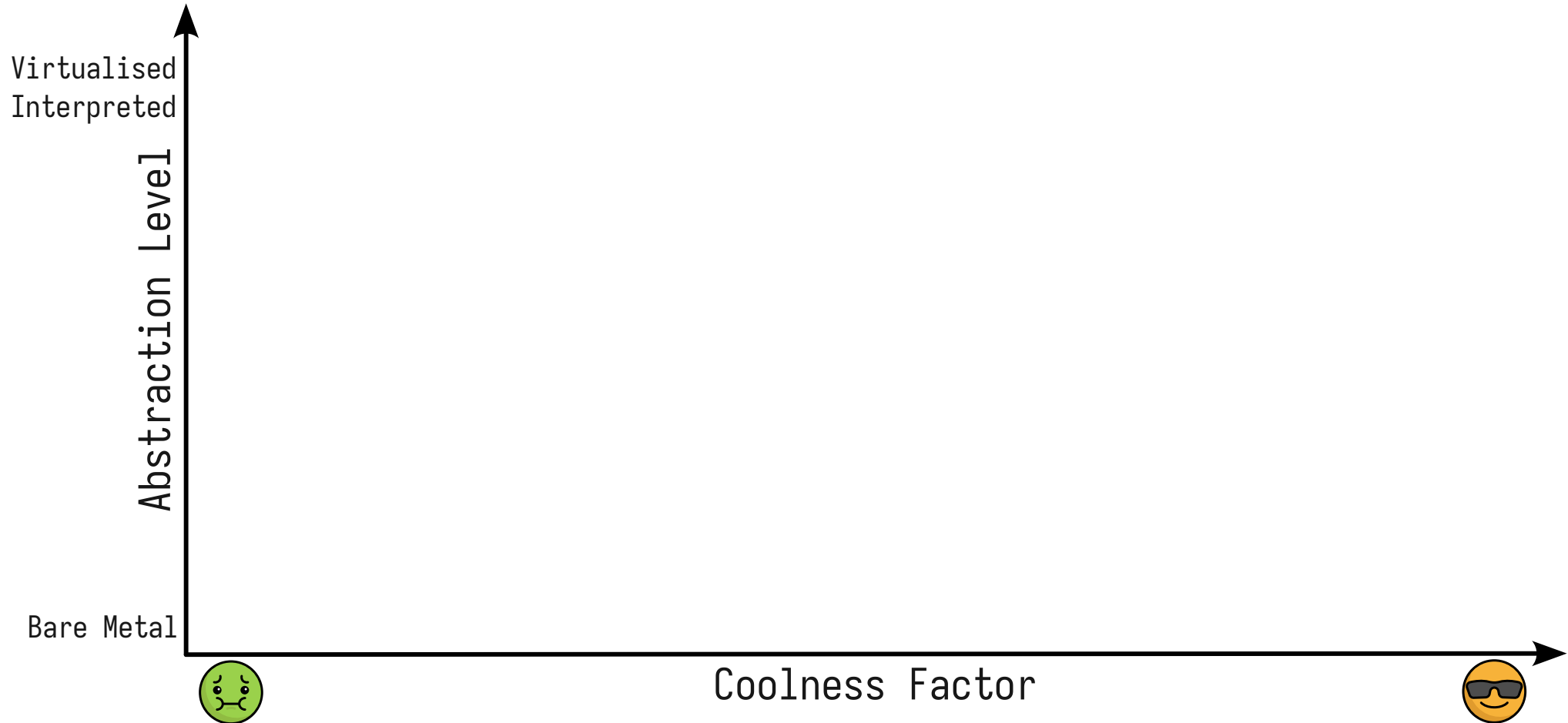
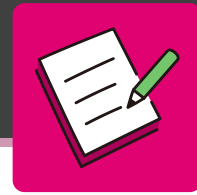
Organization for course documents:
[github hei-synd-swd](#)

Organization for course projects/labs/exercises:
[github hei-synd-swd-stud](#)

Where in the Computer Abstraction are we located?



What programming languages do you know?

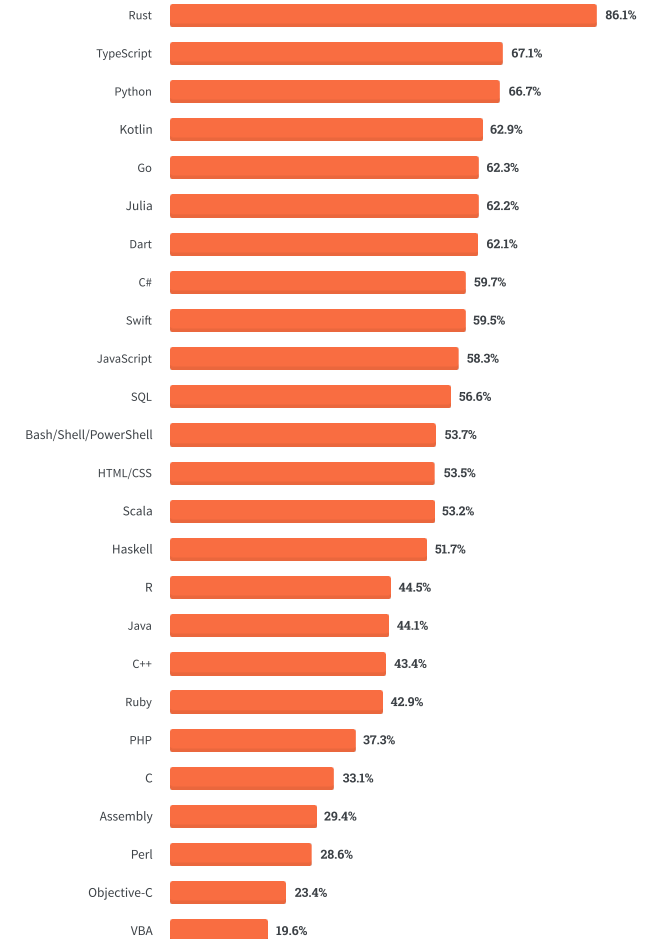


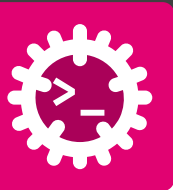
Why Rust?



Most admired language by developers since 2016 according to the [StackOverflow Developer Survey](#)

It is used in the [Linux Kernel](#) for graphics drivers, at Microsoft for the [Windows Kernel](#), at Amazon for [Amazon Web Services \(AWS\)](#) for backend services, at Google for [Android](#) and at [Cloudflare](#) for their edge network.



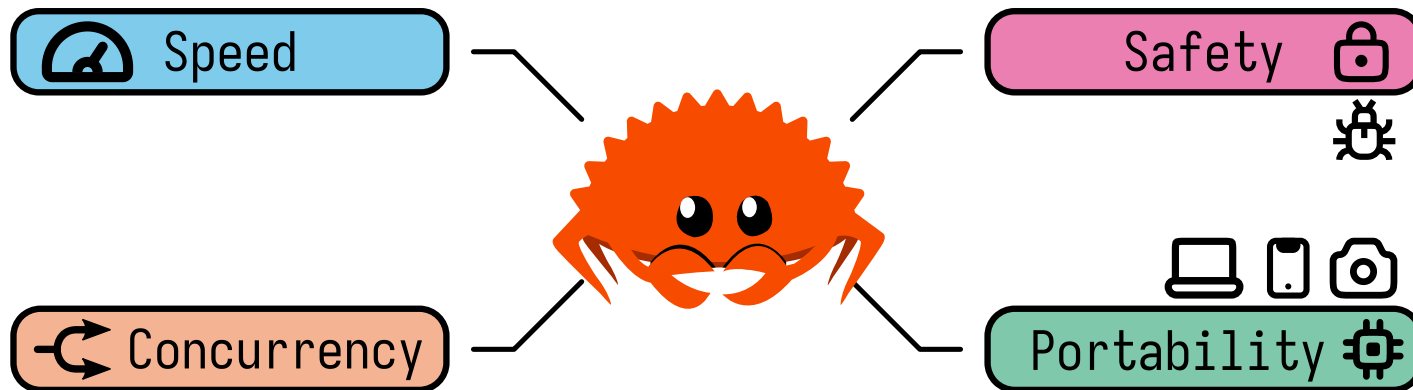


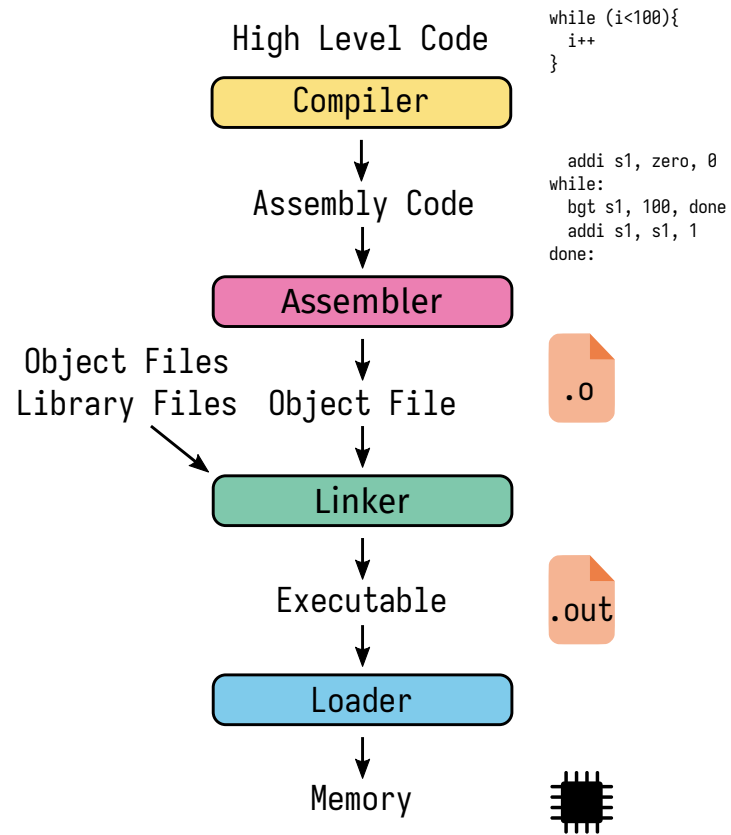
Energy efficient, fast and memory efficient.

Total					
	Energy		Time		Mb
(c) C	1.00	(c) C	1.00	(c) Pascal	1.00
(c) Rust	1.03	(c) Rust	1.04	(c) Go	1.05
(c) C++	1.34	(c) C++	1.56	(c) C	1.17
(c) Ada	1.70	(c) Ada	1.85	(c) Fortran	1.24
(v) Java	1.98	(v) Java	1.89	(c) C++	1.34
(c) Pascal	2.14	(c) Chapel	2.14	(c) Ada	1.47
(c) Chapel	2.18	(c) Go	2.83	(c) Rust	1.54
(v) Lisp	2.27	(c) Pascal	3.02	(v) Lisp	1.92
(c) Ocaml	2.40	(c) Ocaml	3.09	(c) Haskell	2.45
(c) Fortran	2.52	(v) C#	3.14	(i) PHP	2.57
(c) Swift	2.79	(v) Lisp	3.40	(c) Swift	2.71
(c) Haskell	3.10	(c) Haskell	3.55	(i) Python	2.80
(v) C#	3.14	(c) Swift	4.20	(c) Ocaml	2.82
(c) Go	3.23	(c) Fortran	4.20	(v) C#	2.85
(i) Dart	3.83	(v) F#	6.30	(i) Hack	3.34
(v) F#	4.13	(i) JavaScript	6.52	(v) Racket	3.52
(i) JavaScript	4.45	(i) Dart	6.67	(i) Ruby	3.97
(v) Racket	7.91	(v) Racket	11.27	(c) Chapel	4.00
(i) TypeScript	21.50	(i) Hack	26.99	(v) F#	4.25
(i) Hack	24.02	(i) PHP	27.64	(i) JavaScript	4.59
(i) PHP	29.30	(v) Erlang	36.71	(i) TypeScript	4.69
(v) Erlang	42.23	(i) Jruby	43.44	(v) Java	6.01
(i) Lua	45.98	(i) TypeScript	46.20	(i) Perl	6.62
(i) Jruby	46.54	(i) Ruby	59.34	(i) Lua	6.72
(i) Ruby	69.91	(i) Perl	65.79	(v) Erlang	7.20
(i) Python	75.88	(i) Python	71.90	(i) Dart	8.64
(i) Perl	79.58	(i) Lua	82.91	(i) Jruby	19.84



Rust Concepts





```
$ rustc -o hello -C opt-level=3 main.rs
```

Bash



Ariane 5 Flight 501 failure 1997

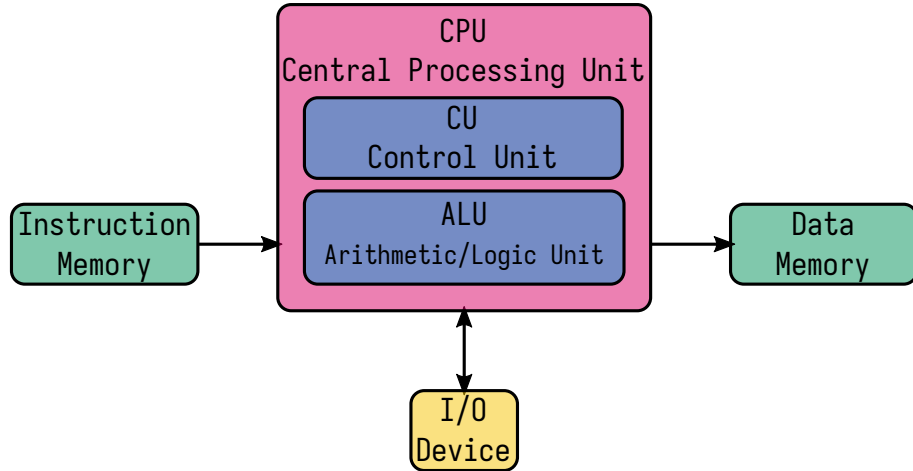


Data conversion from 64-bit floating point to 16-bit signed integer value

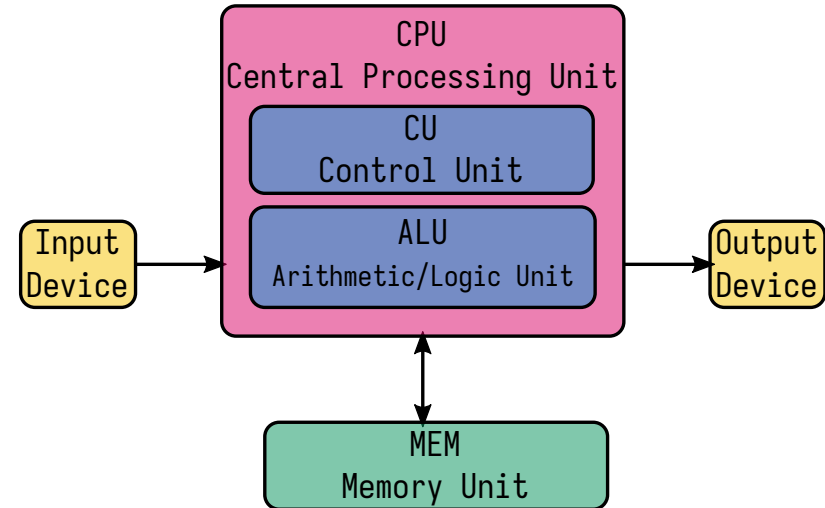
Where does your code run?



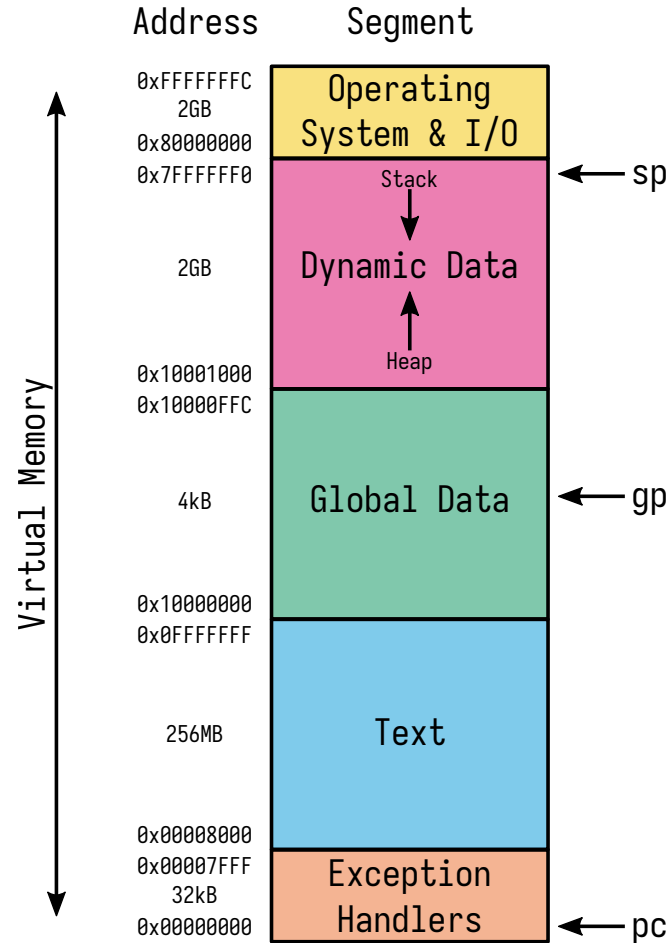
Harvard Architecture



Von Neumann Architecture



What memory is your program using?





Both live in **Random Access Memory (RAM)**, but are managed differently.

The **Stack** holds each function's **local variables**: it grows when a function is **called** and shrinks when it **returns**, so its memory is reserved and freed **automatically**.

The **Heap** is a large pool for values whose **size or lifetime** is only known at **runtime**, allocated and released **on demand**.

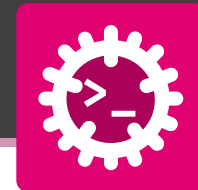
Stack

- ✓ Fast, predictable (just move a pointer)
- ✓ Freed **automatically** at scope end (**Resource Acquisition Is Initialization (RAII)**)
- ✓ No fragmentation
- ✗ Fixed size, reserved **up front**
- ✗ Overflow **crashes** the program

Heap

- ✓ Flexible - **dynamic** size and lifetime
- ✓ Allows **shared ownership**
- ✗ Slower allocation at runtime
- ✗ Manual / managed cleanup - can **leak**
- ✗ **Fragmentation** and out-of-memory over time

Heap Vs. Stack memory allocation?



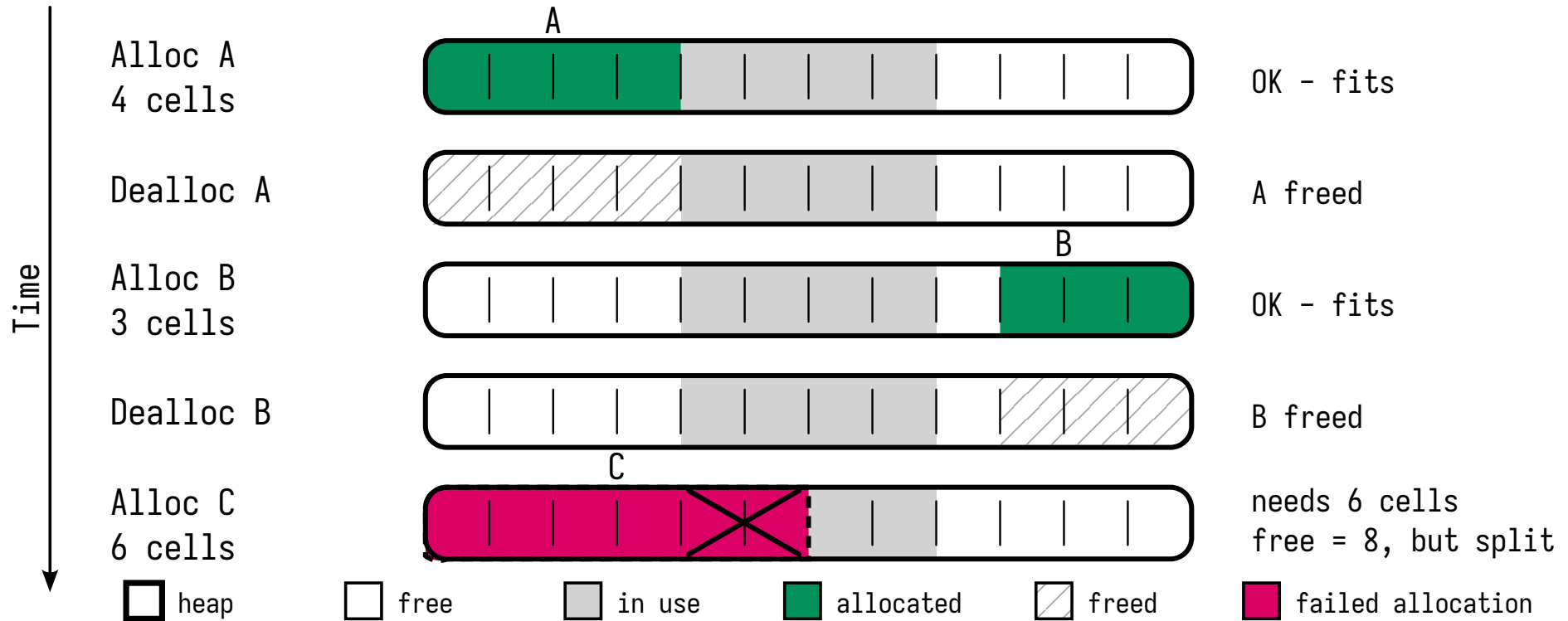
Typical sizes differ by **orders of magnitude** across platforms:

Platform	RAM	Stack (default)	Heap
Desktop / Laptop	8-32 GB	8 MB / thread	up to RAM (GBs)
Raspberry Pi	1-8 GB	8 MB / thread	up to RAM (GBs)
STM32 (Cortex-M)	20 KB SRAM	1-4 KB	0 - a few KB



On a small **STM32**, **Stack** and **Heap** share the **same** 20 KB - many embedded programs use **no heap at all**.

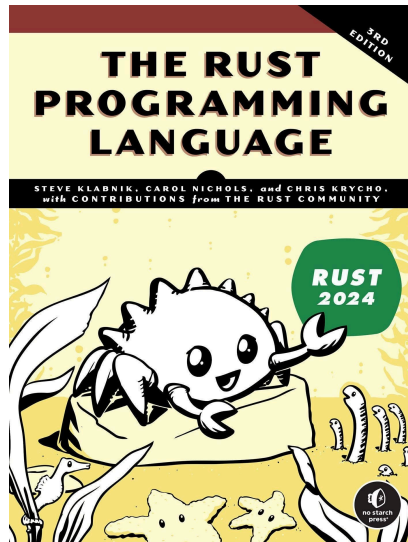
Memory Fragmentation



Fragmentation: the free space is enough, but it is split into non-contiguous holes.



Many additional resources are available. Our library has a good collection of books about programming, software engineering and Rust. Here are some of the most relevant ones for this course:



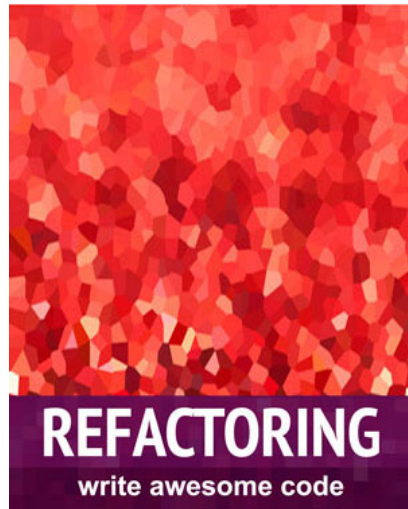
[1]



[2]



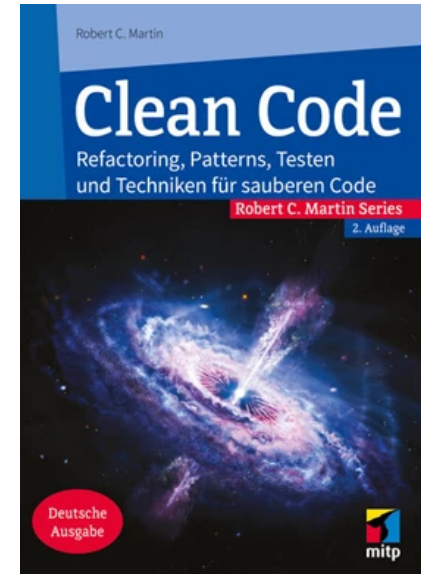
[3]



[4]



[4]



[5]

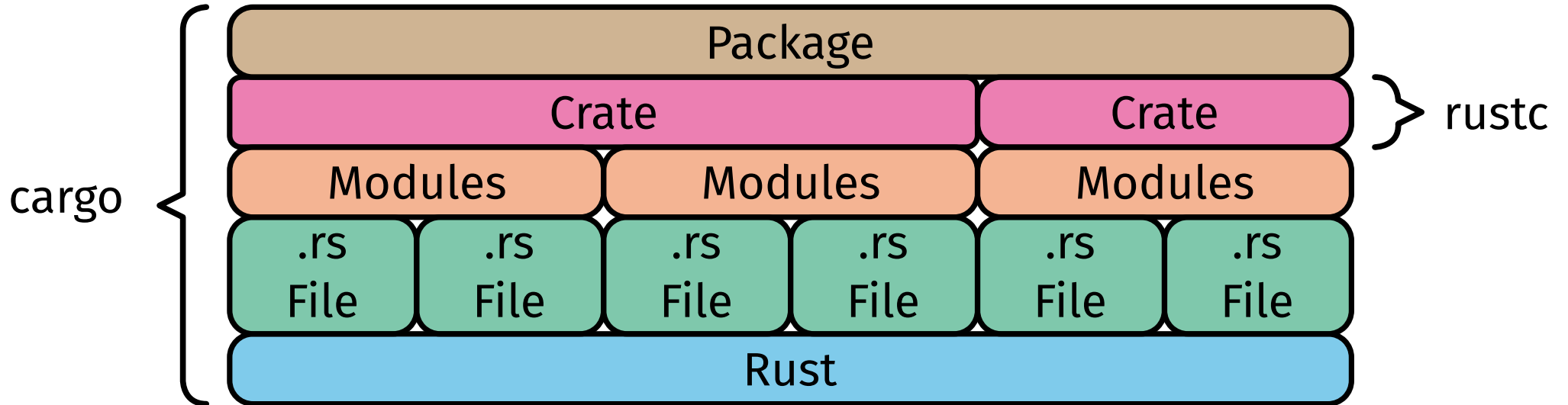


Rust Project and Toolchain

Getting started with Rust projects and toolchain



Package $\Rightarrow$ Crate $\Rightarrow$ Module $\Rightarrow$ Source File





Binary crate

Compiles to an **executable**. Must have a `fn main()` entry point.

```
my_app/
├── Cargo.toml
└── src/
    ├── main.rs    ← entry point
    └── bin/
        └── tool.rs ← extra binary
```

```
cargo new my_app
cargo run
cargo run --bin tool
```

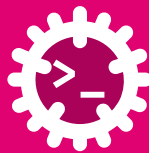
Library crate

Reusable crate (`lib.rs`). No `main()`.

```
my_lib/
├── Cargo.toml
└── src/
    └── lib.rs    ← crate root
```

```
cargo new --lib my_lib
cargo build
cargo test
```

```
# in consumer's Cargo.toml
[dependencies]
my_lib = { path = "../my_lib" }
```



Create and run

```
$ cargo new hello_world
$ cd hello_world
$ cargo run
```

```
hello_world/
├── Cargo.toml
└── src/
    └── main.rs
```

Cargo.toml

```
[package]
name     = "hello_world"
version  = "0.1.0"
edition = "2024"
```

src/main.rs

```
fn main() {
    println!("Hello, world!");
}
```

Output

```
Compiling hello_world v0.1.0
Finished dev in 0.42s
Running `target/debug/hello_world`
Hello, world!
```



A single package can expose both a library (src/lib.rs) and one or more binaries (src/main.rs) at the same time.

```
my_project/
├── Cargo.toml
├── Cargo.lock
└── src/
    ├── main.rs
    ├── lib.rs
    ├── config.rs
    └── network/
        ├── mod.rs
        └── client.rs
```

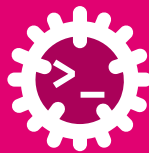
Cargo.toml

```
[package]
name      = "network-project"
version   = "0.1.0"
edition   = "2024"

[dependencies]
serde     = { version = "1", features = ["derive"] }
log       = "0.4"
request   = "0.12"
```

src/lib.rs - declare modules

```
pub mod config;
pub mod network;
```



src/network/mod.rs

```
pub mod client;
```

src/network/client.rs

```
use serde::Deserialize;    // external crate
use crate::config::Config; // internal module
#[derive(Deserialize)]
pub struct Response { pub status: u32 }
pub fn fetch(cfg: &Config) → Response { todo!() }
```

src/main.rs

```
use my_project::network::client;
use my_project::config::Config;

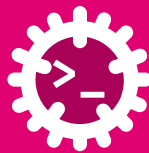
fn main() {
    let cfg = Config::load();
    let res = client::fetch(&cfg);
}
```



The Rust toolchain is a set of tools to write, compile, and manage Rust projects:

- **rustup** - toolchain installer and version manager
- **rustc** - the Rust compiler
- **cargo** - build system and package manager
- **rustdoc** - documentation generator
- **rustfmt** - code formatter
- **clippy** - linter





rustup installs and manages Rust toolchain versions.

Install

```
curl --proto '=https' --tlsv1.2 \
  -sSf https://sh.rustup.rs | sh
```

Common commands

```
rustup update           # update toolchain
rustup show             # active toolchain
rustup target add ...   # cross-
  compilation
rustup component add rustfmt clippy # add components
```

Channels

- **stable** - recommended for production
- **beta** - next stable release candidate
- **nightly** - latest features, may be unstable

```
rustup toolchain install nightly
rustup default stable
```

rust-toolchain.toml

Pin the toolchain for a project:

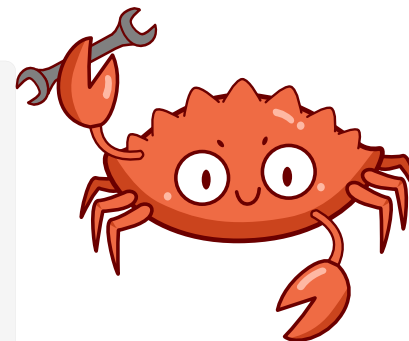
```
[toolchain]
channel = "stable"
components = ["rustfmt", "clippy"]
```



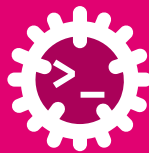

rustc is the Rust compiler. It takes .rs source files and produces a native binary or library.

Usage

```
rustc main.rs                # compile
rustc -o hello main.rs       # named output
rustc --edition 2024 main.rs # compile for a given edition
rustc --explain E0308        # explain error
rustc -o hello -C opt-level=3 main.rs # compile with optimization
rustc -h                     # see help
```



In practice, cargo invokes rustc automatically.

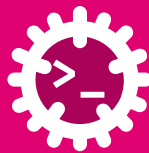


cargo is the Rust build system and package manager.

Most used commands

```
cargo new my_project    # create new project
cargo build              # compile (debug)
cargo build --release    # compile (optimized)
cargo run                # build and run
cargo test               # run tests
cargo check              # type-check, no binary
cargo fmt                # format code
cargo clippy              # run linter
cargo doc --open         # build and open docs
cargo add serde           # add a dependency
cargo update             # update dependencies
```





Every Cargo project has a Cargo.toml manifest at its root:

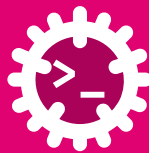
```
[package]
name      = "my_project"
version   = "0.1.0"
edition   = "2024"
authors   = ["Jane Doe <jane@example.com>"]
description = "A sample Cargo project"
license   = "MIT"

[dependencies]
serde = { version = "1", features = ["derive"] }
rand  = "0.8"
hyper = { git = "https://github.com/hyperium/hyper" }

[dev-dependencies]
pretty_assertions = "1"

[profile.release]
opt-level = 3
```

- [package] - metadata (name, version, edition)
- [dependencies] - runtime crates from crates.io or repo links
- [dev-dependencies] - only used for tests/benchmarks
- [profile.*] - compiler flags per build profile



Binary (src/main.rs)

Produces an executable. Entry point is `fn main()`.

```
cargo new my_app
cargo run
```

```
fn main() {
    println!("Hello, world!");
}
```

A project can have **multiple** binaries in `src/bin/`.

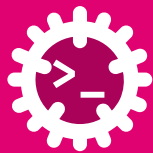
Library (src/lib.rs)

Produces a reusable crate. No `main()`. Other projects depend on it via `Cargo.toml`.

```
cargo new --lib my_lib
cargo build
```

```
pub fn add(a: i32, b: i32) → i32 {
    a + b
}
```

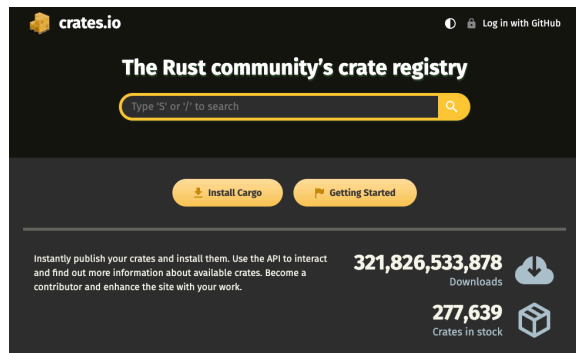
A project can be **both** (lib + bin) at the same time.



<https://crates.io>

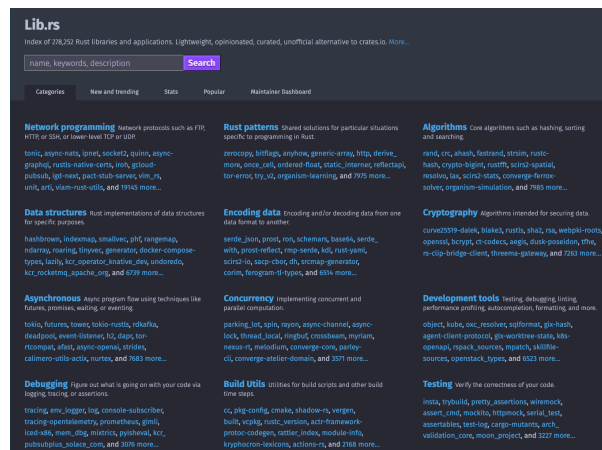


The **official registry** for Rust crates. Publish, search, and manage dependencies. Every crate on crates.io is available for anyone to download and use in their projects.



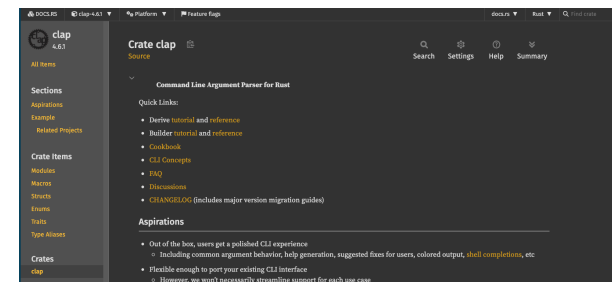
<https://libs.rs>

Curated, searchable index of crates with quality scores, recent activity, and dependency graphs. Good for **discovering** well-maintained libraries.



<https://docs.rs>

Automatically generated **Application Programming Interface (API) documentation** for every crate published to crates.io. Built by rustdoc on each release.





Imperative Programming

A hands-on, step-by-step approach to coding



During lectures or for small experiments use the [Rust Playground](#) - an online editor with instant compilation and sharing.

```
1 fn main() {  
2  
3     println!("Hello, world!");  
4  
5 }
```

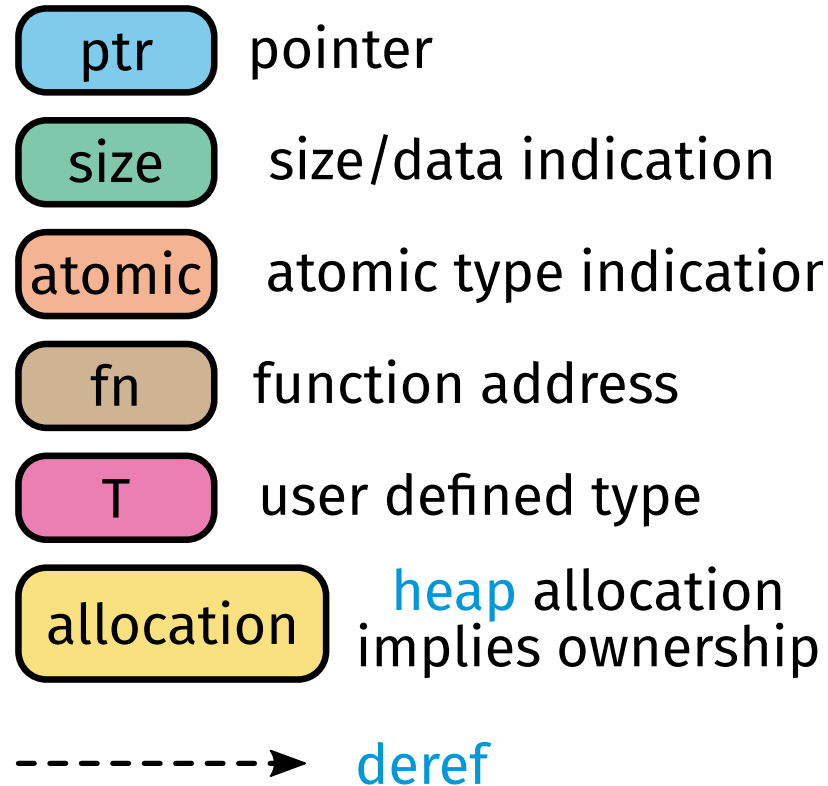
Execution

Standard Error

```
Compiling playground v0.0.1 (/playground)  
Finished `dev` profile [unoptimized + debuginfo] target(s) in 1.38s  
Running `target/debug/playground`
```

Standard Output

```
Hello, world!
```



Rust uses different naming styles for different language elements. The compiler accepts many names, but `rustfmt` and `clippy` expect the common Rust style.

Element	Convention	Example
Variable	snake_case	<code>sensor_value</code>
Function	snake_case	<code>read_sensor()</code>
Module	snake_case	<code>sensor_bus</code>
Constant	SCREAMING_SNAKE_CASE	<code>MAX_RETRIES</code>
Static	SCREAMING_SNAKE_CASE	<code>APP_NAME</code>
Type	UpperCamelCase	<code>SensorValue</code>
Struct	UpperCamelCase	<code>TemperatureSensor</code>
Enum	UpperCamelCase	<code>SensorState</code>
Enum variant	UpperCamelCase	<code>OutOfRange</code>
Trait	UpperCamelCase	<code>Readable</code>
Generic type	UpperCamelCase	<code>T</code> , <code>Value</code>

Names are part of the program design.

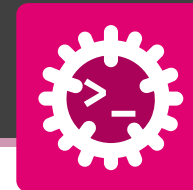


- Good names reduce comments.
- Names should communicate intent.



Constants

Values known at compile time



Definition

A **constant** is a name permanently bound to a **compile-time value**.

- Declared with `const` keyword **anywhere**
- Type annotation is **mandatory**
- Name uses SCREAMING_SNAKE_CASE
- Valid for the **entire** program lifetime
- **Inlined** by the compiler - zero runtime cost

`const` vs `let`

<code>const</code>	<code>let</code>
Compile-time value	Runtime binding
Always immutable	Immutable by default
No memory address	Lives on the Stack
Any scope	Block scope only

```
// Type annotation is required
const MAX_SCORE: u32 = 100_000;
const GRAVITY: f64 = 9.81;
const APP_NAME: &str = "RustApp";
const MAX_RETRIES: u8 = 3;

// Computed constant expression
const SECS_PER_DAY: u64 = 60 * 60 * 24;

fn main() {
    println!("{APP_NAME}");
    println!("max score: {MAX_SCORE}");
    println!("seconds/day: {SECS_PER_DAY}");

    // Constants can be used in other constants
    const DAYS_PER_YEAR: u64 = 365;
    const SECS_PER_YEAR: u64 = SECS_PER_DAY * DAYS_PER_YEAR

    println!("seconds/year: {SECS_PER_YEAR}");
}
```



Min and Max values

Every integer type exposes MIN, MAX and BITS as associated constants:

```
fn main() {
    println!("u8      : {}..{}", u8::BITS, u8::MIN, u8::MAX);
    println!("u16     : {}..{}", u16::BITS, u16::MIN, u16::MAX);
    println!("u32     : {}..{}", u32::BITS, u32::MIN, u32::MAX);
    println!("u64     : {}..{}", u64::BITS, u64::MIN, u64::MAX);
    println!("usize   : {}..{}", usize::MIN, usize::MAX);
    println!("i8      : {}..{}", i8::BITS, i8::MIN, i8::MAX);
    println!("i16     : {}..{}", i16::BITS, i16::MIN, i16::MAX);
    println!("i32     : {}..{}", i32::BITS, i32::MIN, i32::MAX);
    println!("i64     : {}..{}", i64::BITS, i64::MIN, i64::MAX);
    println!("isize   : {}..{}", isize::MIN, isize::MAX);
}
```

Type	Min	Max	Bits
u8	0	255	8
u16	0	65 535	16
u32	0	4.3×10^9	32
u64	0	1.8×10^{19}	64
u128	0	$\approx 3.4 \times 10^{38}$	128
usize	0	platform	32/64
i8	-128	127	8
i16	-32 768	32 767	16
i32	-2.1×10^9	2.1×10^9	32
i64	-9.2×10^{18}	9.2×10^{18}	64
i128	$\approx -1.7 \times 10^{38}$	$\approx 1.7 \times 10^{38}$	128
isize	platform	platform	32/64



Mathematical constants - `std::f64::consts`:

π , e , $\sqrt{2}$, $\ln(10)$, etc.

```
use std::f64::consts;

fn main() {
    println!("π          = {:.10}", consts::PI);
    println!("τ = 2π     = {:.10}", consts::TAU);
    println!("e          = {:.10}", consts::E);
    println!("√2         = {:.10}", consts::SQRT_2);
    println!("ln 2       = {:.10}", consts::LN_2);
    println!("ln 10      = {:.10}", consts::LN_10);
    println!("1/π        = {:.10}", consts::FRAC_1_PI);
    println!("1/√2       = {:.10}", consts::FRAC_1_SQRT_2);
    println!("log10(2)   = {:.10}", consts::LOG10_2);
}
```

```
π          = 3.1415926536
τ = 2π     = 6.2831853072
e          = 2.7182818285
√2         = 1.4142135624
ln 2       = 0.6931471806
ln 10      = 2.3025850930
1/π        = 0.3183098862
1/√2       = 0.7071067812
log10(2)   = 0.3010299957
```

```
use std::f64::consts;

fn main() {
    let radius = 4.234;
    let diameter = 2.0 * radius;
    let area = consts::PI * radius * radius; // πr²
    println!("r={radius} d={diameter} area={area:.3}");
}
```



```
use std::f64::consts;
fn main() {
    let inf      = f64::INFINITY;
    let neg_inf  = f64::NEG_INFINITY;
    let nan      = f64::NaN;

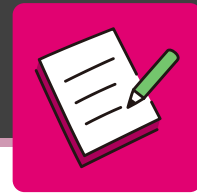
    println!(" ∞  = {inf}");           // inf
    println!("-∞  = {neg_inf}");       // -inf
    println!("NaN = {nan}");           // NaN

    // Arithmetic with special values
    println!("{}", 1.0_f64 / 0.0);     // inf
    println!("{}", -1.0_f64 / 0.0);   // -inf
    println!("{}", 0.0_f64 / 0.0);    // NaN

    // NaN is never equal - even to itself!
    println!("{}", f64::is_nan(f64::NaN)); // true
    println!("{}", f64::NaN.is_nan());     // true
    println!("{}", inf.is_infinite());     // true
    println!("{}", 42.0_f64.is_finite());  // true
}
```

∞ , $-\infty$, NaN

```
∞    = inf
-∞   = -inf
NaN  = NaN
inf
-inf
NaN
true
true
true
true
```



Using **only** `const` declarations (no `let` bindings), build a tiny time budget:

1. Declare the base constants `SECS_PER_MINUTE`, `MINS_PER_HOUR`, `HOURS_PER_DAY` and `DAYS_PER_YEAR`.
2. Derive `SECS_PER_YEAR` as a **computed constant expression** from the constants above.
3. In `main`, print `SECS_PER_YEAR` and report whether it fits in a `u16` and in a `u32`, using the associated boundary constants `u16::MAX` and `u32::MAX`.



Use `SCREAMING_SNAKE_CASE` names and pick an integer type wide enough to hold the result.



Notations (Literals)

Expressing values directly in source code



Decimal, Hex, Octal, Binary, Byte

Supports five **numeric bases** for integer literals

```
fn main() {
    let decimal = 98_222;      // base 10
    let hex     = 0xFF;        // base 16, prefix 0x
    let octal   = 0o77;        // base 8,  prefix 0o
    let binary  = 0b1111_0000; // base 2,  prefix 0b
    let byte: u8 = b'A';       // ASCII byte literal

    println!("decimal : {decimal}"); // 98222
    println!("hex      : {hex}");    // 255
    println!("octal    : {octal}");  // 63
    println!("binary   : {binary}"); // 240
    println!("byte     : {byte}");   // 65

    // All represent the same value
    assert_eq!(0xFF_u8, 0b1111_1111_u8);
    assert_eq!(0o377_u16, 255_u16);
}
```

Number separator _ improves readability:

```
let million: u64 = 1_000_000;
let pi_approx: f64 = 3.141_592_653;
let flags: u8 = 0b0000_1111;
let ip_mask: u32 = 0xFF_FF_00_00;
```

Type suffixes make the type explicit:

```
let a: u8 = 57;      // unsigned 8-bit
let b: i32 = -100;    // signed 32-bit
let c: f32 = 1.5;     // 32-bit float
let d: i64 = 1_000;    // large signed integer
let e      = 57_i16;  // explicit i16 suffix
```

Default types when no suffix is given: integer i32,
float f64, byte u8



Variables

Bindings: immutable by default



In Rust, bindings are **immutable by default**. The compiler **enforces** this at compile time:

```
fn main() {  
    let x = 5;  
    println!("x = {x}");    // x = 5  
    // This does NOT compile:  
    x = 6;  
}
```

Why immutable by default?

- Prevents accidental mutation bugs
- Enables safe concurrent access
- Communicates intent explicitly
- Allows aggressive compiler optimizations
- Forces deliberate design choices

```
error[E0384]: cannot assign twice to  
             immutable variable `x`  
  --> src/main.rs:4:5  
2 |         let x = 5;  
  |         -  
  |         |  
  |         first assignment to `x`  
  |         help: consider making this binding mutable: `mut x`  
4 |         x = 6;  
  |         ^^^^^ cannot assign twice to immutable variable
```



The compiler is your **friend** - it gives clear error messages and suggestions to fix issues.



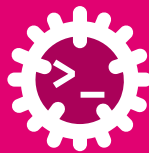
Add mut to allow reassignment. But **start immutable, add mut only when needed.**

```
struct Sensor { value: f64, unit: &'static str }
fn main() {
    // Mutable variable - can be reassigned
    let mut count = 0_u32;
    count += 1;
    println!("count = {count}"); // count = 1
    // Mutable bindings can also hold complex types
    let mut name = String::from("Alice");
    name.push_str(" Smith");
    println!("{name}");           // Alice Smith
    // Mutable structs: `mut` applies to all fields
    let mut s = Sensor { value: 20.0, unit: "°C" };
    // Simulate readings
    s.value += 3.5;
    s.value -= 1.2;
    println!("{:.1} {}", s.value, s.unit); // 22.3 °C
}
```

Real-world example - accumulator loop:

```
fn main() {
    let readings = [17.3, 19.1];
    let mut sum = 0.0_f64;
    for &r in &readings {
        sum += r;
        println!("reading: {r:.1}, sum: {sum:.1}");
    }
    let avg = sum / readings.len() as f64;
    println!("average: {avg:.1}");
}
```

```
reading: 17.3, sum: 17.3
reading: 19.1, sum: 36.4
average: 18.2
```



Rust infers types from **context** - no annotation needed when unambiguous:

Basic inference from value:

```
fn main() {  
    let a = 42;           // i32   (integer default)  
    let b = 3.14;         // f64   (float default)  
    let c = true;         // bool  
    let d = 'z';          // char  
    let e = "hello";      // &str  
    let f = 42_u8;        // u8  
    let g = 42_i128;      // i128  
    // Inferred from another binding  
    let x = 100;  
    let y = x;            // i32 - same as x  
    // Inferred from arithmetic context  
    let sum = a + y;      // i32  
    println!("{a} {b} {c} {d} {e} {f} {g} {sum}");  
}
```

Same code with explicit type annotations:

```
fn main() {  
    let a: i32 = 42;  
    let b: f64 = 3.14;  
    let c: bool = true;  
    let d: char = 'z';  
    let e: &str = "hello";  
    let f: u8 = 42;  
    let g: i128 = 42;  
    let x: i32 = 100;  
    let y: i32 = x;  
    let sum: i32 = a + y;  
    println!("{a} {b} {c} {d} {e} {f} {g} {sum}");  
}
```



Shadowing lets you reuse a name with a **new binding** - even with a different type:

```
fn main() {  
    let x = 5;           // x: i32 = 5  
    let x = x + 1;       // x: i32 = 6 (new binding)  
    {  
        let x = x * 2;   // x: i32 = 12 (inner scope)  
        println!("inner x = {x}"); // 12  
    }  
    println!("outer x = {x}"); // 6 (outer still 6)  
    // Shadow with a completely different TYPE  
    let spaces = " ";    // &str  
    let spaces = spaces.len(); // usize = 3  
    println!("spaces = {spaces}");  
}
```

Shadowing vs mut:

```
// With mut: same type, reassignable  
let mut y = 5;  
y = y + 1;    // ok - same type i32  
// y = "six"; // ERROR: type mismatch  
  
// With shadowing: can change type!  
let z = 5;  
let z = z + 1; // new binding, still i32  
let z = "six"; // new binding, now &str  
println!("{z}");
```



What prints the program out?



Two ways to say “I don’t need this” - keep the value but silence the warning, or throw it away:

```
fn main() {  
    // _name: BOUND but unused → no warning  
    let _guard = setup(); // kept alive, name silenced  
    let count = 5;        // ⚠ warning: unused  
    `count`  
    let _count = 5;        // no warning  
    // bare _ : NOT bound → value thrown away  
    let _ = returns_result(); // ignore the Result  
    let (x, _) = (1, 2);      // keep x=1, drop the 2  
    for _ in 0..3 {           // loop 3x  
        println!("tick");    // index not used  
    }  
}
```

`_name` **vs** `_` - the key difference:

- `_name` - a **real binding**; the value **lives** to end of scope, the leading `_` only mutes the “unused” warning
- `_` - **not a binding**; the value is **dropped immediately**, nothing is named
- So `let _g = guard();` **holds** the guard; `let _ = guard();` drops it **now**
- `_` also works in patterns: `match`, `if let`, tuples, closures - “match anything, bind nothing”



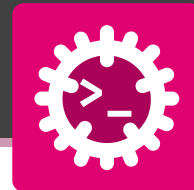
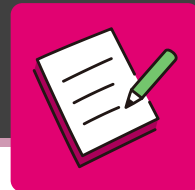
When does the Guard get dropped: with `let _g =` versus `let _ =`?



Lifetime is a compile-time concept that tracks **how long references are valid**. It ensures memory safety without a garbage collector.

In the example below, the inner `x` shadows the outer `x`, but they have different lifetimes - the inner `x` only exists within the inner block and is dropped when the block ends. The outer `x` remains unaffected and valid throughout the entire `main` function.

```
fn main() {  
    let x = 5;           // -----+--- b'  
    let x = x + 1;       //      |  
    {                   //      |  
        let x = x * 2;   // --+--- a'  
        println!("inner x = {x}"); // |  
    }                   // --+--- a' ends, inner x dropped |  
    println!("outer x = {x}"); //      |  
}                       // -----+--- b' ends, outer x dropped
```



Write a small `main` that exercises bindings and shadowing:

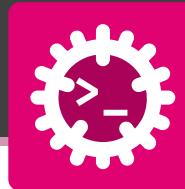
1. Declare an immutable `let level = 1`, then explain in one comment why `level = 2`; would **not compile**
2. Print value of variable `level` with `println!("{level}")`
3. Declare a mutable `let mut score = 0`, then add 10 to it **twice**
4. Remove the warning: variable `score` is assigned to, but never used
5. Starting from the string `" 42 "`, use **shadowing** to reuse the same name: first trim it with `.trim()`, then parse it into an `i32` with `.parse().unwrap()`
6. Print the value of the `i32` variable

Lean on **type inference** and only annotate a type where the compiler needs it.



Primitive Types

Scalar and compound built-in types



Type	Min	Max	Typical use
u8	0	255	bytes, RGB, ASCII
u16	0	65 535	network ports
u32	0	4.3×10^9	file sizes, colours
u64	0	1.8×10^{19}	large counters, hashes
u128	0	$\approx 3.4e38$	UUIDs, checksums
usize	0	platform	lengths, indices
i8	-128	127	small deltas, sensor raw
i16	-32 768	32 767	audio samples
i32	-2.1×10^9	2.1×10^9	default integer
i64	-9.2×10^{18}	9.2×10^{18}	timestamps, large IDs
i128	$\approx -1.7e38$	$\approx 1.7e38$	cryptography, finance
isize	platform	platform	signed index offsets

`isize` / `usize` match the **pointer width** and are platform dependent (32 or 64 bit):

```
fn main() {  
    let v = vec![10, 20, 30, 40];  
    // len() returns usize  
    let len: usize = v.len();  
    println!("last: {}", v[len - 1]); // 40  
    // Signed arithmetic on indices  
    let offset: isize = -2;  
    let i = (len as isize + offset) as usize;  
    println!("v[{}] = {}", i, v[i]); // v[2] = 30  
    // Platform size  
    println!("usize = {} bits", usize::BITS);  
    // 64 on modern machines  
}
```

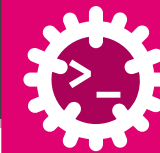


Type	Width	Precision	Range (abs)
f32	32 bit	7 decimal dig	1.2e-38 to 3.4e38
f64	64 bit	15 decimal dig	2.2e-308 to 1.8e308

```
fn main() {  
    // f64 is the default float type  
    let y: f64 = 3.14159265358979;  
  
    let area = std::f64::consts::PI * 5.0_f64.powi(2);  
    println!("area: {area:.4}");    // area: 78.5398  
  
    // Precision difference  
    let a: f32 = 0.1 + 0.2;  
    let b: f64 = 0.1 + 0.2;  
    println!("f32: {a:.20}"); // 0.30000001192092895508  
    println!("f64: {b:.20}"); // 0.30000000000000004441  
}
```

Common math methods:

```
let x = 2.0_f64;  
  
println!("{}", x.sqrt());    // 1.4142135623730951  
println!("{}", x.powi(10));  // 1024.0 (integer exp)  
println!("{}", x.powf(0.5)); // 1.4142135623730951  
println!("{}", x.ln());      // 0.6931471805599453  
println!("{}", x.log2());    // 1.0  
println!("{}", x.log10());   // 0.30102999566398114  
println!("{}", x.sin());     // 0.9092974268256817  
println!("{}", x.cos());     // -0.4161468365471424  
println!("{}", x.abs());     // 2.0  
println!("{}", (-3.7_f64).ceil()); // -3.0  
println!("{}", (-3.7_f64).floor()); // -4.0  
println!("{}", (-3.7_f64).round()); // -4.0  
println!("{}", (-3.7_f64).trunc()); // -3.0
```



bool - exactly 1 byte, values true or false:

```
fn main() {
    let t: bool = true;
    let f      = false; // type inferred

    // Produced by comparison operators
    let is_adult = 18 ≥ 18;           // true
    let is_zero  = 42 = 0;           // false
    let in_range = 5 < 10 && 10 < 20; // true

    // Used in control flow
    if is_adult {
        println!("welcome");
    }

    // Casting to integer
    println!("{}", true as u8);      // 1
    println!("{}", false as u8);     // 0
}
```

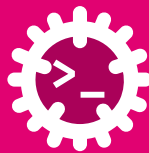
Unit type () - empty tuple, zero bytes, “no value”:

```
// Functions without → return ()
fn greet(name: &str) {
    println!("Hello, {name}!");
}

fn main() {
    let result: () = greet("Alice");
    println!("{:?}", result); // ()
}
```



?: is the debug formatter, which prints values in a human-readable form. For **()**, it simply prints **()**.



Tuple - ordered, fixed-size group of **heterogeneous** values:

```
fn main() {
    // Type annotation: (T1, T2, T3, ...)
    let point: (i32, i32)      = (10, 20);
    let color: (u8, u8, u8)    = (255, 128, 0); // RGB
    let mixed: (i32, f64, bool) = (42, 3.14, true);

    // Access by zero-based index (.0, .1, ...)
    println!("{}", point.0);    // 10
    println!("{}", color.2);    // 0
    println!("{}", mixed.1);    // 3.14

    // Tuples are Copy if all fields are Copy
    let a = point;
    let b = point; // both valid
    println!("{}", a.0, b.0); // 10 10
}
```

Destructuring - unpack a tuple into named variables:

```
fn main() {
    let point = (10, 20);
    let color = (255_u8, 128_u8, 0_u8);
    let mixed = (42, 3.14, true);

    // bind all fields
    let (x, y)      = point;
    let (r, g, b)   = color;

    // _ discards a field
    let (n, _, ok)  = mixed;
    println!("{x},{y} r={r} ok={ok} n={n}");

    // Nested tuples
    let nested = ((1, 2), (3, 4));
    println!("{}", nested.0.1); // 2 (first tuple, second
                                // field)
}
```



Default behaviour depends on build mode. Use explicit methods for full control:

```
fn main() {
    let x: u8 = 255;

    let bad = x + 1; // debug build: panics on overflow
    // release build: wraps silently (255 + 1 = 0)

    // — Explicit overflow strategies —
    // wrapping_*: always wraps (modular arithmetic)
    println!("{}", x.wrapping_add(1));    // 0
    println!("{}", x.wrapping_add(10));   // 9

    // saturating_*: clamps to boundary
    println!("{}", x.saturating_add(1));  // 255
    println!("{}", 0_u8.saturating_sub(5)); // 0

    // checked_*: returns Option
    println!("{}", x.checked_add(1));     // None
    println!("{}", x.checked_add(0));     // Some(255)
}
```

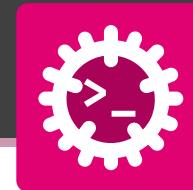
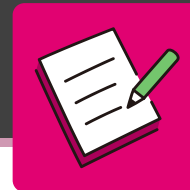
When does overflow matter?

- Embedded systems with small integer buffers
- Network protocol byte counters
- Cryptographic arithmetic
- Loop index arithmetic

Summary of overflow methods:

Method	Behaviour
wrapping_add	wrap around (modular)
saturating_add	clamp to MIN/MAX
checked_add	return Option<T>
overflowing_add	return (T, bool)

All four families exist: `_add`, `_sub`, `_mul`, `_div`, `_shl`, `_shr`, `_neg`.



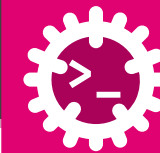
Read the code below (don't compile it yet) and spot the **type errors**. For each line, say **why** the compiler rejects it and how you would fix it.

```
fn main() {  
    let a: u8 = 300;  
    let b: u8 = 10;  
    let sum: u16 = a + b;  
    let ratio = 7 / 2.0;  
    let flag: bool = 1;  
    let point = (1, 2.0);  
    let z: i32 = point.1;  
}
```



Operators

Arithmetic, Comparison, Logical, and Bitwise



Arithmetic - +, -, *, /, %:

```
fn main() {
    let a: i32 = 17;
    let b: i32 = 5;

    println!("{a} + {b} = {}", a + b); // 22
    println!("{a} - {b} = {}", a - b); // 12
    println!("{a} * {b} = {}", a * b); // 85
    println!("{a} / {b} = {}", a / b); // 3 ← int
    println!("{a} % {b} = {}", a % b); // 2

    // Integer division truncates toward zero
    println!("{}", -7_i32 / 2); // -3 (not -4!)
    println!("{}", -7_i32 % 2); // -1

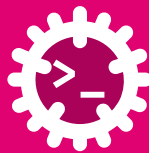
    // Compound assignment
    let mut n = 10;
    n += 5; n -= 3; n *= 2; n /= 4; n %= 4;
    println!("{n}"); // 2
}
```

Comparison - return bool:

```
fn main() {
    let x = 42;
    println!("{}", x == 42); // true (equal)
    println!("{}", x != 42); // false (not equal)
    println!("{}", x > 10); // true (greater than)
    println!("{}", x < 100); // true (less than)
    println!("{}", x >= 42); // true (greater or
    equal)
    println!("{}", x <= 42); // true (less or equal)
}
```

Operator precedence:

Highest $\Rightarrow$ Lowest (*, /, %),
(+, -), (<<, >>), (&), (^), (|), (==, !=, <, >, ≤, ≥), (&&), (||)



Logical Operators - work on `bool` values.

`&&` AND, `||` OR, `^` (XOR) and `!` NOT:

```
fn main() {
    let a: bool = true;
    let b: bool = false;

    println!("{}", a && b); // false (AND)
    println!("{}", a || b); // true  (OR)
    println!("{}", a ^ b);  // true  (XOR)
    println!("{}", !a);     // false (NOT)
}
```

```
fn main() {
    // && eval from left to right
    // RHS not evaluated if result is known
    let x: i32 = 0;
    // Safe: 10/x never runs when x == 0
    if x != 0 && 10 / x > 1 {
        println!("big quotient");
    } else {
        println!("x is zero, skipped division");
    }

    // || RHS not evaluated if LHS is true
    fn expensive() → bool { println!("called!"); true }
    if true || expensive() {
        println!("short-circuited");
    }
    // "short-circuited" - expensive() never ran
}
```



Bitwise operators - work on integer bits:

```
fn main() {
    let a: u8 = 0b1100_1010; // 202
    let b: u8 = 0b1010_1100; // 172

    println!("{:08b}", a & b); // 10001000 AND
    println!("{:08b}", a | b); // 11101110 OR
    println!("{:08b}", a ^ b); // 01100110 XOR (toggle)
    println!("{:08b}", !a);    // 00110101 NOT (bitflip)

    // Shift operators
    let flags: u8 = 0b0000_0001;
    println!("{:08b}", flags << 3); // 00001000 (×8)
    println!("{:08b}", flags >> 1); // 00000000 (÷2)
}
```

Practical: hardware register flags

```
fn main() {
    const LED_ON: u8 = 0b0000_0001;
    const MOTOR_ON: u8 = 0b0000_0010;
    const ALARM_ON: u8 = 0b0000_0100;

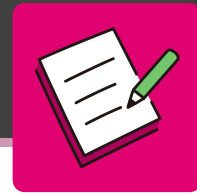
    let mut reg: u8 = 0b0000_0000;

    // set bits → 0b0000_0011
    reg |= LED_ON | MOTOR_ON;

    // flip bits → 0b0000_0111
    reg ^= ALARM_ON;

    // clear bits → 0b0000_0011
    reg &= !ALARM_ON;

    // 0b00000011 (LED & MOTOR)
    println!("0b{:08b}", reg);
}
```



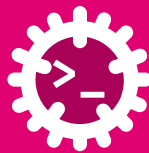
Trace the code by hand and write down the **output value** printed by each line. Mind **integer division**, **bitwise** operators and **operator precedence**.

```
fn main() {  
    let a = 17;  
    let b = 5;  
    println!("{}", a / b);           // 1  
    println!("{}", a % b);           // 2  
    println!("{}", a / b * b);        // 3  
  
    let x = 0b1010;                   // 10  
    let y = 0b0110;                   // 6  
    println!("{}", x & y);            // 4  
    println!("{}", x | y);            // 5  
    println!("{}", x ^ y);            // 6  
    println!("{}", x << 1);           // 7  
  
    println!("{}", true && false || !false); // 8  
    println!("{}", 3 + 4 * 2 > 10);    // 9  
}
```



Type Conversion

Explicit, safe, and idiomatic conversions



as performs **numeric casts** - fast but potentially **lossy**:

```
fn main() {  
    // Widening  
    // (always safe)  
    let small: u8 = 200;  
    let wide: u32 = small as u32;  
    println!("{wide}");           // 200  
  
    // Narrowing  
    // (truncates bits - potential loss!)  
    let big: u32 = 300;  
    let tiny: u8 = big as u8;      // 300 - 256 = 44  
    println!("{tiny}");           // 44  
  
    // Float → integer  
    // (truncates toward zero, no round)  
    let f: f64 = 3.99;  
    let i: i32 = f as i32;  
    println!("{i}");              // 3  
  
    // Negative float → unsigned: saturates
```

```
    let neg: f64 = -1.5;  
    let u: u32 = neg as u32;  
    println!("{u}");              // 0  
  
    // char ↔ u32  
    let c = 'A';  
    println!("{}", c as u32);     // 65  
    println!("{}", 66_u8 as char); // B  
}
```



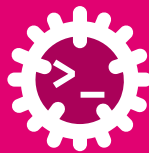

Why is as dangerous?

```
// Example: reading sensor data
fn read_sensor() -> u32 { 300 } // returns 300
fn process() {
    let raw: u32 = read_sensor();
    // Silent data loss - no warning!
    let byte: u8 = raw as u8; // 44, not 300!
    // Use in protocol field -> corrupted packet
    println!("sending byte: {byte}");
}
```

```
300 = 0b00000001_00101100
                ^^^^^^^
                keep these 8 bits
0b00101100 = 44
```

Rule of thumb for as:

Situation	Recommendation
Widening (u8 -> u32)	as is always safe
Float ↔ integer	as is OK (truncates)
Narrowing (u32 -> u8)	prefer try_into()
Pointer casts	as - systems code only



Fallible - returns Result, never silently loses data:

```
use std::convert::TryFrom;
use std::convert::TryInto;

fn main() {
    // TryFrom: explicit, static method
    let big: i32 = 300;
    match u8::try_from(big) {
        Ok(v) => println!("ok: {v}"),
        Err(e) => println!("error: {e}"),
    } // error: out of range integral type conversion

    // TryInto: method syntax (often preferred)
    let ok: i32 = 200;
    let result: Result<u8, _> = ok.try_into();
    println!("{:?}", result); // Ok(200)

    let bad: i32 = -5;
    let result: Result<u8, _> = bad.try_into();
    println!("{:?}", result); // Err(TryFromIntError(()))
}
```

Infallible - From / Into (always succeeds):

```
// From: explicit construction
let s = String::from("hello"); // &str -> String
let n = i64::from(42_i32);      // i32 -> i64

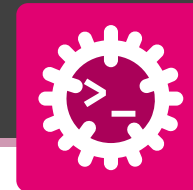
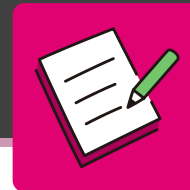
// Into: derived automatically from From
let s: String = "world".into(); // &str -> String
let n: i64 = 42_i32.into();     // i32 -> i64

// Custom conversion
struct Celsius(f64);
struct Kelvin(f64);

impl From<Celsius> for Kelvin {
    fn from(c: Celsius) -> Self {
        Kelvin(c.0 + 273.15)
    }
}

let boiling = Kelvin::from(Celsius(100.0));
println!("{:.2} K", boiling.0); // 373.15 K

// Into is auto-derived
let freezing: Kelvin = Celsius(0.0).into();
println!("{:.2} K", freezing.0); // 273.15 K
```



Trace each as cast and write down the **output value**. Watch for **narrowing**, **truncation toward zero** and **saturation**.

```
fn main() {  
    println!("{}", 300_i32 as u8);    // 1  
    println!("{}", 3.99_f64 as i32);  // 2  
    println!("{}", -1.5_f64 as u32);  // 3  
    println!("{}", 'A' as u32);       // 4  
    println!("{}", 66_u8 as char);    // 5  
    println!("{}", 255_u8 as i8);     // 6  
    println!("{}", -1_i8 as u8);      // 7  
}
```

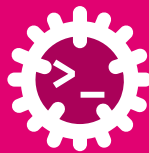


as never fails - it silently reinterprets or clamps. Prefer TryFrom/TryInto when loss must be caught.



Text Types

char, &str, String



char - a single **Unicode Scalar Value (USV)**, always 4 bytes wide:

```
fn main() {
    let letter: char = 'A';
    let emoji: char = '🦀';           // Ferris the crab!
    let accent: char = 'é';           // accented letter
    let japanese: char = '日';        // "day" in Japanese
    let ascii_a: i32 = 'A' as i32;    // 65 - see ASCII

    table

    // Escape Characters
    let newline: char = '\n';
    let carriage_return: char = '\r';
    let tab: char = '\t';
    let quote: char = '\"';
    let backslash: char = '\\';
    let 7bit-unicode: char = '\u{7F}'; // DEL
    let 8bit-unicode: char = '\u{1F600}'; // 😊
    let 24bit-unicode: char = '\u{10FFFF}';
```

```
// Classification
println!("{}", 'a'.is_alphabetic()); // true
println!("{}", '5'.is_ascii_digit()); // true
println!("{}", '5'.is_alphanumeric()); // true
println!("{}", '2'.is_numeric()); // true
println!("{}", ' '.is_whitespace()); // true
println!("{}", 'A'.is_uppercase()); // true
println!("{}", 'z'.is_lowercase()); // true
println!("{}", 'A'.is_ascii()); // true

// Case conversion
println!("{}", 'a'.to_uppercase()); // A (it)
println!("{}", 'Z'.to_lowercase()); // z (it)

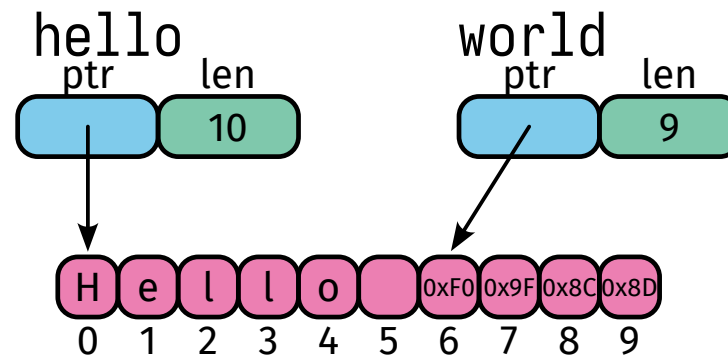
// Digit conversion
println!("{}", 'F'.to_digit(16)); // Some(15)
println!("{}", char::from_digit(10, 16)); // Some('a')
}
```



&str - an immutable **borrowed** view into string bytes.

```
fn main() {
    // String literals have static lifetime
    let hello: &str = "Hello 🌍";
    // Byte length (not char count!)
    println!("{}", hello.len());           // 10 (🌍 = 4 bytes)
    // Char count
    println!("{}", hello.chars().count()); // 7
    // Slice (byte-indexed - must be on char boundaries!)
    let world: &str = &hello[6..10];       // 6-9 (10 excluded)
    println!("{}", world);                 // 🌍
    // Iterate characters
    for c in hello.chars().take(5) {
        print!("{}", c); // H e l l o
    }
}
```

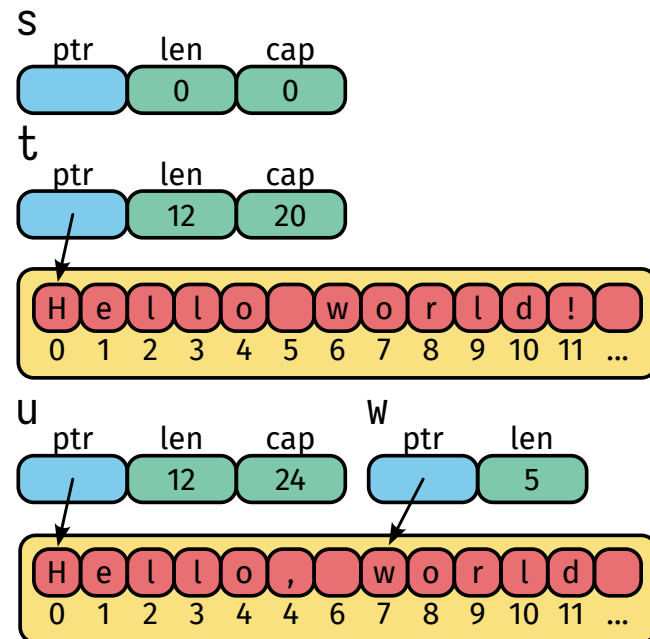
They are stored as literal data in the binary and have a 'static lifetime.





String - Heap-allocated, growable, UTF-8 text:

```
fn main() {  
    let s = String::new();           // empty string  
    // Append  
    let mut t = String::from("Hello"); // from literal  
    t.push(' ');                     // single char  
    t.push_str("world!");           // string slice  
    println!("{t}");                 // Hello world!  
    println!("len={} cap={}", t.len(), t.capacity());  
    // Insert  
    let mut u = String::from("Hello world!");  
    u.insert(5, ',');                // insert char at index  
    println!("{u}");                 // Hello, world!  
    // Remove  
    u.pop();                         // removes last char  
    println!("{u}");                 // Hello, world  
    // Slice  
    let w = &u[7..12];               // "world" (7-11)  
}
```

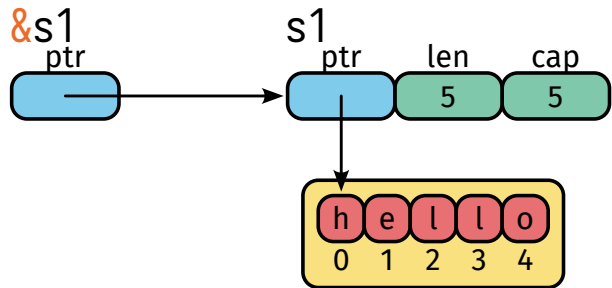


- cap is the max number of bytes the string can hold without reallocating
- len is the current number of bytes used

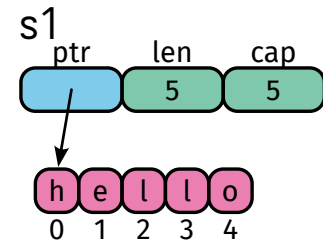


String	&str
Owned (on heap)	Borrowed (a view)
Mutable	Immutable
Has len, capacity	Has len only
Growable	Fixed slice
Allocates memory	Zero allocation

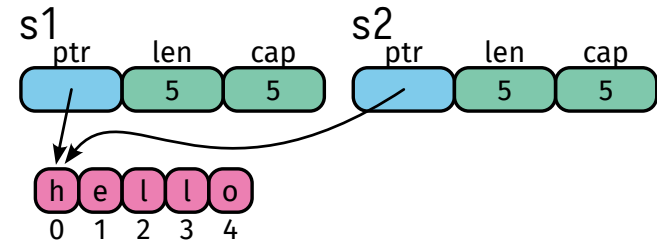
```
let s1 = String::from("hello");
&s1
```



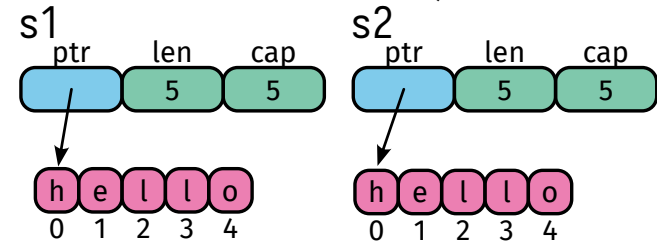
```
let s1 = "hello";
```

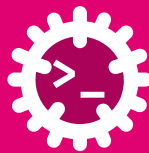


```
let s2 = s1;
```



```
let s2 = s1.clone();
```





Searching:

```
let s = "Hello, Rust world!";
println!("{}", s.contains("Rust")); // true
println!("{}", s.starts_with("Hello")); // true
println!("{}", s.ends_with("!")); // true
println!("{}", s.find("Rust")); // Some(7)
println!("{}", s.rfind('o')); // Some(13)
```

Replacing and splitting:

```
let r = s.replace("Rust", "Go");
println!("{}", r); // Hello, Go world!
let words: Vec<&str> = s.split_whitespace().collect();
println!("{}", words:?);
// ["Hello,", "Rust", "world!"]
let parts: Vec<&str> = "a,b,c".split(',').collect();
println!("{}", parts:?); // ["a", "b", "c"]
```

Trimming and transforming:

```
let padded = " hello world ";
println!("{}", padded.trim()); // 'hello world'
println!("{}", padded.trim_start()); // 'hello world '
println!("{}", padded.trim_end()); // ' hello world'
println!("{}", "hello".to_uppercase()); // HELLO
println!("{}", "WORLD".to_lowercase()); // world
println!("{}", "=".repeat(20)); // =====
```

Collecting:

```
// Chars to Vec
let chars: Vec<char> = "abc".chars().collect();
println!("{}", chars:?); // ['a', 'b', 'c']
// Join lines
let lines = ["line1", "line2", "line3"];
let joined = lines.join("\n");
println!("{}", joined); //line1\nline2\nline3
```



Value → String:

```
fn main() {  
    let n: i32 = 42;  
    let f: f64 = 3.14;  
    let b: bool = true;  
  
    // to_string() works on any Display type  
    let s1 = n.to_string(); // "42"  
    let s2 = f.to_string(); // "3.14"  
    let s3 = b.to_string(); // "true"  
    println!("{s1} {s2} {s3}");  
  
    // format! - more control  
    let msg = format!("n={n:05}, f={f:.2}");  
    println!("{msg}"); // n=00042, f=3.14  
}
```

String → value - parse():

```
let text = "42";  
let n: i32 = text.parse().expect("bad input");
```

&str → String - to_owned():

```
let a = "hi".to_owned(); // clearest for &str  
let b = "hi".to_string(); // more general  
let c = String::from("hi"); // explicit  
let d: String = "hi".into(); // inferred  
println!("{a} {b} {c} {d}"); // hi hi hi hi
```

When to use which:

Method	Best for
to_string()	any Display type → String
to_owned()	&str specifically → String
String::from	explicit, readable constructor
.into()	idiomatic generic / trait code
.parse()	text → numeric or other types
format!(...)	multi-value or formatted strings



Arguments

```
let s = format!("Hello, {}, the answer is {}", "Alice", 42);           // Positional Arguments
let s = format!("Hello, {name}, the answer is {age}", name = "Bob", age = 7); // Named Arguments
let s = format!("{0} is {1} years old. {0} loves Rust!", "Carol", 25); // Reusing Arguments
```

Align

```
format!("{:<8}", 1); // 1..... - left aligned
format!("{:>8}", 1); // .....1 - right aligned
format!("{:^8}", 1); // ...1.... - center aligned
```

Padding

```
format!("{:*<8}", 1); // 1***** - left align, padding "*"
format!("{:0>8}", 1); // 00000001 - right align, padding "0"
format!("{:>08}", 1); // 00000001 - right align, padding leading "0"
```



```
format!("{: >8}", 1); //      1 - right align, padding " "  
format!("{: #^8}", 1); // ###1#### - right align, padding "#"
```

Sign

```
format!("{: >+8}", 1); //      +1 - right align, sign +  
format!("{: >8}", -1); //      -1 - right align, sign -  
format!("{: >+08}", 1); // +0000001 - right align, sign +, padding leading "0"  
format!("{: 0>+8}", 1); // 000000+1 - right align, padding "0", sign +
```

Precision

```
format!("{: * <8}", 1); // 1***** - left align, padding "*"   
format!("{: 0 >8}", 1); // 00000001 - right align, padding "0"   
format!("{: >08}", 1); // 00000001 - right align, padding leading "0"   
format!("{: >8}", 1); //      1 - right align, padding " "   
format!("{: #^8}", 1); // ###1#### - right align, padding "#"
```



Type

```
format!("{:}", E as u32); // 2 - Integer (follows type)
format!("{:}", E); // 2.718281828459045 - Float (follows type)
format!("{:b}", 49374); // 1100000011011110 - Binary
format!("{:#b}", 49374); // 0b1100000011011110 - Binary alternative (adds "0b")
format!("{:#034b}", 49374); // 0b00000000000000001100000011011110 - Binary 32bit (adds "0b" and 32 digits)
format!("{:o}", 49374); // 140336 - Octal
format!("{:#o}", 49374); // 0o140336 - Octal alternative (adds "0o")
format!("{:#018o}", 49374); // 0o00000000000140336 - Octal 16 digits (adds "0o" and 16 digits)
format!("{:x}", 49374); // c0de - Hex lower case
format!("{:#x}", 49374); // 0xc0de - Hex lower case alternative (adds "0x")
format!("{:#010x}", 49374); // 0x0000c0de - Hex 8 digits (adds "0x" and 8 digits)
format!("{:X}", 49374); // C0DE - Hex upper case
format!("{:#X}", 49374); // 0xC0DE - Hex upper case alternative (adds "0X")
format!("{:#010X}", 49374); // 0x0000C0DE - Hex 8 digits (adds "0X" and 8 digits)
format!("{:e}", E.pow(123.0)); // 2.6195173187490456e53 - Scientific notation lower case (adds "e")
format!("{:E}", E.pow(123.0)); // 2.6195173187490456E53 - Scientific notation upper case (adds "E")
format!("{:p}", &1); // 0x1021872c0 - Pointer address
```


Rust

```
formatting = [argument][:][[fill]align][sign][#][0][width][.precision][type]
argument   = integer | identifier
fill       = <any character>
align      = "<" | ">" | "^"
sign       = "+"
"#"        = alternate formatting
"0"        = zero-padding
width      = integer
precision  = integer
type       = "b" | "o" | "x" | "X" | "e" | "E" | "p" | "?"
```

```
"{" [argument][":"][[fill]align[sign][#"["0"][width][.precision][type]""]
```

- padding "0"

- alternate formatting
- (human-readable)

- "" - default
- + - + prefix by tschinz

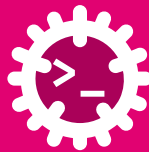
nbr of descendants - integer -

- < - left (default) "" - default follows type -
- > - right "b" - binary _____
- ^ - center "o" - octal _____
 "x" - hex _____
 "X" - Hex _____
 "e" - scientific _____
 "E" - Scientific _____
 "p" - pointer addr _____
 "? " - debug (trait req) _____

- "" - follows arg sequence (default)
- integer - selects arg by position
- identifier - selects arg by name

```
print!("{0:*>+10.3e}", std::f64::consts::E)
```

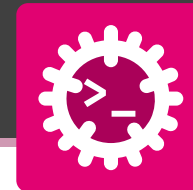
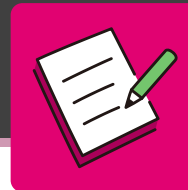
<https://wiki.zahno.dev/blog/2024/11/20/rust-string-formatting---an-ascii-art-cheatsheet.html>



Building a formatted table:

```
fn main() {
    let items = [
        ("Alice", 92, 98.7),
        ("Bob", 87, 91.2),
        ("Carol", 95, 99.1),
    ];
    // Header (left-align name, right-align numbers)
    println!("| {:<5} | {:>5} | {:>6} |", "Name", "Score", "Pct");
    // Alignment row (: marks alignment direction)
    println!("|:{0:-<6}|{0:-<6}:|{0:-<7}:|", "");
    // Data rows
    for (name, score, pct) in items {
        println!("| {:<5} | {:>5} | {:>5.1}% |", name, score, pct);
    }
}
```

	Name		Score		Pct	
	:		:		:	
	Alice		92		98.7%	
	Bob		87		91.2%	
	Carol		95		99.1%	



Part 1 - What is the output?

Trace the conversions and format specifiers.

```
fn main() {  
    let mut s = String::new();  
    s.push_str("Rust");  
    s.push('!');  
  
    let n: i32 = "42".parse().unwrap();  
    let label = "id".to_owned() + "-" + &n.to_string();  
  
    println!("{s} {}", s.len()); // a  
    println!("{label}");         // b  
    println!("{:>6}", "hi");     // c  
    println!("{:*<6}", "hi");    // d  
    println!("{:08.3}", 3.14159); // e  
    println!("{:#x}", 255);      // f  
}
```

Part 2 - Build the string

Given the bindings below, write a **single** format! call that produces **exactly** this line:

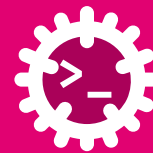
```
Bob    : 87
```

```
let name = "Bob"; // left-aligned, width 6  
let score = 87;   // right-aligned, width 3  
let line: String = format!("{: ? }");
```



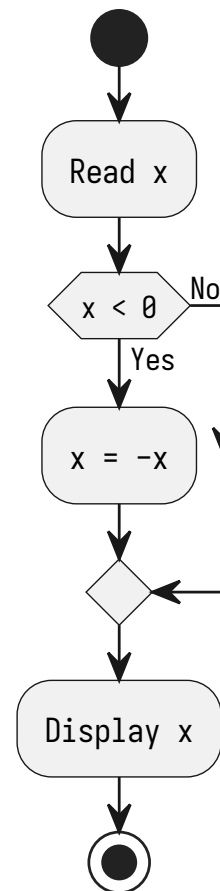

Control Flow

if, else if, else, match, loopS, and more



if is an **expression**:

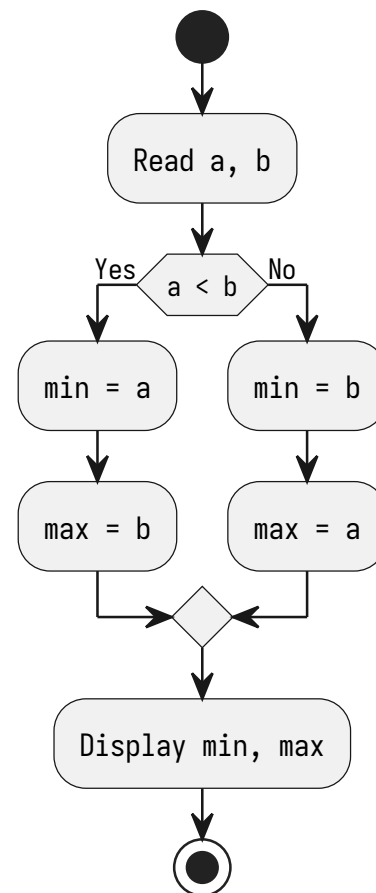
```
fn main() {  
    let mut x: i32 = -7;  
    // Statement form  
    if x < 0 {  
        x = -x; // make positive  
    }  
    println!("{x}"); // +7  
    // Expression form - both arms must return same type  
    x = if x < 0 { -x };  
    println!("{x}"); // +7  
}
```





if / else - conditional branch; if is an **expression**:

```
fn main() {  
    let (a, b): (i32, i32) = (42, 7);  
    let (max, min);  
    // Statement form  
    if a < b {  
        max = b; min = a;  
    } else {  
        max = a; min = b;  
    }  
    println!("Max {max}, Min {min}");  
    // Expression form - both arms must return same type  
    let (a, b): (i32, i32) = (7, 42);  
    let (max, min);  
    (max, min) = if a < b { (b, a) } else { (a, b) };  
    println!("Max {max}, Min {min}");  
}
```

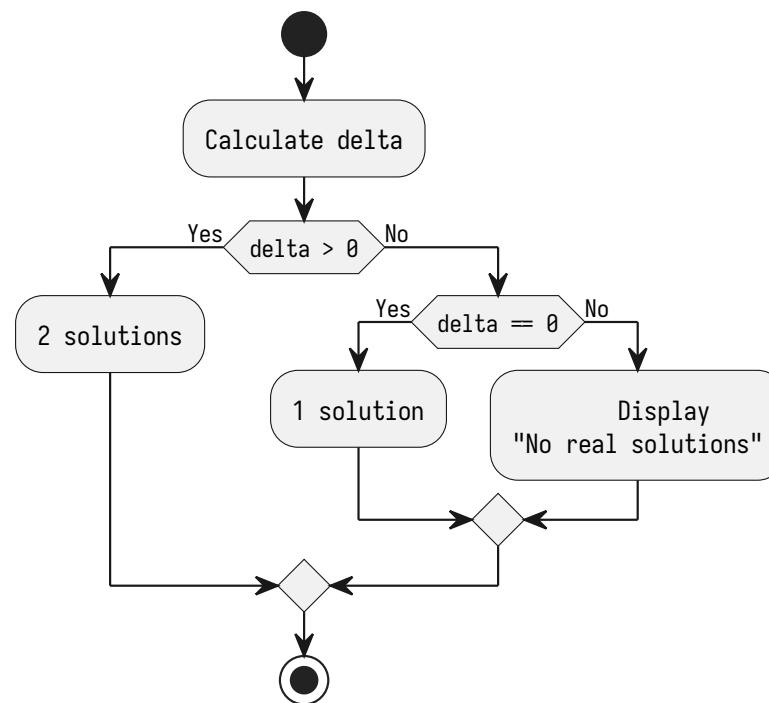


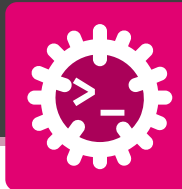


else if - evaluates branches top-to-bottom, first match wins:

```
fn main() {  
    // ax2 + bx + c = 0  
    let (a, b, c) = (1.0, -3.0, 2.0);  
    // Δ = b2 - 4ac  
    let delta = b * b - 4.0 * a * c;  
    if delta > 0.0 {  
        // x1,2 = (-b ± √Δ) / (2a)  
        let x1 = (-b + delta.sqrt()) / (2.0 * a);  
        let x2 = (-b - delta.sqrt()) / (2.0 * a);  
        println!("x1 = {x1}, x2 = {x2}");  
    } else if delta == 0.0 {  
        // x = -b / (2a)  
        let x = -b / (2.0 * a);  
        println!("x = {x}");  
    } else {  
        println!("No real solutions");  
    }  
}
```

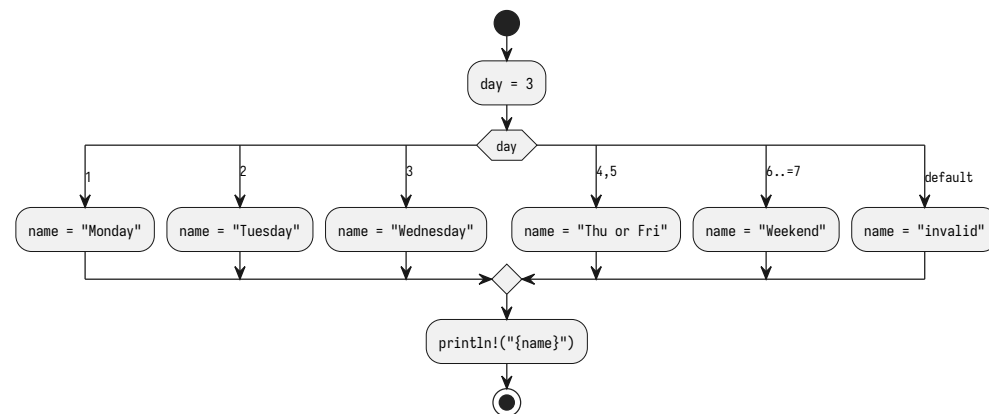
$$y = ax^2 + bx + c \Rightarrow x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

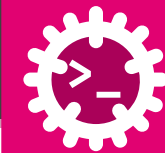




match - exhaustive multi-way branch on a value or pattern:

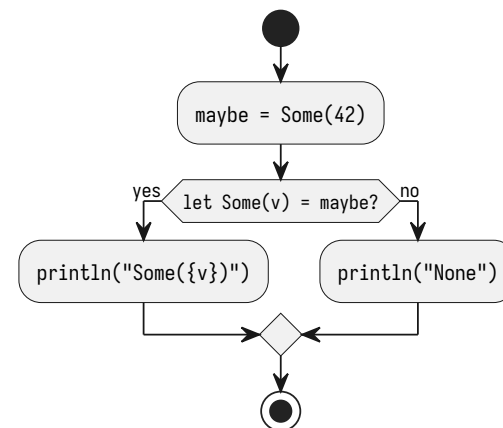
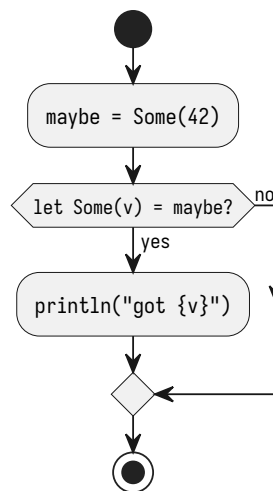
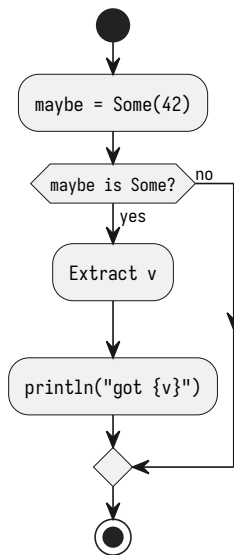
```
fn main() {  
    let day: u8 = 3;  
    let name = match day {  
        1      => "Monday",  
        2      => "Tuesday",  
        3      => "Wednesday",  
        4 | 5   => "Thu or Fri", // OR pattern  
        6..=7   => "Weekend",    // inclusive range  
        _       => "invalid",    // wildcard (must be last)  
    };  
    println!("{name}"); // Wednesday  
}
```





if let - concise pattern match for a **single** variant:

```
fn main() {  
    let maybe: Option<i32> = Some(42);  
    // Verbose: full match  
    match maybe {  
        Some(v) => println!("got {v}"),  
        None    => {},  
    }  
    // Concise: if let  
    if let Some(v) = maybe {  
        println!("got {v}"); // 42  
    }  
    // with else branch  
    if let Some(v) = maybe {  
        println!("Some({v})"); // Some(42)  
    } else {  
        println!("None");  
    }  
}
```

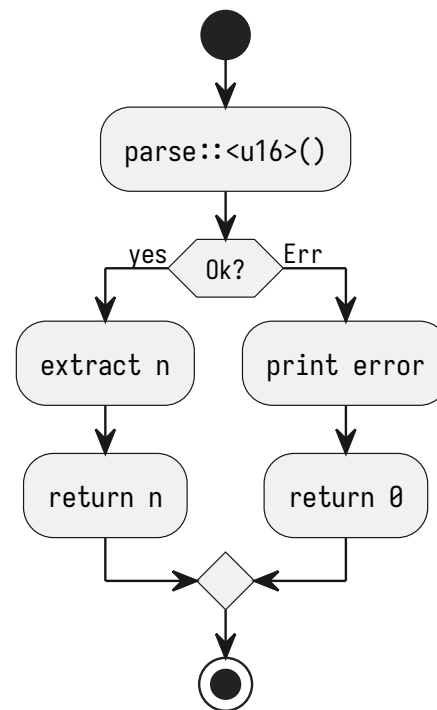


Don't worry it will get more clear with the introduction of enums later.



let else - bind a pattern or **diverge** (return / panic / break):

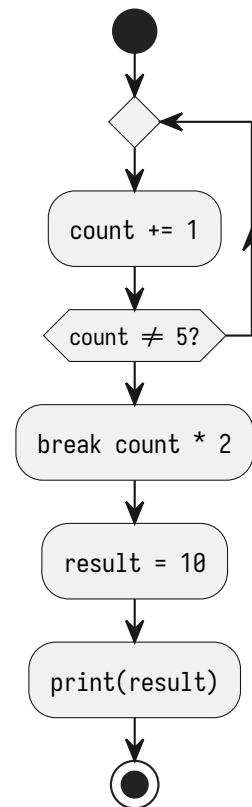
```
fn parse_port(s: &str) → u16 {  
    // Must diverge in the else branch  
    let Ok(n) = s.parse::<u16>() else {  
        eprintln!("invalid port: {s}");  
        return 0;           // diverge!  
    };  
    // n is in scope from here on  
    n  
}  
  
fn main() {  
    println!("{}", parse_port("8080")); // 8080  
    println!("{}", parse_port("abc"));  // 0  
}
```





loop - runs forever; exit with **break**; can **return a value**:

```
fn main() {  
    let mut count = 0;  
    // loop is an expression - break carries the value  
    let result = loop {  
        count += 1;  
        if count == 5 {  
            break count * 2;    // returns 10  
        }  
    };  
    println!("{result}"); // 10  
    // Labelled loop  
    'retry: loop {  
        // ... break 'retry to exit by name  
        break 'retry;  
    }  
}
```





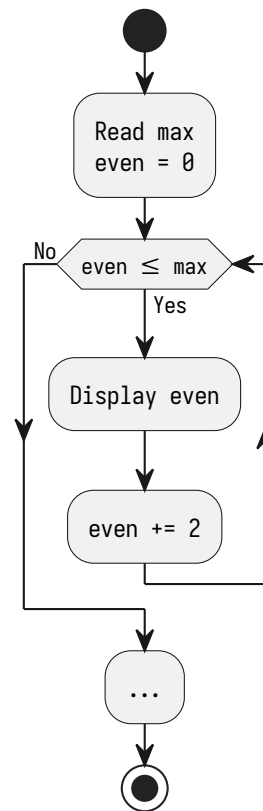
while - checks condition **before** each iteration:

Classic while

```
fn main() {  
    let max = 10;  
    let mut even = 0;  
    while even ≤ max {  
        println!("{even}");  
        even += 2;  
    }  
}
```

while let - loop while pattern matches

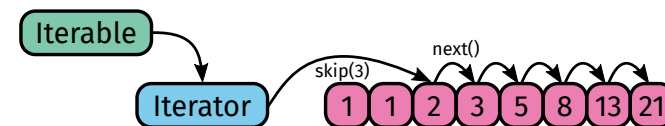
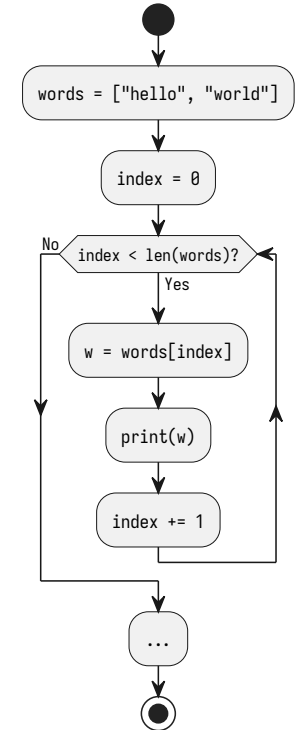
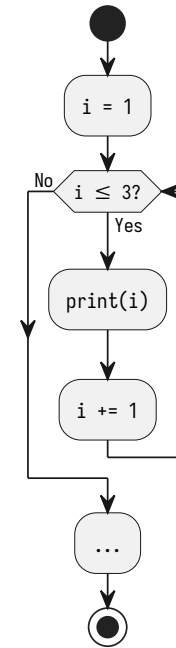
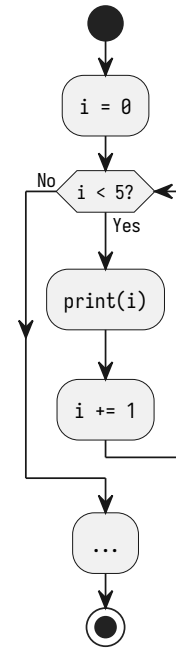
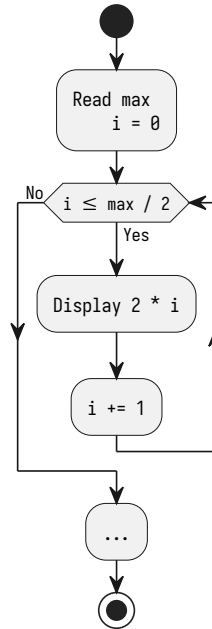
```
fn main() {  
    let mut stack = vec![3, 2, 1];  
    while let Some(top) = stack.pop() {  
        print!("{top} ");  
    }  
}
```

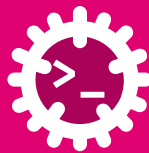




for - iterates over any IntoIterator

```
fn main() {  
    let max = 10;  
    for i in 0..=max / 2 {  
        println!("{}", 2 * i);  
    }  
    // Half-open range [0, 4]  
    for i in 0..5 {  
        print!("{}", i);    // 0 1 2 3 4  
    }  
    // Inclusive range [1, 3]  
    for i in 1..=3 {  
        print!("{}", i);    // 1 2 3  
    }  
    // Iterate over a collection  
    let words = ["hello", "world"];  
    for w in words {  
        println!("{}", w);  
    }  
}
```





```
fn main() {
    let words = ["alpha", "beta", "gamma"];
    let nums = [10, 20, 30];

    // enumerate() - (index, value)
    for (i, w) in words.iter().enumerate() {
        println!("{i}: {w}"); // 0: alpha ...
    }

    // zip() - pair two iterators
    for (w, n) in words.iter().zip(nums) {
        println!("{w}={n}"); // alpha=10 ...
    }

    // rev() - reverse a range
    for i in (1..=5).rev() {
        println!("{i} "); // 5 4 3 2 1
    }

    // step_by() - custom step (replaces i+=3)
    for i in (0..12).step_by(3) {
        println!("{i} "); // 0 3 6 9
    }
}
```

```
fn main() {
    let nums = [1, 2, 3, 4, 5, 6];

    // filter() - skip elements
    for x in nums.iter().filter(|&x| x % 2 == 0) {
        println!("{x} "); // 2 4 6
    }

    // map() - transform each element
    for x in nums.iter().map(|&x| x * x) {
        println!("{x} "); // 1 4 9 16 25 36
    }

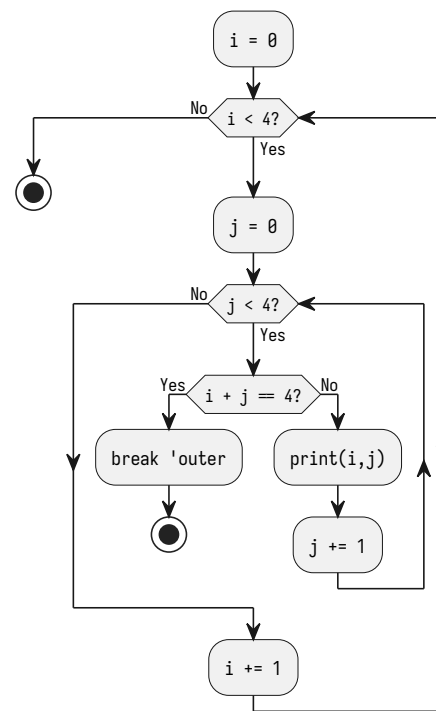
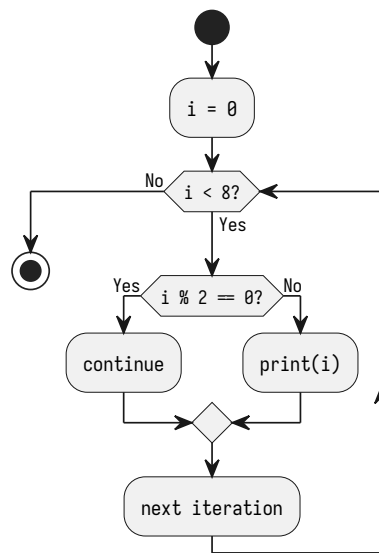
    // chain() - join two iterables
    let a = [1, 2];
    let b = [3, 4];
    for x in a.iter().chain(b.iter()) {
        println!("{x} "); // 1 2 3 4
    }

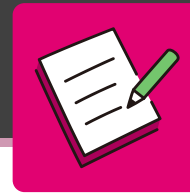
    // flatten() - one level of nesting
    let nested = [[1, 2], [3, 4]];
    for x in nested.iter().flatten() {
        println!("{x} "); // 1 2 3 4
    }
}
```



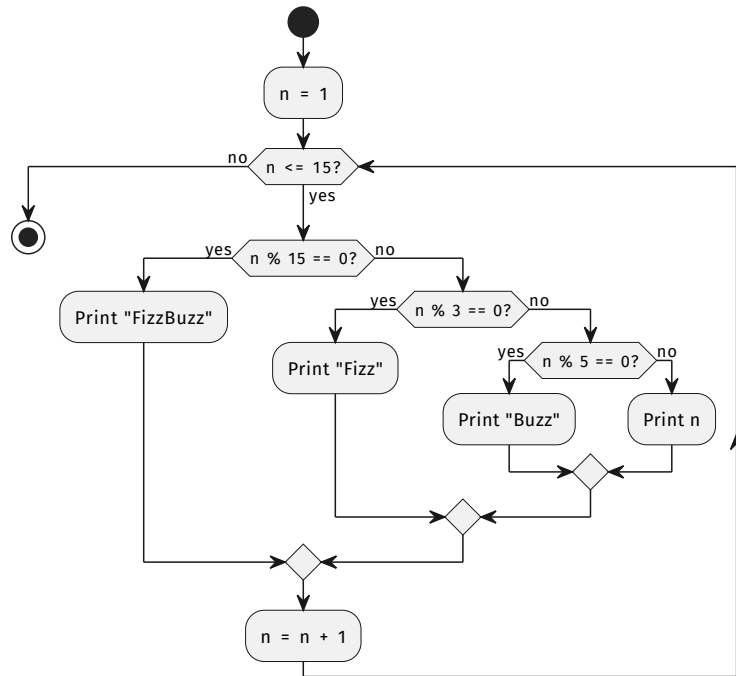
continue skips the rest of the body; **break** exits the loop:

```
fn main() {  
    // continue: skip even numbers  
    for i in 0..8 {  
        if i % 2 == 0 { continue; }  
        print!("{i} ");    // 1 3 5 7  
    }  
  
    // break with label: exit outer loop  
    'outer: for i in 0..4 {  
        for j in 0..4 {  
            if i + j == 4 { break 'outer; }  
            print!("{i},{j} ");  
        }  
    }  
}
```





Translate the flowchart below into Rust using a for loop and an if / else if / else chain.



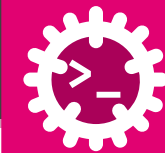
Order matters: the **most specific** test (% 15) must come **first**, otherwise % 3 would catch it.

Bonus: can you express the same logic with a single match?



Functions

Let's make spaghetti!



fn - explicit type on every parameter and on the return:

```
//      parameters (name: Type)
//      |          |          |  return type
fn add(a: i32, b: i32) → i32 {
    a + b    // implicit return: last expr, no `;`
}

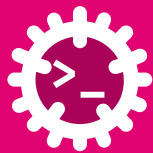
// no return type `→ T` means `→ ()` (unit)
fn greet(name: &str) {
    println!("Hello, {name}!");
}

// explicit early return
fn abs(x: i32) → i32 {
    if x < 0 { return -x; } // early exit
    x                      // implicit return
}

fn main() {
    println!("{}", add(3, 4)); // 7
    greet("Alice");           // Hello, Alice!
    println!("{}", abs(-9));  // 9
}
```

teaser: this is where functions can go

```
//      function name
//      |
//      'a      lifetime parameter
//      |
//      |
//      |      generic type T
//      |      |
//      |      |      first input slice
//      |      |      |
//      |      |      |      second input slice
//      |      |      |      |
//      |      |      |      |      return type:
//      |      |      |      |      a borrowed
//      |      |      |      |      slice
fn longest<'a, T>(x: &'a [T], y: &'a [T]) → &'a [T]
where T: Ord,
//      |      T must be orderable, so slices [T] can be compared
//      |      trait bound
{
    if x > y { x } else { y }
}
```

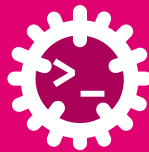


Single value and no value (()):

```
fn square(x: f64) → f64 { x * x }
// returns () implicitly - same as void
fn log(msg: &str) {
    println!("[LOG] {msg}");
}
// early return to guard against bad input
fn safe_div(a: f64, b: f64) → f64 {
    if b == 0.0 { return f64::NaN; }
    a / b
}
fn main() {
    println!("{}", square(4.0));           // 16
    log("started");
    println!("{}", safe_div(1.0, 0.0)); // NaN
}
```

Multiple values via tuple destructuring:

```
// return a tuple - idiomatic multi-value return
fn min_max(v: &[i32]) → (i32, i32) {
    let (mut lo, mut hi) = (v[0], v[0]);
    for &x in v {
        if x < lo { lo = x; }
        if x > hi { hi = x; }
    }
    (lo, hi) // return tuple
}
fn main() {
    let data = [3, 1, 4, 1, 5, 9, 2, 6];
    // destructure the tuple on the left
    let (lo, hi) = min_max(&data);
    println!("min={lo}, max={hi}"); // min=1, max=9
    // ignore one field with _
    let (_, hi) = min_max(&data);
    println!("max={hi}");           // max=9
}
```

Ownership rules

1. Each value in Rust has an **owner**.
2. There can only be **one owner** at a time.
3. When the owner goes out of scope, the value will be **dropped**.

Rules of References

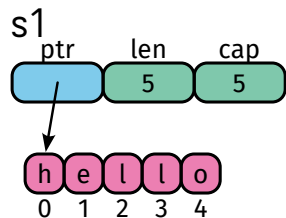
1. You can have **either** one mutable reference **or** any number of immutable references.
2. References must always be valid (no dangling references).

```
let s1 = String::from("hello");  
// Ownership moves to s2  
let s2 = s1;  
// println!("{s1}"); // (!) error: moved  
// Borrowing: many immutable refs OK  
let r1 = &s2;  
let r2 = &s2;  
println!("{r1} {r2}");  
// Or one mutable ref...  
let mut s3 = String::from("world");  
let m = &mut s3;  
m.push_str("!");  
// let r3 = &s3; // (!) error: already  
// borrowed as mutable  
println!("{m}");
```

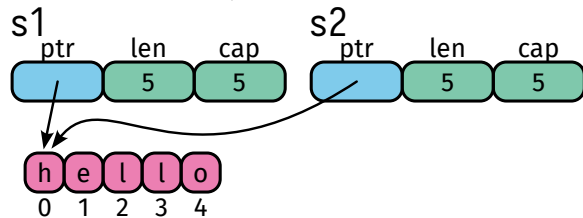


`&str` is a Copy type (a pointer to a string slice), while `String` is a Move type (a heap-allocated string):

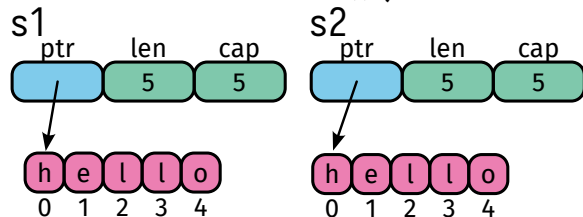
```
let s1 = "hello";
```



```
let s2 = s1;
```

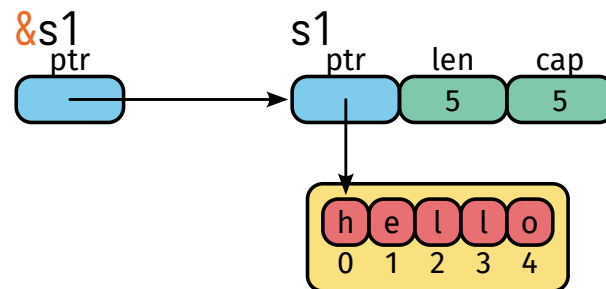


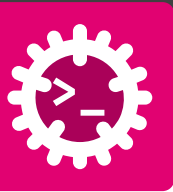
```
let s2 = s1.clone();
```



```
let s1 = String::from("hello");
```

```
&s1
```





Passing a value **transfers** or **copies** it depending on the type:

```
// i32 is Copy - the caller keeps its copy
fn double(n: i32) → i32 { n * 2 }

// String is NOT Copy - ownership moves into the fn
fn shout(s: String) → String {
    s.to_uppercase() // takes ownership, returns new value
}

fn main() {
    // Copy types: original still valid
    let x = 5;
    println!("{}", double(x)); // 10
    println!("{}", x);          // 5 ✓ still usable

    // Move types: original is gone after the call
    let name = String::from("alice");
    let loud = shout(name);      // name moved into shout()
    // println!("{}", name);    // ✗ compile error: moved!
    println!("{}", loud);       // ALICE
}
```

Copy types - value is duplicated	Move types - ownership transfers
bool, char	String
i8...i128, isize	Vec<T>
u8...u128, usize	HashMap<K, V>
f32, f64	Box<T>
(T, U) if all Copy	File, TcpStream
[T; N] if T: Copy	any struct by default
&T (shared ref)	&mut T (unique ref)



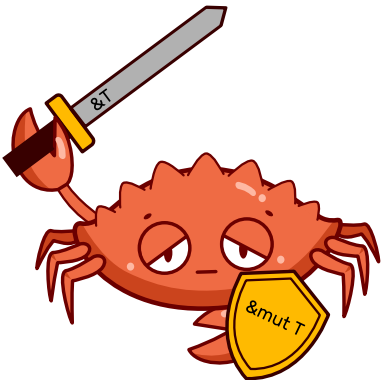
Borrow a value instead of taking ownership:

```
// &T - immutable borrow: read-only, never moves
fn length(s: &str) -> usize {
    s.len()           // caller keeps ownership
}

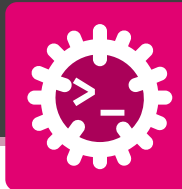
// &mut T - mutable borrow: read+write, still no move
fn double_all(v: &mut Vec<i32>) {
    for x in v.iter_mut() { *x *= 2; }
}

fn main() {
    let hello = String::from("hello");
    println!("{}", length(&hello)); // 5
    println!("{}", hello);           // ✓ still owned here

    let mut nums = vec![1, 2, 3];
    double_all(&mut nums);           // lend mutably
    println!("{}", nums);            // [2, 4, 6]
}
```



Syntax	Alias	Rule
T	move/copy	1 owner
&T	shared ref	many readers
&mut T	unique ref	1 writer



Anonymous functions

A closure is a function-like value written directly where it is needed. In **Python** and **C/C++**, they are called **lambda functions**.

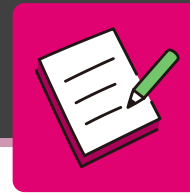
```
fn main() {  
    let square = |x: i32| x * x;  
    println!("{}", square(4)); // 16  
  
    let factor = 10;  
    let scale = |x: i32| x * factor;  
    println!("{}", scale(3)); // 30  
}
```

Closure syntax

```
|x| x + 1  
|x: i32| x * x  
|x: i32| → i32 {  
    x + 1  
}
```

Key idea

- Anonymous function
- Can be stored in a variable
- Can use values from the surrounding scope



Write **two small functions** for an exam grading system.

1. `swiss_grade(points: u32) → f64`

The exam has **5 exercises** of **0-10 points** each (max = 50). Convert the points to a **Swiss grade** (1-6):

$$\text{grade} = \frac{5}{\text{points}_{\text{max}}} * \text{points}_{\text{student}} + 1 \quad (1)$$

2. `bologna_grade(grade: f64) → char`

Map the Swiss grade to a **Bologna letter** (see table).

3. In main, print points, Swiss grade (2 decimals) and the letter for 50, 45, 40, 35, 30, 25 points.

Letter	Swiss grade
A	5.75 <= 6.00
B	5.25 <= 5.74
C	4.75 <= 5.24
D	4.25 <= 4.74
E	3.75 <= 4.24
F	1.00 <= 3.74

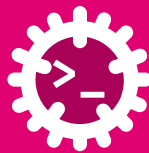


Use as `f64` to mix `u32` points with floating-point maths.



User-Defined Types

Let's make it objects!



One type, several variants

An enum defines a type that can be **one of several alternatives**.

```
enum TrafficLight {  
    Red,  
    Yellow,  
    Green,  
}  
  
fn main() {  
    let light = TrafficLight::Green;  
    match light {  
        TrafficLight::Red    => println!("stop"),  
        TrafficLight::Yellow => println!("prepare"),  
        TrafficLight::Green  => println!("go"),  
    }  
}
```

Key idea

- A value has exactly **one** variant
- Variants belong to the enum type
- **match** is used to handle all cases
- The compiler checks completeness



A variant can store values

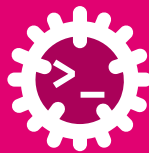
```
enum SensorValue {  
    Temperature(f64),  
    Pressure(f64),  
    Humidity(f64),  
}  
  
fn main() {  
    let value = SensorValue::Temperature(23.7);  
    match value {  
        SensorValue::Temperature(t) => println!("{t} °C"),  
        SensorValue::Pressure(p)    => println!("{p} Pa"),  
        SensorValue::Humidity(h)    => println!("{h} %"),  
    }  
}
```

Pattern matching extracts the value

```
SensorValue::Temperature(t)  
                        ^  
                        |  
                    extracted value
```



`Option<T>` and `Result<T,E>` are simply enums whose variants also contain data.



Value or no value

`Option<T>` is used when a value may be absent.

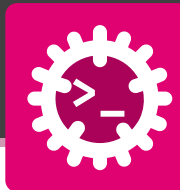
```
fn first(values: &[i32]) → Option<i32> {
    if values.len() > 0 {
        Some(values[0])
    } else {
        None
    }
}

fn main() {
    let data = [10, 20, 30];
    match first(&data) {
        Some(v) ⇒ println!("first = {v}"),
        None    ⇒ println!("empty list"),
    }
}
```

```
enum Option<T> {
    Some(T),
    None,
}
```

Meaning

- `Some(T)` contains a value
- `None` means no value
- Rust has no `null`

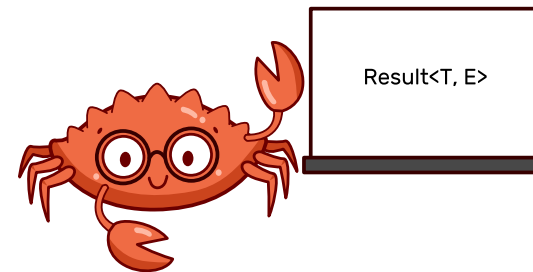


Success or error

`Result<T, E>` is used when an operation can fail.

```
fn parse_port(s: &str) → Result<u16, String> {
    match s.parse::<u16>() {
        Ok(port) ⇒ Ok(port),
        Err(_)   ⇒ Err(String::from("invalid port")),
    }
}

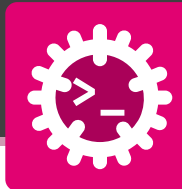
fn main() {
    let result = parse_port("8080");
    match result {
        Ok(port) ⇒ println!("port = {port}"),
        Err(msg) ⇒ println!("error: {msg}"),
    }
}
```



```
enum Result<T, E> {
    Ok(T),
    Err(E),
}
```

Meaning

- `Ok(T)` contains the successful value
- `Err(E)` contains the error value
- The caller must handle both cases



unwrap() was a mistake

Sometimes we are sure that an `Option` contains `Some(T)` or a `Result` contains `Ok(T)`. `unwrap()` extracts the value directly.



Option<T>

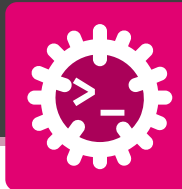
```
fn main() {  
    let port: Option<u16> = Some(8080);  
    let value = port.unwrap();  
    println!("port = {value}"); // 8080  
}
```

```
fn main() {  
    let port: Option<u16> = None;  
    let value = port.unwrap(); // panic  
    println!("port = {value}");  
}
```

Result<T, E>

```
fn main() {  
    let port: Result<u16, &str> = Ok(8080);  
    let value = port.unwrap();  
    println!("port = {value}"); // 8080  
}
```

```
fn main() {  
    let port: Result<u16, &str> = Err("invalid port");  
    let value = port.unwrap(); // panic  
    println!("port = {value}");  
}
```

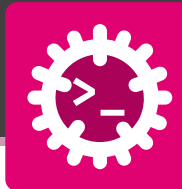


`unwrap()` was a mistake



i `expect("message")` works like `unwrap()`, but adds your own panic message.

```
fn main() {  
    let port: u16 = "8080".parse::<u16>().expect("port must be a number between 0 and 65535");  
}  
  
fn main() {  
    let port: u16 = "8080"  
        .parse::<u16>()  
        .expect("port must be a number between 0 and 65535");  
}
```



`unwrap()` was a mistake

Useful in examples, dangerous in real applications

`unwrap()` and `expect()` stop the program when the value is `None` or `Err`.



Better error message with `expect()`

Acceptable in small examples or tests

```
#[test]
fn parses_valid_port() {
    let port: u16 = "8080".parse::<u16>().unwrap();
    assert_eq!(port, 8080);
}
```

```
fn main() {
    let config = std::fs::read_to_string("config.toml")
        .expect("config.toml must exist next to the program");
    println!("{config}");
}
```



Use `expect()` only when crashing is acceptable and the message helps debugging.



`unwrap()` was a mistake

Better: handle the error

```
fn main() {
    let text: &str = "42";
    // unwrap: give me the value, or PANIC
    // parse() returns Result<u32, ParseIntError>
    let n: u32 = text.parse().unwrap();
    println!("{n}");           // 42

    // expect: same, but with a message
    let m: u32 = text.parse().expect("not a
number");
    println!("{m}");           // 42

    // if text were "abc", BOTH would crash:
    // thread 'main' panicked: ParseIntError ...
}
```

Better: provide a safe fallback

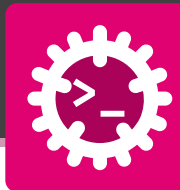


```
fn main() {
    let text = "abc";
    // match: handle BOTH outcomes explicitly
    match text.parse::<u32>() {
        Ok(n) => println!("got {n}"),
        Err(_) => println!("not a number"),
    }

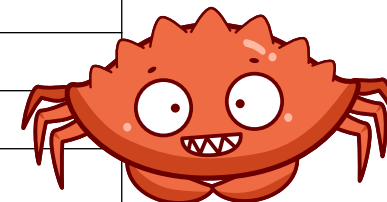
    // if let: act ONLY on success, skip the rest
    if let Ok(n) = text.parse::<u32>() {
        println!("got {n}");
    }

    // unwrap_or: supply a fallback, no crash
    let n = text.parse::<u32>().unwrap_or(0);
    println!("n = {n}");       // n = 0
}
```

Be Careful with `unwrap()` and `expect()`



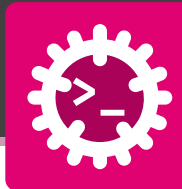
`unwrap()` was a mistake



Method	For	Behaviour
<code>unwrap()</code>	Option/Result	extract value or panic
<code>expect(msg)</code>	Option/Result	panic with custom message
<code>unwrap_or(default)</code>	Option/Result	extract value or use default
<code>unwrap_or_else(f)</code>	Option/Result	extract value or call closure
<code>unwrap_or_default()</code>	Option/Result	extract value or <code>Default::default()</code>
<code>ok_or(err)</code>	Option	convert to Result with error
<code>ok_or_else(f)</code>	Option	convert to Result with closure error
<code>unwrap_err()</code>	Result	extract error or panic
<code>expect_err(msg)</code>	Result	panic with msg if not Err
<code>match</code>	Option/Result	Handle success and failure explicitly
<code>?</code>	Option/Result	Return the error to the caller



`unwrap()` is convenient for examples and quick prototypes, but can crash the program if the value is `None`. Use it with caution.



Named fields

A struct groups related values into a single custom type.

```
#[derive(Debug)]
struct Sensor {
    id: u32,
    value: f64,
    unit: String,
}

fn main() {
    let s = Sensor {
        id: 1,
        value: 23.7,
        unit: String::from("°C"),
    };

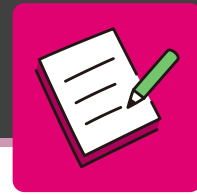
    println!("{:?}", s);
    println!("Sensor {}: {} {}", s.id, s.value, s.unit);
}
```

```
Sensor { id: 1, value: 23.7, unit: "°C" }
Sensor 1: 23.7 °C
```

Key idea

- Create your own type
- Group related data together
- Fields have names and types
- Access fields with .
- Improves readability over tuples

```
sensor.id
sensor.value
sensor.unit
```



Define a small **struct** and write one `Option` and one `Result` function:

1. `Struct Item { name: String, stock: u32 }`.
2. `take(item: &Item) → Option<u32>` - `Some(stock - 1)` if `stock > 0`, otherwise `None`.
3. `parse_qty(s: &str) → Result<u32, String>` - `Ok(n)` if `s` parses as a `u32`, otherwise `Err("not a number")`.
4. In `main`, create an `Item` and print the result of each function.

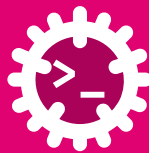


Reach for `Option` when a value **might be missing**, and `Result` when it **might fail**.



Ownership and Borrowing

And what the hell are Lifetimes 'a?



Every value has one owner

In Rust, every value has an **owner**. When the owner goes out of scope, the value is dropped.

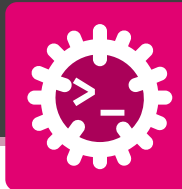
```
fn main() {  
    {  
        let name = String::from("Ferris");  
        println!("{name}");  
    } // name goes out of scope here  
    println!("{name}"); // error  
}
```

Ownership rules

1. Each value in Rust has exactly one owner.
2. There can only be one owner of a value at any given time.
3. When the owner goes out of scope, the value is dropped



Ownership is Rust's way to manage memory safely at compile time.



Ownership can be transferred

Values like String are moved when assigned to another variable.

```
fn main() {  
    let a = String::from("Ferris");  
    let b = a; // ownership moves to b  
    println!("{b}");  
    println!("{a}"); // error  
}
```

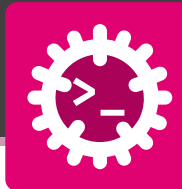
What happens?

```
let b = a;
```

- a no longer owns the string
- b is now the owner
- The data is not copied
- This prevents double-free errors



After a move, the old variable can no longer be used.



Some values are copied instead of moved

Simple values like integers, floats, booleans, and characters are copied.

```
fn main() {  
    let a = 42_u8;  
    let b = a; // copy  
    println!("a = {a}");  
    println!("b = {b}");  
}
```

```
fn main() {  
    let p = (10_u8, 20_u128);  
    let q = p;  
    println!("{:?}", p);  
    println!("{:?}", q);  
}
```

Copy types

- i32, u32, f64
- bool
- char
- Tuples containing only Copy types



Copy values are small and simple.
Rust duplicates them
automatically.



Function calls can move ownership

Passing a non-Copy value to a function transfers ownership.

```
fn print_name(name: String) {  
    println!("{name}");  
}  
  
fn main() {  
    let name = String::from("Ferris");  
    print_name(name);  
    println!("{name}"); // error  
}
```

Why?

- name is moved into print_name
- The function becomes the owner
- At the end of the function, the value is dropped



If a function does not need ownership, pass a reference instead.



Read without taking ownership

A reference lets a function use a value without owning it.

```
fn print_name(name: &String) {  
    println!("{name}");  
}  
fn main() {  
    let name = String::from("Ferris");  
    print_name(&name);  
    println!("{name}"); // still valid  
}
```

Borrowing

- `&String` is a reference
- The function can read the value
- Ownership stays in `main`
- The value is not dropped by the function

```
print_name(&name);  
//           ^  
//           borrow
```




Change without taking ownership

A mutable reference allows a function to modify a value.

```
fn add_unit(text: &mut String) {
    text.push_str(" °C");
}
fn main() {
    let mut value = String::from("23.7");
    add_unit(&mut value);
    println!("{value}");
}
```

Rules

- The variable must be declared `mut`
- The reference must be `&mut`
- Only one mutable reference is allowed at a time

```
let mut value = String::from("23.7");
add_unit(&mut value);
```



Mutable borrowing allows controlled modification without losing ownership.



Many readers or one writer

This prevents data races and invalid references at compile time.

Allowed: many immutable borrows

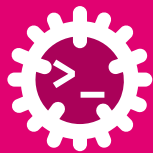
```
fn main() {  
    let value = String::from("23.7 °C");  
    let a = &value;  
    let b = &value;  
    println!("{a}");  
    println!("{b}");  
}
```

Not allowed: read and write at same time

```
fn main() {  
    let mut value = String::from("23.7");  
    let read = &value;  
    let write = &mut value; // error  
    println!("{read}");  
    println!("{write}");  
}
```



Rule of thumb: either many readers or exactly one writer.



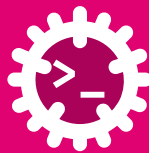
The compiler tracks how long references are valid

A **lifetime** is the span for which a reference stays valid. The **borrow checker** rejects any reference that could outlive the data it points to - so no dangling pointers and no use-after-free, all without a garbage collector.

✗ Dangling reference - rejected at compile time

```
fn main() {  
    let r;  
    {  
        let value = String::from("Ferris");  
        r = &value;           // r borrows value  
    }                         // value dropped here!  
    println!("{r}");          // r would read freed memory  
}
```

```
error[E0597]: `value` does not live long enough  
    r = &value;  
      ^^^^^^ borrowed value does not live long  
enough  
    } - `value` dropped here while still borrowed  
    println!("{r}");  
        — borrow later used here
```



✓ Valid - the reference never outlives its owner

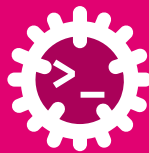
```
fn main() {  
    let value = String::from("Ferris"); //  
    let r = &value;                     // value  
    println!("{r}");                   // r  
}                                       // both end
```

Key idea

- A reference must **not outlive** the value it borrows
- Checked at **compile time only** - zero runtime cost
- Usually **inferred**; explicit 'a syntax shown next



Lifetimes don't change **when** a value is dropped - they let the compiler **prove** every reference is valid for as long as it is used.



Naming a lifetime so the compiler can relate references

Lifetimes are usually **inferred**. You only name one when a function **returns a reference** and the compiler cannot tell which input it borrows from.

```
// 'a links the inputs to the output:  
// "the result lives as long as both inputs do"  
fn longest<'a>(in1: &'a str, in2: &'a str) -> &'a str {  
    if in1.len() > in2.len() { in1 } else { in2 }  
}  
  
fn main() {  
    let s1 = String::from("Ferris");  
    let s2 = String::from("Rust");  
    println!("{}", longest(&s1, &s2)); // Ferris  
}
```

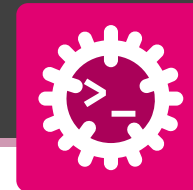
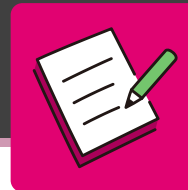
Reading the signature

- <'a> - declare a lifetime named a
- &'a str - “a reference valid for 'a”
- inputs and output **share** 'a, so the returned reference cannot outlive in1 or in2

Think of 'a as a label that links references together



Rule of thumb: if a function takes references and returns one, it usually needs a lifetime to say **which** input the result borrows from.



Each snippet below **does not compile**. Find the **ownership / borrowing problem** and say how you would fix it.

A

```
let s = String::from("hi");
let t = s;
println!("{s} and {t}");
```

B

```
let mut v = vec![1, 2, 3];
let first = &v[0];
v.push(4);
println!("first = {first}");
```

C

```
fn first_word() → &str {
    let s = String::from("hello world");
    s.split(' ').next().unwrap()
}
```

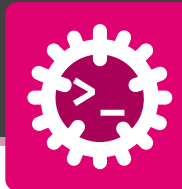


Think about **who owns** the value, **how long** a reference stays valid, and the rule **many readers XOR one writer**.



Composite Types

Let's mix and match!



Grouping several values - Composite types store several values together.

Type	Use case
[T; N]	Fixed-size array
Vec<T>	Growable list
HashMap<K, V>	Fast key-value lookup
BTreeMap<K, V>	Sorted key-value lookup
HashSet<T>	Unique values, fast lookup
BTreeSet<T>	Unique values, sorted
VecDeque<T>	First In First Out (FIFO) queue



Most collection types are generic i.e. support any type.

```
use std::collections::{
    HashMap,
    HashSet,
    BTreeMap,
    BTreeSet,
    VecDeque,
};

fn main() {
    let numbers = [1, 2, 3];           // array
    let names = vec!["Ana", "Bob"];    // vector

    let mut ages = HashMap::new();     // HashMap
    ages.insert("Ana", 21);

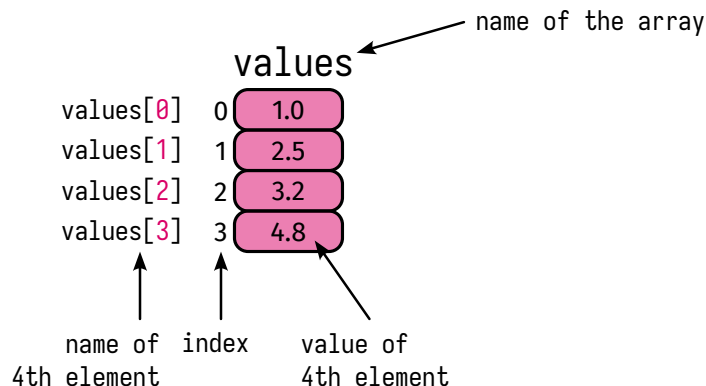
    let mut queue = VecDeque::new();   // VecDeque
    queue.push_back("task 1");
    queue.push_back("task 2");
}
```




[T; N] Fixed size, same type - An array stores a fixed number of values of the same type.

```
fn main() {  
    let values: [f32; 4] = [1.0, 2.5, 3.2, 4.8];  
    println!("length = {}", values.len()); // 4  
    println!("first = {}", values[0]);     // 1  
    println!("last  = {}", values[3]);     // 4.8  
    for value in values {  
        println!("{}", value); // 1  
    }                               // 2.5  
    }                               // 3.2  
    }                               // 4.8
```

```
[i32; 4]  
  |  |  
  |  +-- length  
  +----- element type
```



Array properties

- Fixed length
- Length is part of the type
- Stored contiguously
- Fast indexed access
- Useful for small fixed data

[[T; N]; M] Arrays can contain arrays - A multidimensional array is an array whose elements are also arrays.

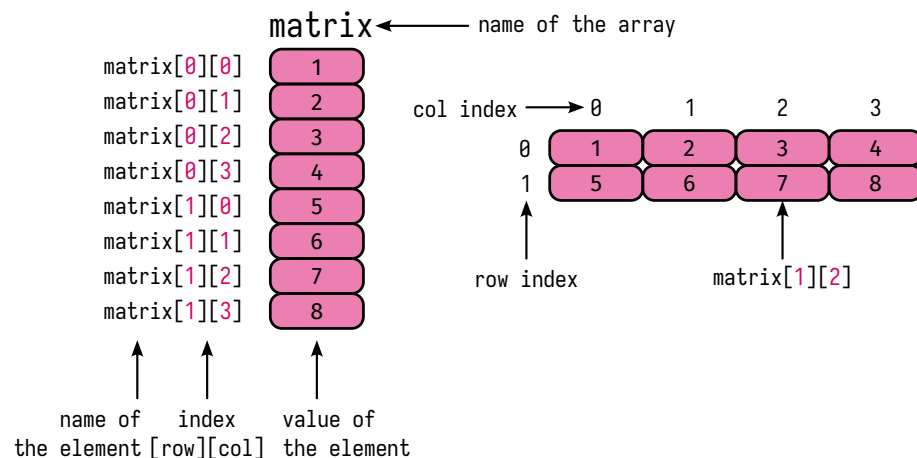
```
fn main() {
    let image: [[u8; 3]; 2] = [
        [255, 128, 0],
        [10, 20, 30],
    ];
    println!("{}", image[0][0]); // 255
    println!("{}", image[1][2]); // 30

    for row in image {
        for pixel in row {
            print!("{}", pixel); // 255 128 0
        }
        println();
    }
}
```

[[u8; 3]; 2]

- rows
- columns
- element type

```
let matrix: [[i32; 4]; 2] = [[1, 2, 3, 4], [5, 6, 7, 8]];
```



Typical use cases

- Small matrices
- Lookup tables
- Image kernels
- Fixed sensor grids

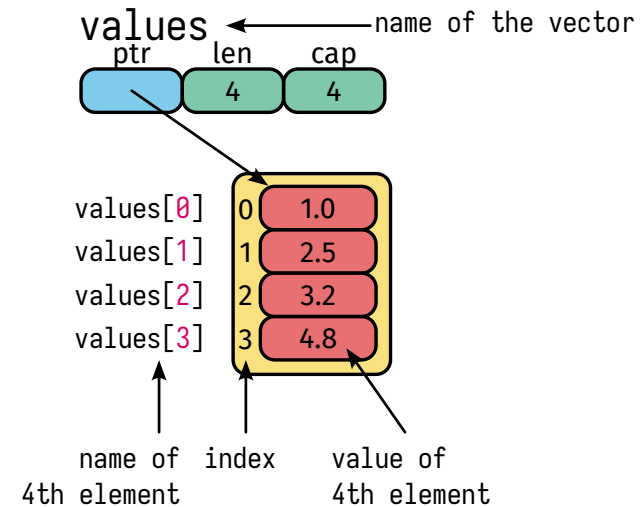


Vec<T> Growable list - A vector stores values of the same type and can grow at runtime.

```
fn main() {  
    let mut readings: Vec<f64> = Vec::new();  
  
    readings.push(23.7);  
    readings.push(24.1);  
    readings.push(23.9);  
  
    println!("len = {}", readings.len());  
    println!("first = {}", readings[0]);  
  
    for value in &readings {  
        println!("{value} °C");  
    }  
}
```

```
let mut values = Vec::new();  
values.push(10);  
values.push(20);
```

```
let values: Vec<f32> = vec![1.0, 2.5, 3.2, 4.8]
```



Vector properties

- Dynamic length
- Fast access by index
- Same element type
- Can add values with push



Safe Access with `get()`, avoiding index panics.

Indexing with `[i]` panics if the index is invalid. `get(i)` returns an `Option`.

```
fn main() {
    let readings = vec![23.7, 24.1, 23.9];
    // Direct indexing
    println!("{}", readings[1]); // 24.1
    // Safe access
    match readings.get(10) {
        Some(value) => println!("{value}"),
        None => println!("no such reading"),
    }
}
```

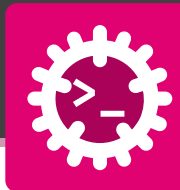
Why Option?

```
let value: Option<f64> = readings.get(10)
```

- `Some(value)` if the index exists
- `None` if the index is invalid



Prefer `get()` when the index may come from user input, files, sensors, or network data.



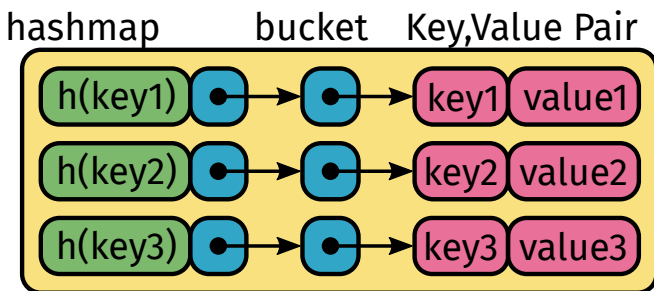
HashMap<K, V> Fast key-value lookup - Stores values under keys.

```
use std::collections::HashMap;

fn main() {
    let mut sensors = HashMap::new();
    sensors.insert("T1", 23.7);
    sensors.insert("T2", 24.1);
    sensors.insert("P1", 1013.2);
    match sensors.get("T1") {
        Some(value) => println!("T1 = {value}"),
        None => println!("sensor not found"),
    }
    for (id, value) in &sensors {
        println!("{id}: {value}");
    }
}
```

```
HashMap<String, f64>
      |           |
      |           | value type
      |           | key type
```

HashMap<K,V>



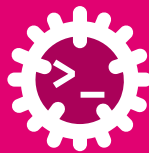
h() hash function

Typical use cases

- Sensor id to value
- Lookup tables
- User id to user data
- Configuration values
- Counting occurrences



- A HashMap is fast, but iteration order is not sorted.
- A bucket is a storage slot in a hash map.



A common real-world pattern - `entry()` is useful when a key may or may not already exist.

```
use std::collections::HashMap;

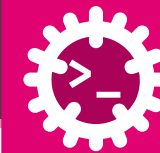
fn main() {
    let events = [
        "open", "close", "open",
        "error", "open", "error",
    ];
    let mut counts = HashMap::new(); // new empty HashMap
    for event in events {
        // if the key doesn't exist, insert it with value 0,
        // then get back a mutable reference to the value
        let counter = counts.entry(event).or_insert(0);
        // this modifies the value inside the HashMap
        *counter += 1;
    }
    println!("{counts:?}");
}
```

```
{
  "open": 3,
  "close": 1,
  "error": 2
}
```

Meaning

```
counts.entry(event).or_insert(0)
```

- Get existing counter
- Or insert 0
- Then increment it via `&mut`



BTreeMap<K, V> Sorted key-value lookup - stores key-value pairs sorted by key.

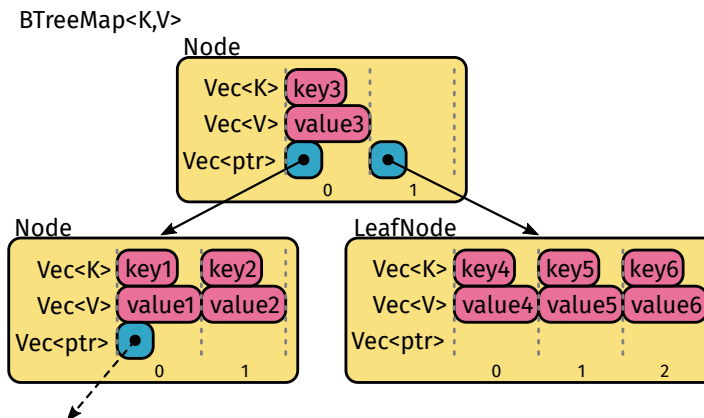
```
use std::collections::BTreeMap;

fn main() {
    let mut measurements = BTreeMap::new();

    measurements.insert("10:00", 23.7);
    measurements.insert("09:00", 22.9);
    measurements.insert("11:00", 24.1);

    for (time, value) in &measurements {
        println!("{time}: {value}");
    }
}
```

```
09:00: 22.9
10:00: 23.7
11:00: 24.1
```

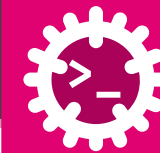


Use BTreeMap when you need

- Sorted keys
- Ordered iteration
- Range queries
- Predictable output order



Use HashMap for fast lookup. Use BTreeMap for ordered data.



Unique values - A HashSet<T> stores each value only once.

```
use std::collections::HashSet;

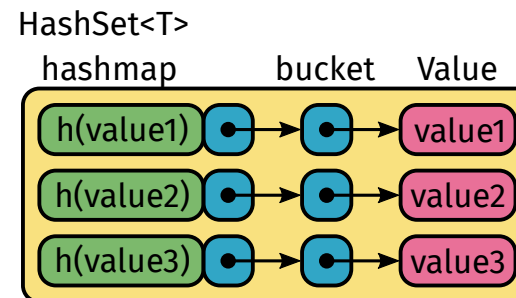
fn main() {
    let mut ids = HashSet::new();

    ids.insert("T1");
    ids.insert("T2");
    ids.insert("T1"); // duplicate

    println!("count = {}", ids.len());

    if ids.contains("T2") {
        println!("T2 exists");
    }
}
```

```
count = 2
T2 exists
```

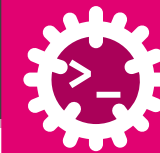


Typical use cases

- Remove duplicates
- Track seen elements
- Membership tests
- Compare groups



A HashSet<T> is basically a HashMap<T, ()> without values.



Unique and sorted values - A BTreeSet<T> stores unique values in sorted order.

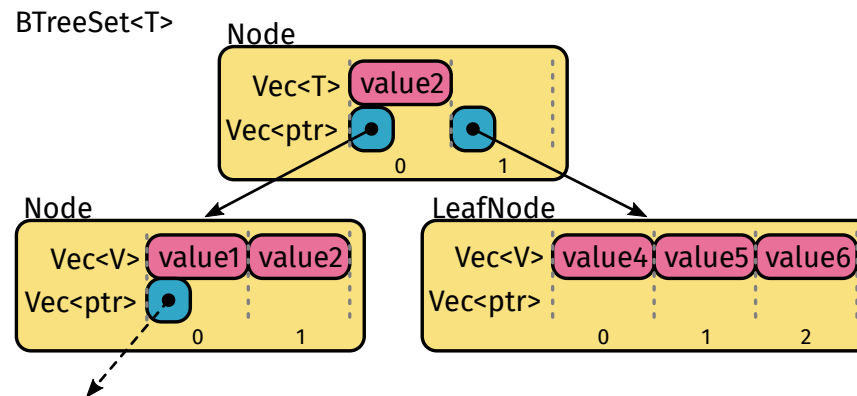
```
use std::collections::BTreeSet;

fn main() {
    let mut ids = BTreeSet::new();

    ids.insert("T3");
    ids.insert("T1");
    ids.insert("T2");
    ids.insert("T1");

    for id in &ids {
        println!("{}", id);
    }
}
```

```
T1
T2
T3
```



Use BTreeSet when you need

- Unique values
- Sorted output
- Deterministic iteration
- Range-like operations



Use HashSet for speed. Use BTreeSet for order.



First in, first out - A queue processes values in the same order as they arrive.

```
use std::collections::VecDeque;

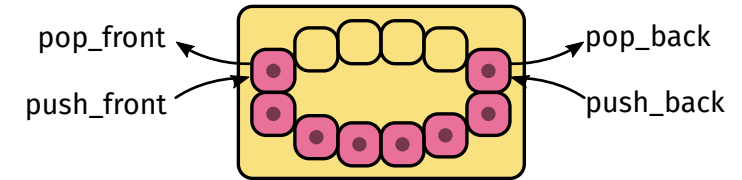
fn main() {
    let mut queue = VecDeque::new();

    queue.push_back("read sensor");
    queue.push_back("filter value");
    queue.push_back("send packet");

    while let Some(task) = queue.pop_front() {
        println!("processing: {task}");
    }
}
```

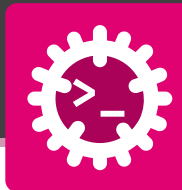
```
processing: read sensor
processing: filter value
processing: send packet
```

Double Ended Queue VecDeque<T>



Queue operations

push_front(x)	add at the front
push_back(x)	add at the end
pop_front()	remove from the front
pop_back()	remove from the back
front()	read first item
back()	read last item



Practical rule of thumb

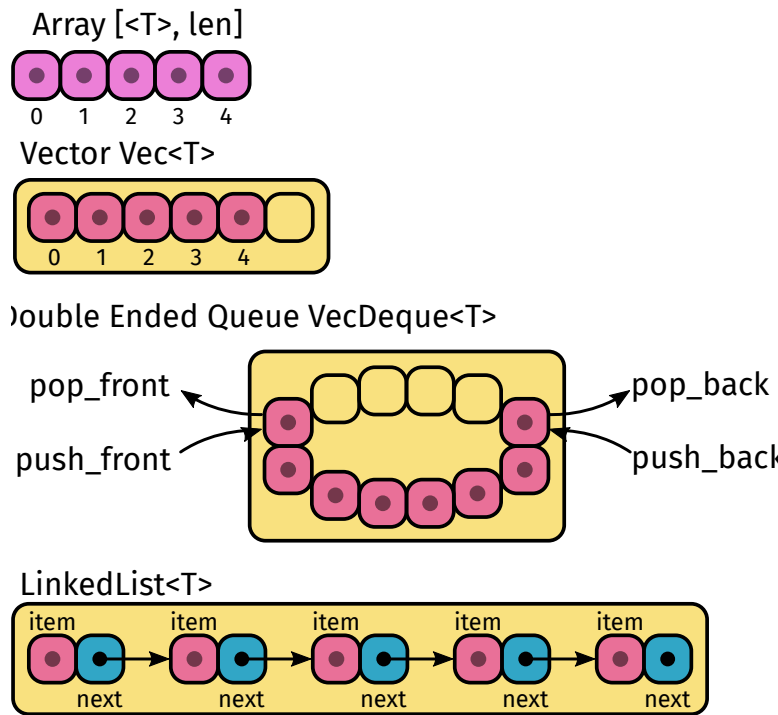
Need	Use
Fixed number of values	Array [T; N]
Growable list	Vector Vec<T>
Fast lookup by key	HashMap<K, V>
Sorted lookup by key	BTreeMap<K, V>
Unique values	HashSet<T>
Unique sorted values	BTreeSet<T>
FIFO queue	VecDeque<T>



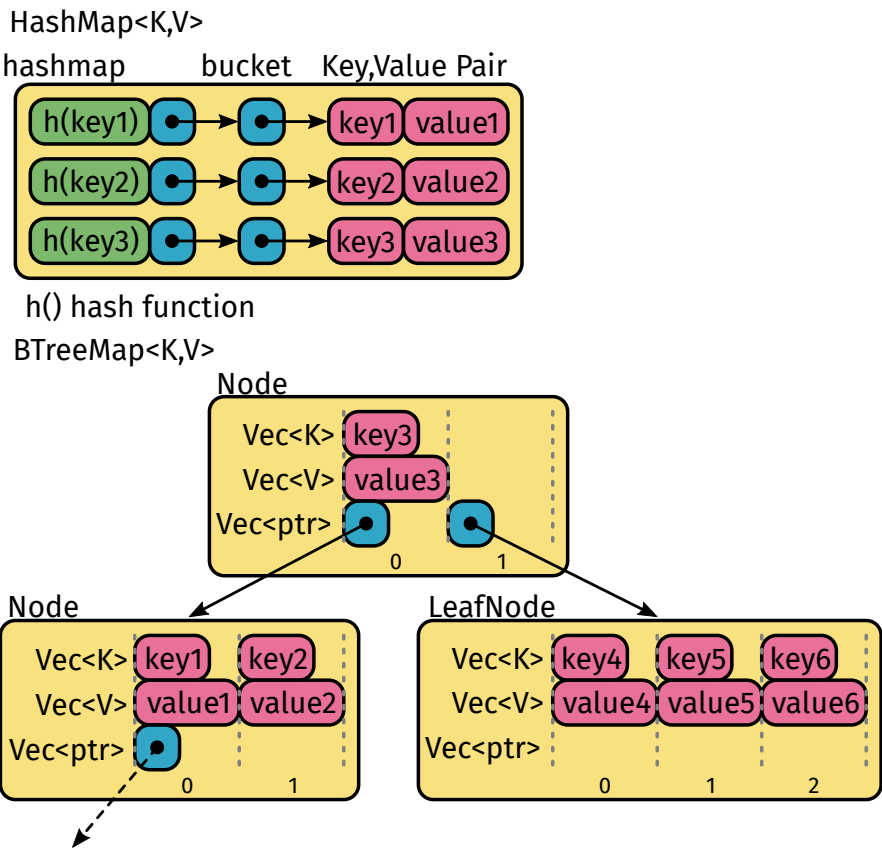
Start with Vec<T>. Use maps, sets, and queues when the access pattern requires them.

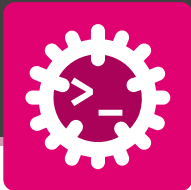


Sequences



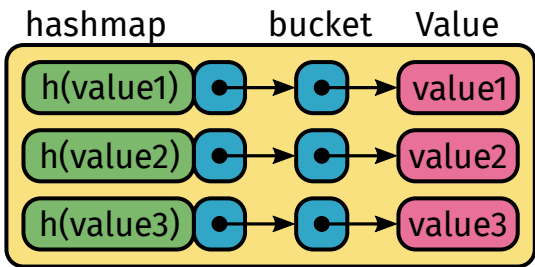
Maps



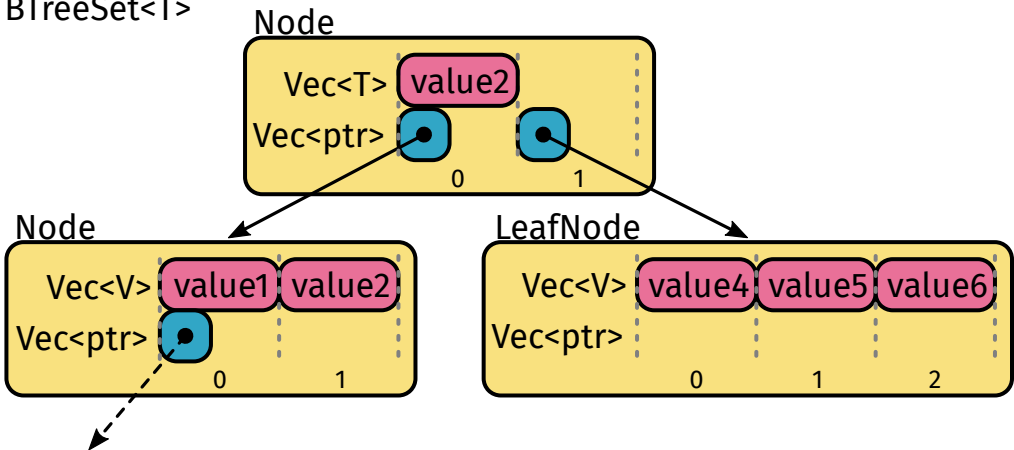


Sets

HashSet<T>

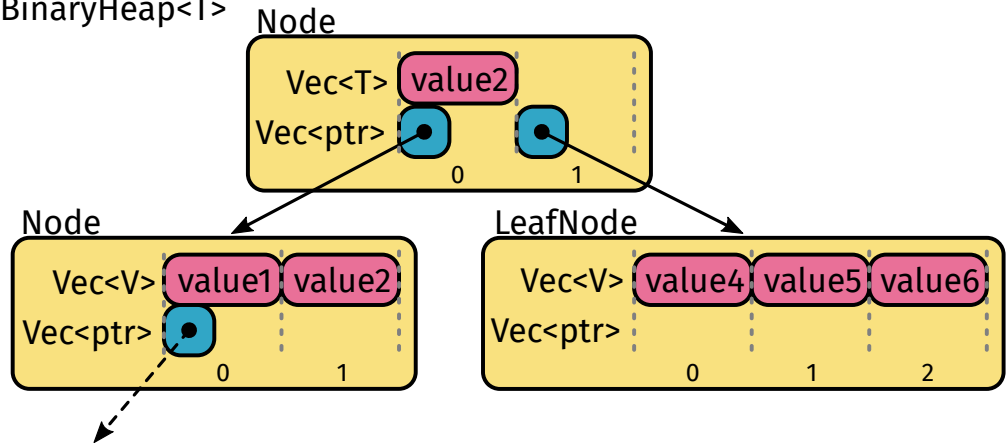


BTreeSet<T>



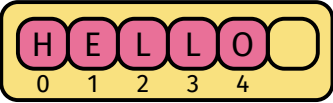
🔗 Misc

BinaryHeap<T>

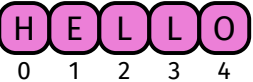




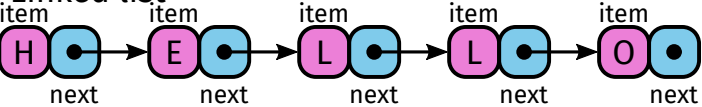
Vector



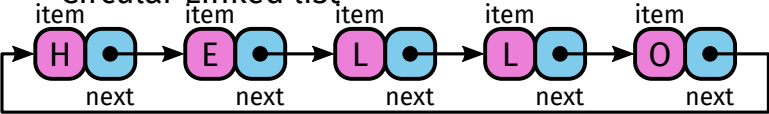
Array



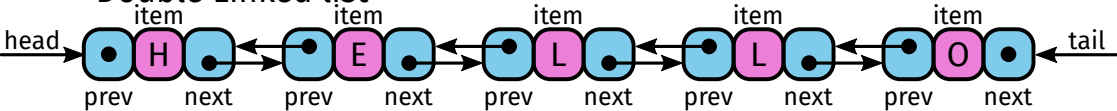
Linked list



Circular Linked list

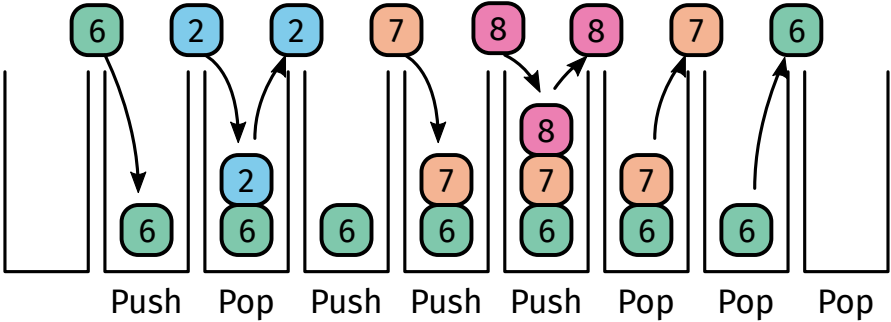


Double Linked list



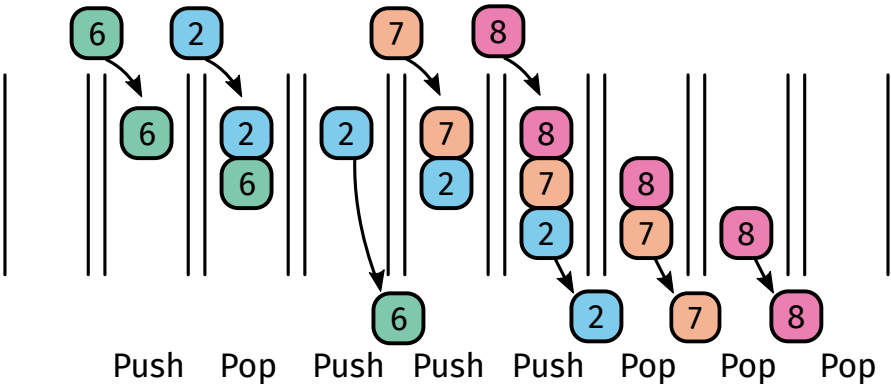
Stack

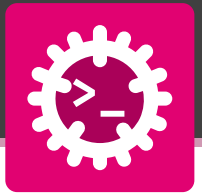
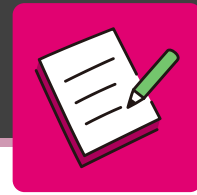
LIFO - Last In First Out



Queue

FIFO - First In First Out





Read the program and write the value printed for a, b, c and d.

```
use std::collections::HashMap;

fn main() {
    let votes = vec!["red", "blue", "red",
                     "green", "red", "blue"];
    let total = votes.len();

    let mut tally: HashMap<&str, u32> = HashMap::new();
    for v in votes {
        *tally.entry(v).or_insert(0) += 1;
    }

    println!("a {}", total);           // a
    println!("b {}", tally.len());     // b
    println!("c {}", tally["red"]);    // c
    println!("d {:?}", tally.get("yellow")); // d
}
```



`entry(k).or_insert(0)` returns a `&mut` to the value, inserting 0 first if the key is new.

`tally.get(k)` returns an `Option`, while `tally[k]` panics on a missing key.



Object-Oriented Programming

Teaching data to behave



An **object** bundles **state** (data) with the **behaviour** (operations) that acts on that data. A program becomes a set of objects collaborating through their public operations.

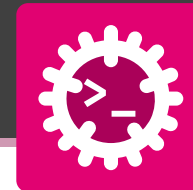
Procedural view - data and the functions on it live apart:

```
struct Account { balance: u64 }  
// free functions, anyone can touch the field  
fn deposit(a: &mut Account, n: u64) {  
    a.balance += n;  
}  
fn balance(a: &Account) → u64 {  
    a.balance  
}
```

Object view - data and behaviour belong together:

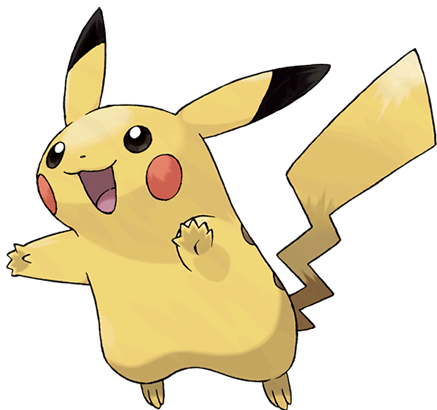
```
struct Account { balance: u64 }  
impl Account {  
    // behaviour attached  
    fn deposit(&mut self, n: u64) { self.balance += n; }  
    fn balance(&self) → u64 { self.balance }  
}
```

- Same data, but the operations now **belong to the type**
- This bundling is the seed of every pillar that follows



A **class** is the blueprint - the **type** and its behaviour. An **object** is one concrete **instance** built from that blueprint.

Object



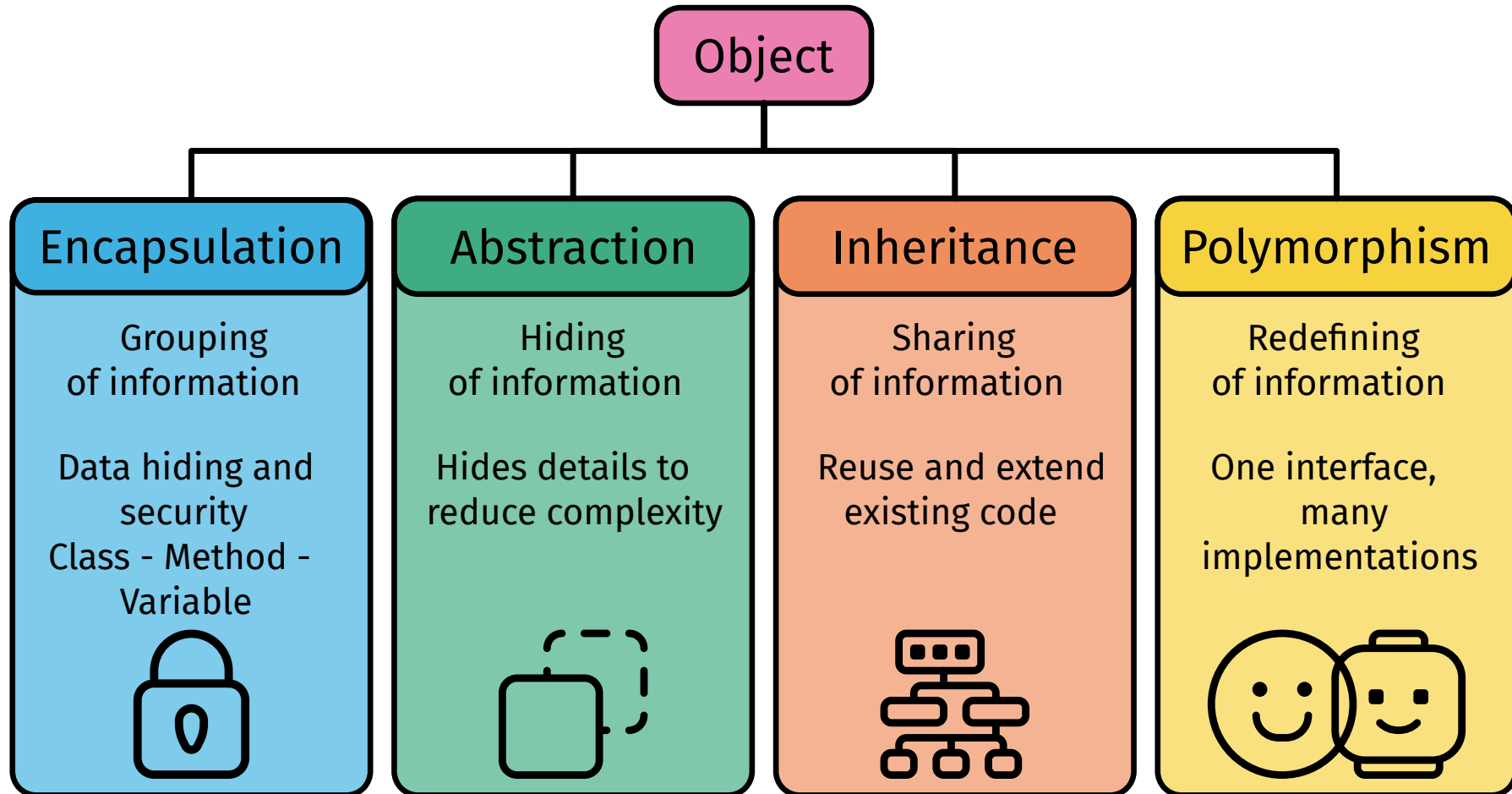
Class

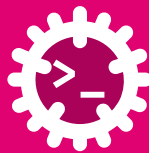
C Pokemon	
Attributes	
□ name: String	
□ type: String	
□ health: Integer	
□ attack: Integer	
□ defense: Integer	
□ speed: Integer	
Methods	
● speak(): String	
● attack(target: Pokemon): Void	
● takeDamage(amount: Integer): Void	
● dodge(): Boolean	
● evolve(): Pokemon	

Attributes

Methods

- **Class** - written once: Pokemon defines what every Pokémon **is** and **can do**
- **Object** - created many times: pikachu, charmander, ... each with its own field values
- All objects of a class **share the same behaviour** but hold **independent state**

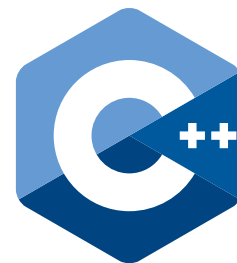




Classic OOP rests on four pillars.

C++ supports all four directly through the class model:

- **Encapsulation** $\Rightarrow$ class + private / public access specifiers
- **Abstraction** $\Rightarrow$ abstract classes with pure virtual functions
- **Inheritance** $\Rightarrow$ public inheritance (class Dog : public Animal)
- **Polymorphism** $\Rightarrow$ templates (compile-time) + virtual functions (run-time)

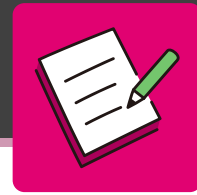


Rust keeps the **goals** but changes the **tools**:

- **Encapsulation** $\Rightarrow$ modules + pub visibility
- **Abstraction** $\Rightarrow$ trait (the contract)
- **Inheritance** $\Rightarrow$ *politely declined* - replaced by traits + composition
- **Polymorphism** $\Rightarrow$ generics (static dispatch) + trait objects (dynamic dispatch)

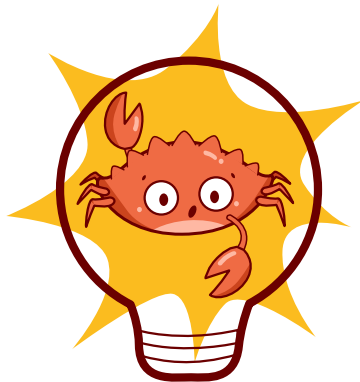


We meet each pillar in turn, then recap the full mapping at the end.



A smart **Lamp** can be switched on or off and dimmed from 0 to 100 %.

1. List its **state** - the data the object holds.
2. List its **behaviour** - the operations acting on that data.
3. Which part becomes the `struct`, which becomes the `impl`?



Building a Class in Rust

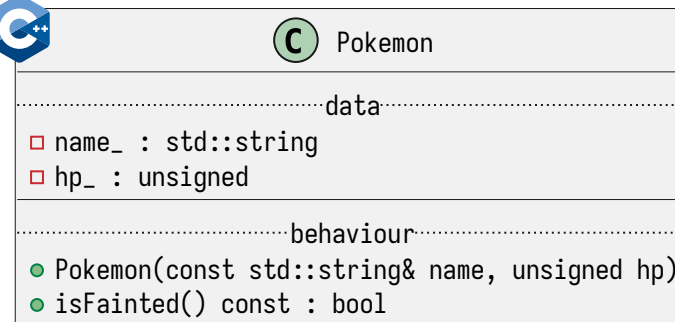
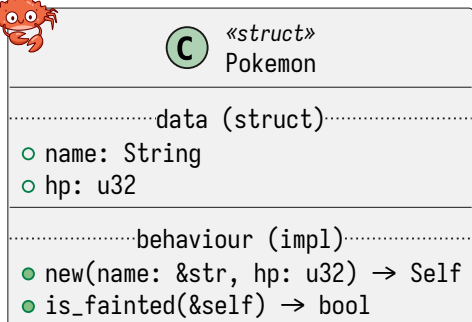
struct for data / impl for behaviour



A class splits into two halves: a struct holds the **data**, an impl block holds the **behaviour**.

```
struct Pokemon { name: String, hp: u32 }
// inherent behaviour can be split across blocks
impl Pokemon {
    fn new(name: &str, hp: u32) -> Self {          // construction
        Self { name: name.into(), hp }
    }
}
impl Pokemon {
    fn is_fainted(&self) -> bool { self.hp == 0 } // queries
}
```

- **Field / attribute** $\Rightarrow$ a member of the struct
- **Method** $\Rightarrow$ a function in an impl block whose first parameter is **self**
- A type may have **several** impl blocks - group behaviour as you like
- Data and behaviour are written **separately** but belong to the same type



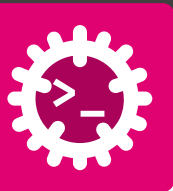


An object is a **value** of the type. Create one, then call its methods.

```
fn main() {  
    // two independent objects of the same class  
    let pikachu = Pokemon {  
        name: "Pikachu".into(), hp: 35  
    };  
    let charmander = Pokemon::new("Charmander", 39);  
    println!("{}", pikachu.name);           // own state  
    println!("{}", charmander.is_fainted()); // shared behaviour  
}
```

- One **class**, many **objects** - each carries its **own** field values
- All objects share the **same** methods

```
// The same as calling the method on the object  
pikachu.is_fainted()  
// Is equivalent to the fully qualified call  
Pokemon::is_fainted(&pikachu)
```

The first parameter - the **receiver** - decides whether a method **reads**, **modifies**, or **consumes** the object.

```
struct Counter { value: u32 }

impl Counter {
    fn get(&self) -> u32 { self.value }           // read
    fn increment(&mut self) { self.value += 1; } // modify
    fn into_value(self) -> u32 { self.value }      // consume
}

fn main() {
    let mut c = Counter { value: 0 };
    c.increment();           // borrows &mut c
    println!("{}", c.get()); // borrows &c  arrow.stroked1
    let v = c.into_value();  // moves c
    // c.get();              // × c was consumed
    println!("{}", v);       // 1
}
```



«struct»
Counter

○ value: u32

● get(&self) -> u32

● increment(&mut self)

● into_value(self) -> u32

&self	borrow to read
&mut self	borrow to modify
self	take ownership (consume)
none	associated fn $\Rightarrow$ Type::f()

- Self (capital) - alias for the type
- Prefer &self; use self only when the value must be consumed



Counter

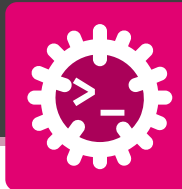
□ value : unsigned

● Counter(unsigned value = 0)

● get() const : unsigned

● increment() : void

● intoValue() && : unsigned



A **constructor** builds a value and puts it in a **valid state**. Rust has **no constructor keyword** - by convention an **associated function** (no `self`) named `new` returns `Self`.

- **Constructor** $\Rightarrow$ associated fn returning `Self`, called `Type::new()`
- Offer **several** named constructors (`origin`, `from_tuple`, ...)
- **Fallible** construction $\Rightarrow$ return `Result<Self, E>` instead of throwing
- No hidden default constructor - you write the ones you want

```
struct Point { x: i32, y: i32 }
impl Point {
    fn new(x: i32, y: i32) → Self { Self { x, y } }
    fn origin() → Self { Self { x: 0, y: 0 } } // named constructor
    // fallible construction returns a Result
    fn try_new(x: i32, y: i32) → Result<Self, String> {
        if x ≥ 0 && y ≥ 0 { Ok(Self { x, y }) }
        else { Err("coords must be ≥ 0".into()) }
    }
}

fn main() {
    let a = Point::new(1, 2); // Type::fn() ⇒ associ
    let b = Point::origin();
    let c = Point::try_new(-1, 0); // Err(...)
}
```



«struct»
Point

○ x: i32
○ y: i32

—associated fns (no self)—

● new(x: i32, y: i32) → Self
● origin() → Self
● try_new(x: i32, y: i32) → Result<Self, String>



Point

□ x : int
□ y : int

—constructors / factories—

● Point(int x, int y)
● origin() : Point
● create(int x, int y) : std::optional<Point>



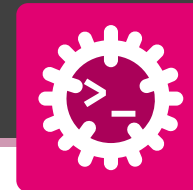
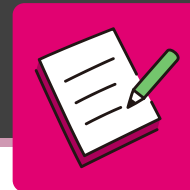
For a **no-argument** constructor, Rust offers a standard Default trait - usually just **derived**.

```
#[derive(Default, Debug)]
struct Config {
    verbose: bool,    // → false
    retries: u32,     // → 0
}
// Custom default when 0 and false aren't right
struct ServerConfig { port: u16, host: String }
impl Default for ServerConfig {
    fn default() → Self {
        Self { port: 8080, host: "localhost".into() }
    }
}
fn main() {
    let c = Config::default();
    println!("{c:?}"); // verbose: false, retries: 0

    // struct-update: set a few fields, default the rest
    let c2 = Config { retries: 3, ..Default::default() };
    println!("{c2:?}"); // verbose: false, retries: 3

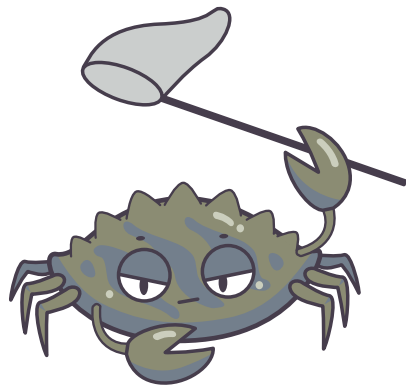
    let s = ServerConfig::default();
    println!("{s:?}"); // localhost:8080
}
```

- `#[derive(Default)]` works when **every field** implements Default:
 - numbers $\Rightarrow$ 0
 - bool $\Rightarrow$ false
 - String/Vec $\Rightarrow$ empty
- Call it for free as `Type::default()`
- **Struct update** `..Default::default()` fills the untouched fields
- Need a non-zero default? Implement Default by hand - a **trait impl**, which we meet under Abstraction



Build the Lamp class:

1. A struct `Lamp` with fields `on: bool` and `brightness: u32`.
2. A constructor `new()` returning a lamp that is **off** at brightness 0.
3. Methods `turn_on(&mut self)` (sets `on = true`) and `is_on(&self) → bool`.
4. In `main`, create a lamp, turn it on and print `is_on()`.



Pillar 1 - Encapsulation

Hide the wiring, expose the buttons

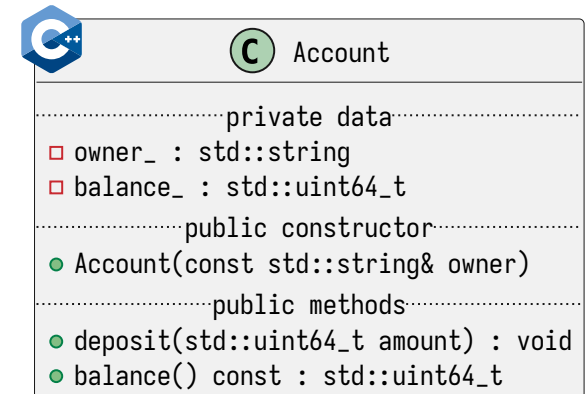
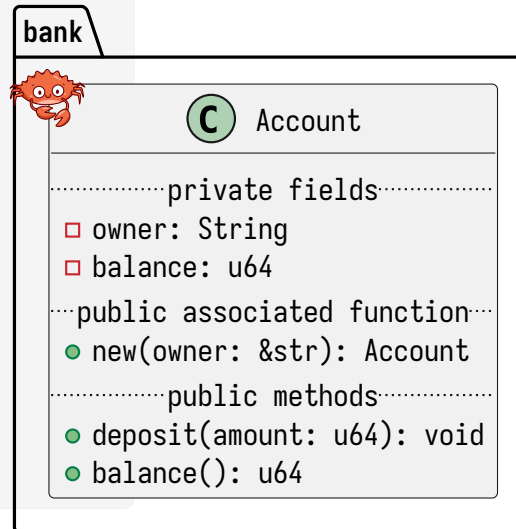


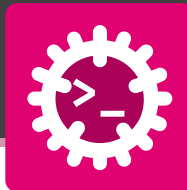
Keep fields **private**; let callers change state only through a small **public** set of methods.

```
mod bank {
  pub struct Account {
    owner: String, // private - hidden outside `bank`
    balance: u64,
  }
  impl Account {
    pub fn new(owner: &str) → Self {
      Self { owner: owner.into(), balance: 0 }
    }
    pub fn deposit(&mut self, amount: u64) {
      self.balance += amount; // mutate via method
    }
    pub fn balance(&self) → u64 { self.balance }
  }
}

fn main() {
  let mut acc = bank::Account::new("Ada");
  acc.deposit(100);
  println!("{}", acc.balance()); // 100
  // acc.balance += 1; // × field is private
}
```

- **Encapsulation** ⇒ bundle data + methods, expose only a public API
- **Visibility** ⇒ pub opt-in, **private by default**
- **Data protection** ⇒ outside code cannot corrupt balance directly

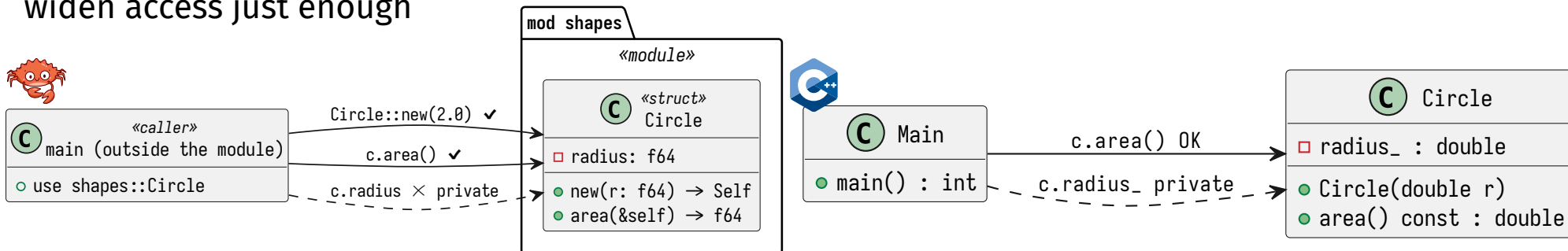




Privacy in Rust is enforced by the **module**, not by the class.

- **Private by default** - an item is visible only inside its own **module**
- **pub** opens it up - applies to a struct, an fn, a single **field**, or a trait
- A **pub struct** with **private fields** exposes the **type** but hides its **data** - callers must go through methods
- Hiding internals lets you **change them later** without breaking callers
- Finer control exists: **pub(crate)**, **pub(super)** to widen access just enough

```
mod shapes {  
    pub struct Circle {           // type is public  
        radius: f64,             // field stays private  
    }  
    impl Circle {  
        pub fn new(r: f64) → Self { Self { radius: r } }  
        pub fn area(&self) → f64 { 3.14159 * self.radius * self.radius }  
    }  
} // end of the module shapes  
  
use shapes::Circle;  
let c = Circle::new(2.0); // OK  
// let r = c.radius;      // × private field
```





Private fields plus a **getter** (read) and a **setter** (write) let the type **enforce its invariants**.

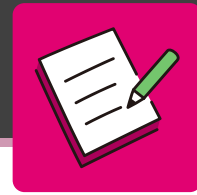
```
pub struct Temperature { celsius: f64 } // private field
impl Temperature {
    pub fn new(celsius: f64) → Self { Self { celsius } }
    // getter - Rust idiom: name it after the field
    pub fn celsius(&self) → f64 { self.celsius }
    // setter - validate before mutating
    pub fn set_celsius(&mut self, value: f64) {
        if value ≥ -273.15 { // physical lower bound
            self.celsius = value;
        }
    }
}

fn main() {
    let mut t = Temperature::new(20.0);
    t.set_celsius(25.0); println!("{}", t.celsius()); // 25
    t.set_celsius(-300.0); // rejected, stays 25
}
```

- **Getter** ⇒ read access, named after the field (**no** `get_` prefix in Rust)
- **Setter** ⇒ `set_xxx(&mut self, ...)`, the place to **validate**
- Protects the **invariants** the struct must always uphold
- Return `&T` (not a clone) when the field is large



Don't add getters/setters by reflex -
expose only what the API truly needs.



Make the brightness **impossible** to set out of range 0..=100.

1. Keep the field brightness **private**.
2. Add a getter `brightness(&self) → u32`.
3. Add a setter `set_brightness(&mut self, pct: u32)` that **clamps** anything above 100 to 100.
4. Show that writing `lamp.brightness = 250`; from outside would **not compile**.

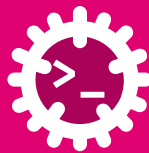


A private field plus a validating setter is how a type **protects its invariants**.



Pillar 2 - Abstraction

Program to ideas, not details



A **trait** states **what** a type can do; each type decides **how**. It is Rust's **interface**.

```
// the abstraction: a contract, no data
trait Shape {
    fn area(&self) → f64;           // required (abstract method)
    fn name(&self) → &str { "shape" } // default method
}

struct Circle { r: f64 }
struct Rect   { w: f64, h: f64 }

impl Shape for Circle {
    fn area(&self) → f64 { 3.14159 * self.r * self.r }
    fn name(&self) → &str { "circle" } // override default
}

impl Shape for Rect {
    fn area(&self) → f64 { self.w * self.h } // keep default name
}
```

- **Interface / contract** $\Rightarrow$ the trait's **required** methods
- **Abstract method** $\Rightarrow$ a required method, no body
- **Default method** $\Rightarrow$ a method **with** a body; types may override it
- **Implementation** $\Rightarrow$ `impl Trait for Type`
- Each type supplies its **own** `area()` - the **how** is hidden behind the **what**

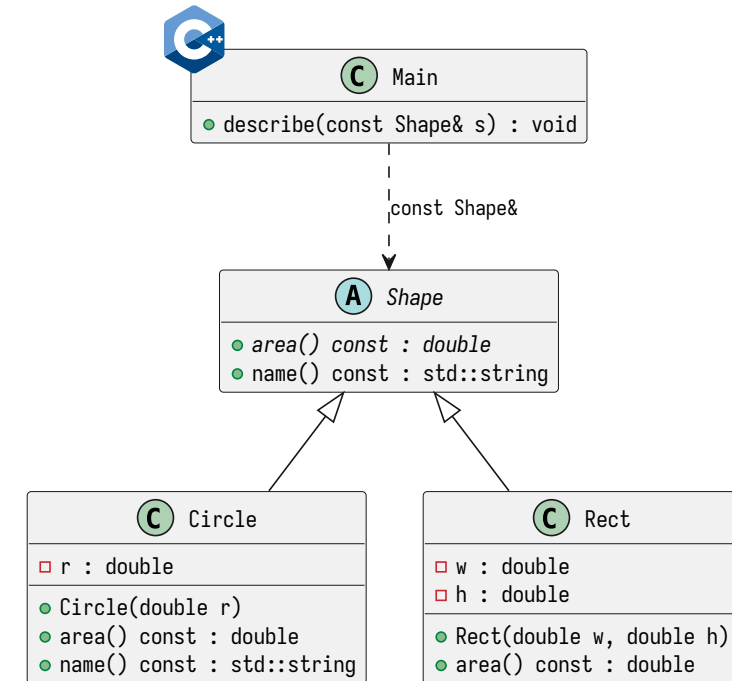
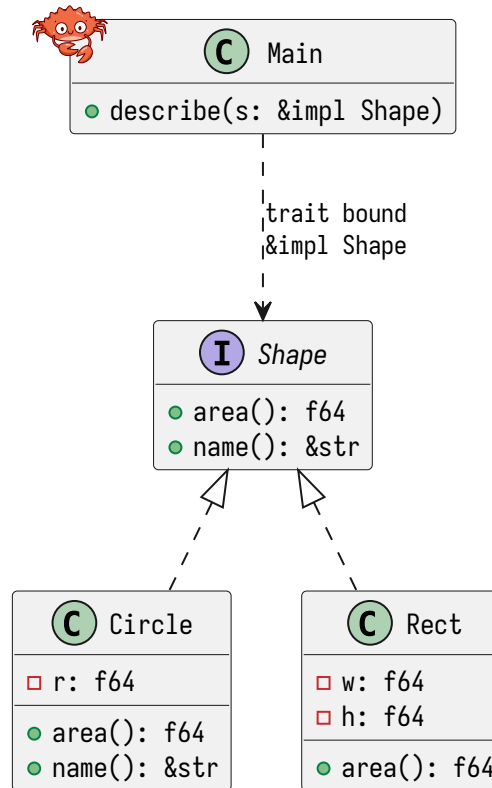


Write functions against the **trait**, not the concrete type - any present or future implementor fits.

```
// the caller knows only the contract `Shape`
fn describe(s: &impl Shape) {
    println!("{}", area = {:.2}", s.name(), s.area());
}

fn main() {
    let c = Circle { r: 2.0 };
    let r = Rect { w: 3.0, h: 4.0 };
    describe(&c); // circle area = 12.57
    describe(&r); // shape area = 12.00
}
```

The caller depends on the **trait**, not the types $\Rightarrow$ **open for extension**: add a new Shape and describe just works.





`&impl Shape` is **shorthand** for a generic with a trait bound - same machinery, shorter spelling.

Shorthand - `impl Trait` in the argument:

```
fn describe(s: &impl Shape) {  
    println!("{}", s.name(), s.area());  
}
```

Desugared - an explicit generic + trait bound:

```
fn describe<T: Shape>(s: &T) {  
    println!("{}", s.name(), s.area());  
}
```

The compiler treats these as **identical**.

- `&impl Shape` reads as “**a reference to some type that implements Shape**”
- The concrete type is chosen **per call site**, resolved at **compile time** (static dispatch)
- **Trade-off:** `impl Trait` can't be **named**, so it can't tie two arguments to the **same** type:

```
// two DIFFERENT Shapes allowed:  
fn pair(a: impl Shape, b: impl Shape)  
// forces a and b to the SAME type:  
fn pair<T: Shape>(a: T, b: T)
```

- Contrast with `&dyn Shape` - a **trait object**, resolved at **runtime** (covered under Polymorphism)



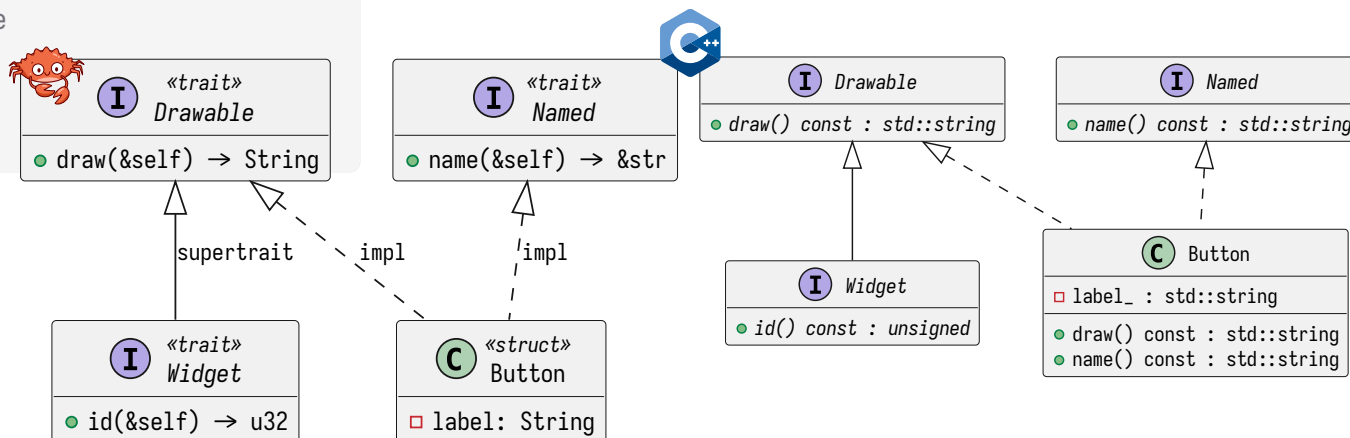
A type can keep **many** contracts at once, and traits can **build on** other traits.

```
trait Drawable { fn draw(&self) → String; }
trait Named    { fn name(&self) → &str; }

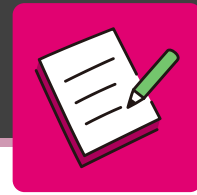
struct Button { label: String }

// one type implements several interfaces
impl Drawable for Button {
    fn draw(&self) → String { format!("Button({})", self.label) }
}
impl Named for Button {
    fn name(&self) → &str { &self.label }
}

// supertrait: a Widget must ALSO be Drawable
trait Widget: Drawable {
    fn id(&self) → u32;
}
```



- **Multiple interfaces** $\Rightarrow$ implement many traits per type (cf. multiple inheritance - but of **behaviour**, never of **data**)
- **Supertrait** $\Rightarrow$ trait `Widget: Drawable` requires `Drawable` too
- **Dependency inversion** $\Rightarrow$ depend on trait bounds, not concrete types

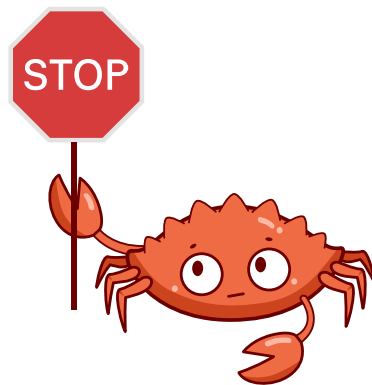


Give different devices **one shared contract** with a trait.

1. Define trait Device { fn status(&self) → String; }.
2. Implement it for Lamp ("lamp on" / "lamp off") and for Thermostat ("22 °C").
3. Write report(d: &impl Device) that prints the status.
4. Call report with one lamp and one thermostat.



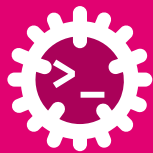
Writing report against the **trait** means any future Device works without changing it.



Pillar 3 - Inheritance

The pillar Rust politely declines

Rust Has No Class Inheritance



There is **no** `struct Dog : Animal`. Shared **behaviour** comes from **traits with default methods**; shared **data** comes from **composition**.

```
// ✗ not valid Rust:
// struct Dog : Animal { ... }

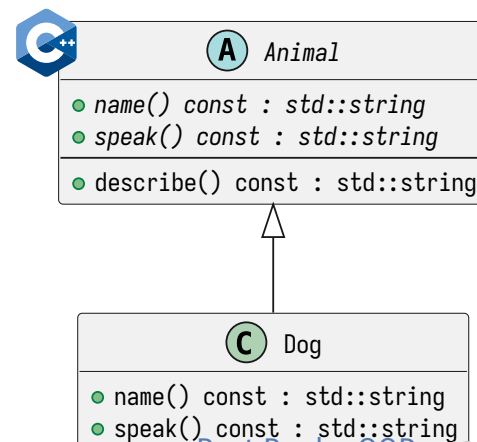
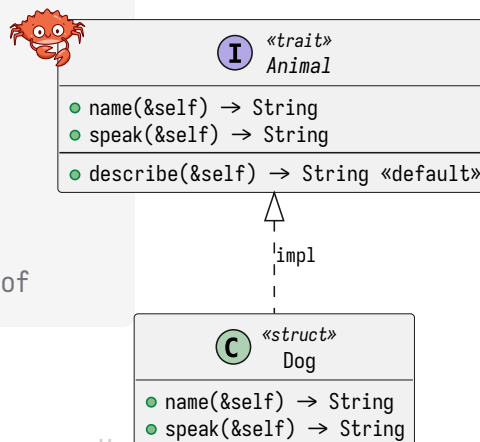
// ✓ a trait + a default method = shared behaviour
trait Animal {
    fn name(&self) → String;
    fn speak(&self) → String;
    fn describe(&self) → String {           // "inherited" behaviour
        format!("{}", self.name(), self.speak())
    }
}

struct Dog;
impl Animal for Dog {
    fn name(&self) → String { "Dog".into() }
    fn speak(&self) → String { "woof".into() }
}

fn main() { println!("{}", Dog.describe()); } // Dog says woof
```

Mapping the inheritance vocabulary:

- **Base class** $\Rightarrow$ a trait
- **Derived class** $\Rightarrow$ a type that impls it
- **Override** $\Rightarrow$ the type's own method body
- **Extend** $\Rightarrow$ implement **more** traits
- **Reuse** $\Rightarrow$ default methods - **behaviour only**, never inherited fields





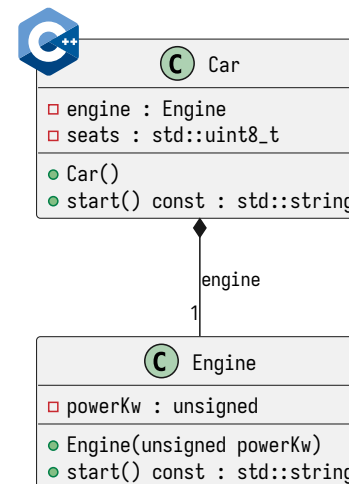
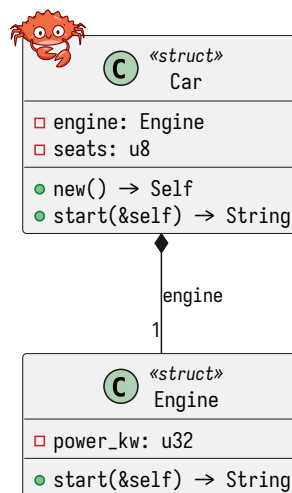
Instead of inheriting, a type contains other types (**has-a**) as fields and **delegates** work to them - reusing their **data and behaviour** without subclassing. e.g. Car **has-a** Engine.

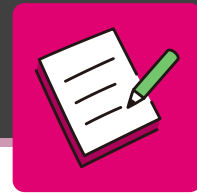
```
struct Engine { power_kw: u32 }
impl Engine {
    fn start(&self) → String {
        format!("{}", kW engine running", self.power_kw)
    }
}

struct Car {
    engine: Engine,    // has-a Engine
    seats: u8,
}
impl Car {
    fn new() → Self {
        Self { engine: Engine { power_kw: 150 }, seats: 5 }
    }
    // delegation: forward to the inner part
    fn start(&self) → String { self.engine.start() }
}

fn main() {
    println!("{}", Car::new().start());
}                                     // 150 kW engine running
```

- **Has-a relationship** ⇒ fields hold other types
- **Delegation** ⇒ forward calls to an inner field
- **Reuse** ⇒ compose small parts instead of inheriting
- **Flexibility** ⇒ swap the inner type
- **Loose coupling** ⇒ depend on traits, not parents





Rust has no class inheritance - reuse a Lamp through **composition**.

1. A Room **has-a** Lamp (a field) plus a name: String.
2. `Room::new(name)` builds a room whose lamp is **off**.
3. **Delegate**: `Room::turn_on(&mut self)` forwards to the lamp's `turn_on()`.
4. In main, build a room, turn it on, print whether its lamp is on.



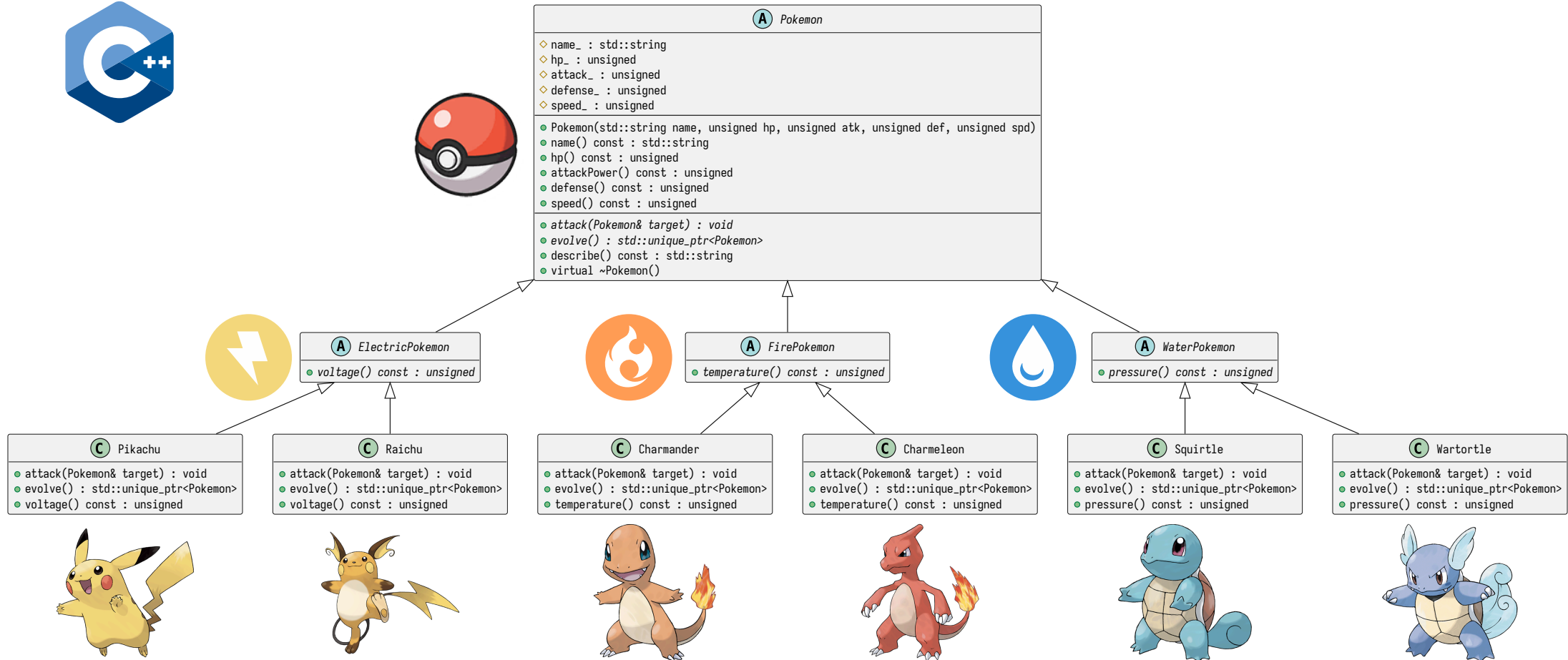
Has-a (a field), not **is-a** (a subclass): the room **delegates** to the lamp it owns.



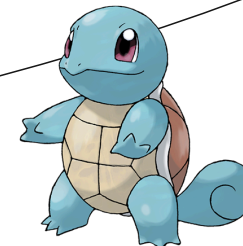
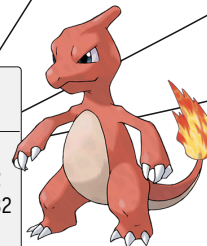
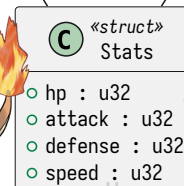
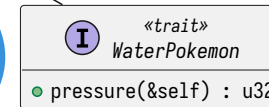
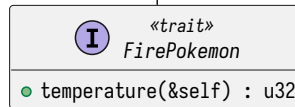
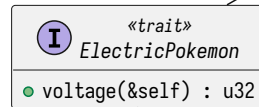
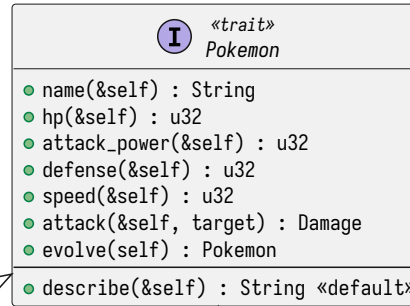
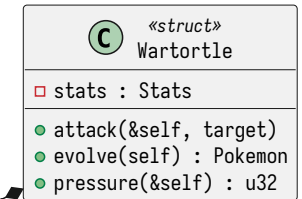
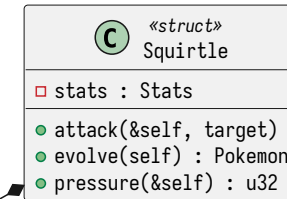
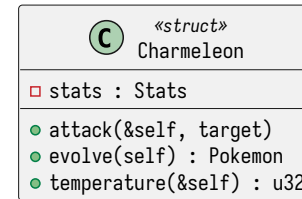
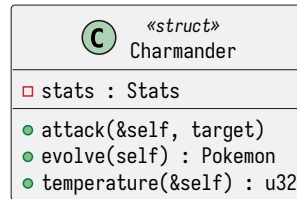
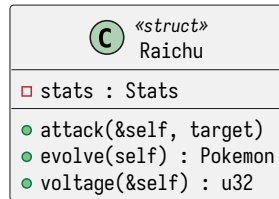
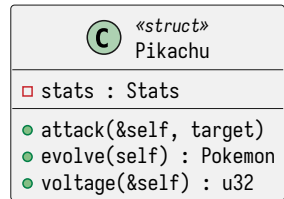
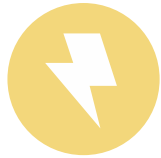
Pillar 4 - Polymorphism

One name, many shapes

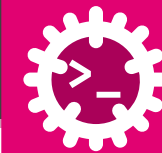
Polymorphism: One Name, Many Types



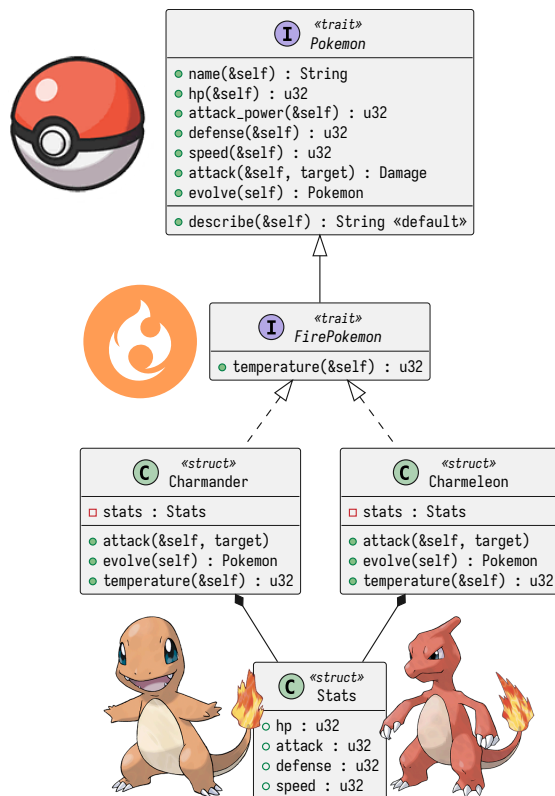
Polymorphism: One Name, Many Types



Polymorphism: One Name, Many Types



The **same** call works for **any** type that implements the trait. Rust resolves it two ways - at **compile time** or at **run time**.



- **One call, many types** - write `p.attack(t)` once a Pikachu thunderbolts, a Charmander embers
- Rust resolves that one call in one of **two ways**:
 - **Static** (compile time) $\Rightarrow$ **generics + trait bounds** `T: Pokemon`
 - **Dynamic** (run time) $\Rightarrow$ **trait objects** `dyn Pokemon`
- The call **looks identical** - only **how the right method is found** differs
- In any case: **substitutability**, one function serves every species



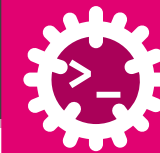
A **generic** with a **trait bound** `T`: `Pokemon` is compiled into one specialised copy **per concrete type**, resolved at **compile time**.

```
trait Pokemon { fn hp(&self) → u32; }
struct Pikachu { hp: u32, }
impl Pokemon for Pikachu {
    fn hp(&self) → u32 {
        self.hp
    }
}
// means T can be any type that implements Pokemon
fn total_hp<T: Pokemon>(team: &[T]) → u32 {
    team.iter().map(|p| p.hp()).sum()
}
fn main() {
    // a slice holds ONE species, so T is fixed to Pikachu
    let squad = [
        Pikachu { hp: 35 }, Pikachu { hp: 40 },
    ];
    let total = total_hp(&squad); // → total_hp::<Pikachu>
    println!("{}", total);      // 75
}
```

- **Trait bound** `T`: `Pokemon` constrains the generic
- **Monomorphization** - one copy generated per species used:

```
// conceptually generated:
fn total_hp_pikachu(&[Pikachu]) → u32 {...}
fn total_hp_charmander(&[Charmander]) {...}
```

- Calls **inlined** $\Rightarrow$ fastest, no indirection
- Cost: **larger** binary; a slice holds **one** species only
- Similar to C++ **templates**



A **trait object** `dyn Pokemon` hides the concrete species behind a pointer; the right method is found at **run time** via a **vtable**.

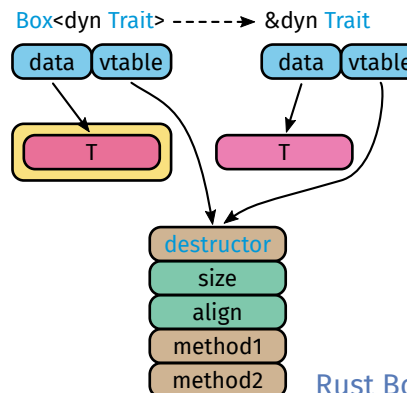
```
trait Pokemon { fn hp(&self) -> u32; }
struct Pikachu { hp: u32 }
impl Pokemon for Pikachu { fn hp(&self) -> u32 { self.hp } }
struct Charmander { hp: u32 }
impl Pokemon for Charmander { fn hp(&self) -> u32 { self.hp } }

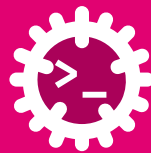
// ONE function, no generics - works on a trait object
fn total_hp(team: &[Box<dyn Pokemon>]) -> u32 {
    team.iter().map(|p| p.hp()).sum()
}

fn main() {
    // a real TEAM mixes species, so we need `dyn`
    let team: Vec<Box<dyn Pokemon>> = vec![
        Box::new(Pikachu { hp: 35 }),
        Box::new(Charmander { hp: 39 }),
    ];

    let total = total_hp(&team); // each hp() found via vtable
    println!("{}", total);      // 74
}
```

- **Trait object** `dyn Pokemon`, behind `Box<dyn ...>` or `&dyn ...`
- `&dyn Pokemon` is a **fat pointer**: (**data ptr**, **vtable ptr**)
- **vtable** $\Rightarrow$ run-time table of method addresses
- **Heterogeneous**: one `Vec` holds `Pikachu` **and** `Charmander`
- Cost: a pointer jump to the right method
- Similar to **virtual functions** in C++ / Java

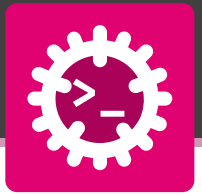
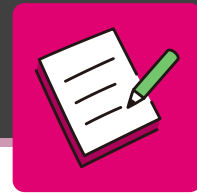




	Static - generics	Dynamic - trait objects
Syntax	<code>fn f<T: Pokemon>(t: &[T])</code>	<code>fn f(t: &[Box<dyn Pokemon>])</code>
Resolved	compile time (monomorphized)	run time (vtable)
Speed	fastest, inlinable	one pointer indirection
Binary size	larger (a copy per species)	smaller (one shared copy)
Collection	one species per slice	mixed species in one Vec
Classic OOP	templates	virtual methods



Rule of thumb: reach for **generics** by default; switch to **trait objects** when one collection must hold **different species** - a real team.



A home holds **different** device types in **one** list - that needs **dynamic dispatch**. Reusing the Device trait, **read** the code and give the output.

```
fn main() {  
  let home: Vec<Box<dyn Device>> = vec![  
    Box::new(Lamp { on: true }),  
    Box::new(Thermostat { celsius: 22 }),  
    Box::new(Lamp { on: false }),  
  ];  
  for d in &home {  
    println!("{}", d.status()); // ?  
  }  
}
```



One species per list $\Rightarrow$ **generics** (static). A **mix** of species $\Rightarrow$ `Box<dyn Device>` (dynamic, via vtable).



OOP in Rust - Recap

Same goals, different tools



OOP concept	Rust mechanism
Class	<code>struct / enum + impl</code>
Object / instance	a value
Attribute / field	<code>struct</code> field
Method	<code>fn(&self)</code> in an <code>impl</code>
Constructor	associated <code>fn new() → Self</code>
Encapsulation	modules + <code>pub</code> (private by default)

OOP concept	Rust mechanism
Interface / abstraction	<code>trait</code>
Abstract method	required trait method
Virtual method / override	default trait method
Inheritance	<i>none</i> - traits + composition
Subtype polymorphism	trait objects <code>dyn Trait</code>
Generics / templates	generics + trait bounds <code>T: Trait</code>



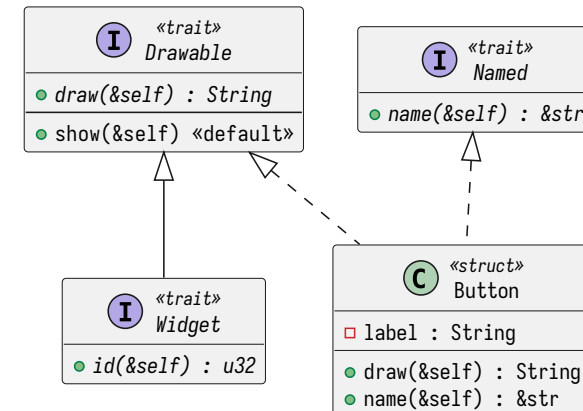
The four pillars survive; only **inheritance** is reshaped into **traits** (behaviour) and **composition** (data)



A trait **is** an interface: the set of methods a type promises to provide.

```
trait Drawable {  
    fn draw(&self) -> String;    // required (abstract)  
    fn show(&self) {             // default method  
        println!("{}", self.draw());  
    }  
}  
  
trait Named { fn name(&self) -> &str; }  
  
struct Button { label: String }  
  
// one type implements several interfaces  
impl Drawable for Button {  
    fn draw(&self) -> String { format!("Button({})", self.label) }  
}  
  
impl Named for Button {  
    fn name(&self) -> &str { &self.label }  
}  
  
// supertrait: a Widget must also be Drawable  
trait Widget: Drawable {  
    fn id(&self) -> u32;  
}
```

- **Abstract method** - required, no body
- **Default method** - has a body
- **Implementation** - impl Trait for Type
- **Multiple interfaces** - many traits per type
- **Supertrait** - trait A: B
- **Dependency inversion** - depend on trait bounds, not concrete types





Design Patterns

Solutions to problems you'll eventually create



A **design pattern** is a named, reusable solution to a problem that keeps recurring

- **Classic OOP** expresses them as arrangements of classes (the “Gang of Four” catalogue)
- **Rust** often bakes them into the **type system** and **standard traits** - less boilerplate, checked by the compiler
- This chapter covers two families:
 - **Standard-library traits** - implement them and your type plugs into the language
 - **Idiomatic patterns** - the shapes that feel native in Rust



Standard Library Traits

Implement these and your type speaks Rust



Rust has **no class inheritance** - types share behaviour by **implementing traits**, never by extending a base class (polymorphism comes from `dyn Trait` / generics). Implement these standard traits to plug your type into the language - most are derivable with `#[derive(...)]`.

- **Formatting** - `Debug` (`{:?}`), `Display` (`{}`)
- **Duplication** - `Clone` (explicit), `Copy` (implicit, bitwise)
- **Defaults** - `Default` $\Rightarrow$ `T::default()`
- **Equality & ordering** - `PartialEq`/`Eq`, `PartialOrd`/`Ord`, `Hash`
- **Conversion** - `From`/`Into`, `TryFrom`/`TryInto`, `AsRef`/`AsMut`
- **Iteration** - `Iterator` (`map` / `filter` / `collect`), `IntoIterator`
- **Resource cleanup** - `Drop`
- **Operators** - `std::ops` (`Add`, `Mul`, `Index`, ...)



For the common traits, the compiler writes the implementation for you with `#[derive(...)]`.

```
#[derive(Debug, Clone, PartialEq, Default)]
struct Point { x: i32, y: i32 }

fn main() {
    let a = Point::default(); // Default → { x: 0, y: 0 }
    let b = a.clone();        // Clone
    println!("{a:?}");        // Debug → Point { x: 0, y: 0 }
    println!("{}", a == b);   // PartialEq → true
}
```

- One line generates correct, idiomatic implementations

```
#[derive(Debug, Clone, PartialEq, Default, ...)]
```

- Derivable when **every field** of a struct also implements the **trait**
- Hand-write the `impl` only when you need custom behaviour
- Same mechanism you've seen on `Debug`, `Clone`, `Default`



Overloading an operator means **implementing the matching trait** from `std::ops`.

```
use std::ops::Add;
use std::fmt;

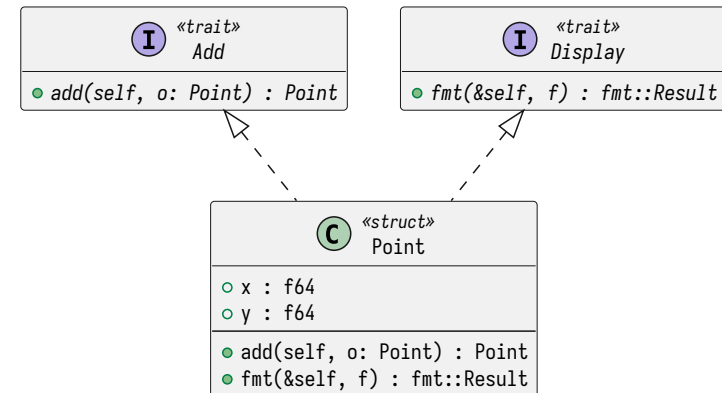
#[derive(Clone, Copy)]
struct Point { x: f64, y: f64 }

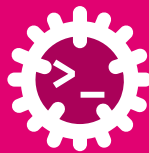
impl Add for Point {                // enables a + b
    type Output = Point;
    fn add(self, pt: Point) → Point {
        Point { x: self.x + pt.x, y: self.y + pt.y }
    }
}

impl fmt::Display for Point {       // enables {}
    fn fmt(&self, f: &mut fmt::Formatter) → fmt::Result {
        write!(f, "{{}, {}}", self.x, self.y)
    }
}

fn main() {
    let pt = Point { x: 1.0, y: 2.0 } + Point { x: 3.0, y: 4.0 };
    println!("{}", pt);              // (4, 6)
}
```

- `+` $\Rightarrow$ Add, `*` $\Rightarrow$ Mul, `[]` $\Rightarrow$ Index, ...
- `Display` is used in String formatting as `{}`
- `Debug` is used in String formatting as `{:?}`
- An operator is **just a method call** on a trait
- Operator behaviour is **opt-in** and type-checked





Display (the {} formatter) is **not derivable** - you decide how your type reads to a human.

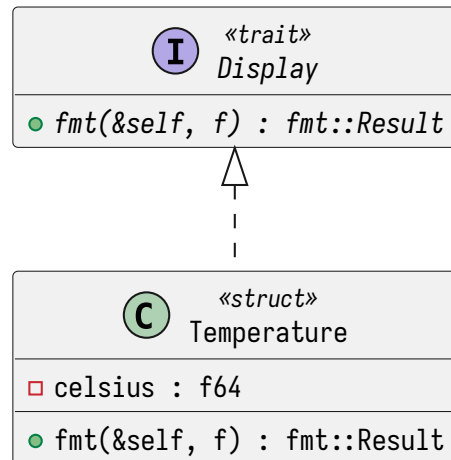
```
use std::fmt;

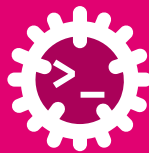
struct Temperature { celsius: f64 }

impl fmt::Display for Temperature {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(f, "{:.1} °C", self.celsius) // your format
    }
}

fn main() {
    let t = Temperature { celsius: 21.5 };
    println!("{}", t); // 21.5 °C (uses Display)
    let s = t.to_string(); // "21.5 °C" (ToString is free)
}
```

- Implement the `fmt::Display` trait.
- `write!(f, ...)` returns `fmt::Result`
- Implementing Display gives you `.to_string()` **for free**

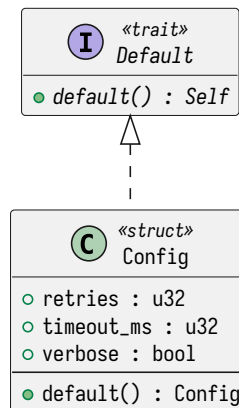




`#[derive(Default)]` fills every field with **its** default (0, false, ""). Implement `Default` by hand when you need **different** default values.

```
struct Config {
    retries: u32,
    timeout_ms: u32,
    verbose: bool,
}
impl Default for Config {
    fn default() → Self {
        Self {
            retries: 3,          // not 0
            timeout_ms: 5000,    // not 0
        }
    }
}
fn main() {
    let c = Config::default(); // retries: 3, timeout: 5000
    let c2 = Config {
        retries: 5,             // override one field
        ..Default::default()    // fill the rest with defaults
    };
}
```

- Derived default = **all-zero / empty**; sometimes that's wrong
- Implement `fn default() → Self` to pick **meaningful** defaults
- `..Default::default()` still works to fill the untouched fields





Both duplicate a value - the difference is **cost** and **when** it happens. Copy requires Clone.

```
#[derive(Clone, Copy)]           // Copy requires Clone
struct Point { x: i32, y: i32 }  // cheap, on the stack

#[derive(Clone)]                 // String owns a heap buffer → no Copy
struct Name { value: String }

fn main() {
    let p = Point { x: 1, y: 2 };
    let q = p;                    // COPIED implicitly
    println!("{}", p.x);         // ✓ p still usable

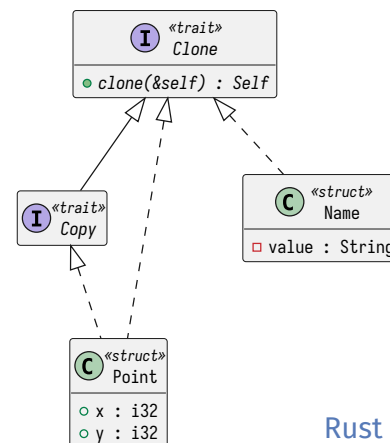
    let n = Name { value: "Ada".into() };
    let m = n.clone();            // CLONED explicitly (deep copy)
    println!("{}", m);           // ✓ both still usable
    println!("{}", n);           // ✓

    let m = n;                    // would MOVE n, not copy it
    println!("{}", m);           // ✓ n was moved in to m
    println!("{}", n);           // ✗ n is no longer valid
}
```

- **Copy** - **implicit**, bit-for-bit duplicate on assignment; the original stays valid
 - only for stack-only types (ixx, fxx, bool, ...)
- **Clone** - **explicit** .clone()
 - expensive (deep copy)



If Copy does not exist $\Rightarrow$ assignment = **moves** instead of copies.





Derive these to compare, sort, and key your type.

```
use std::collections::HashMap;

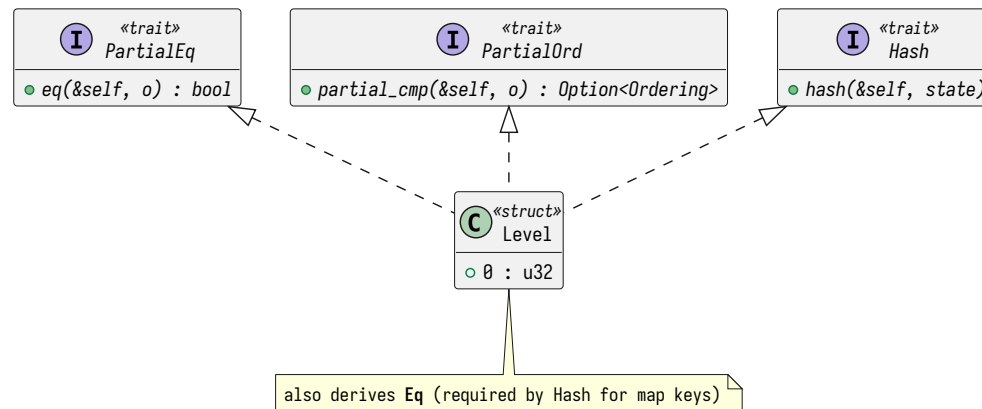
#[derive(PartialEq, PartialOrd, Eq, Hash)]
struct Level(u32);

fn main() {
    println!("{}", Level(1) == Level(1)); // PartialEq → true
    println!("{}", Level(1) < Level(2)); // PartialOrd → true

    let mut v = vec![Level(3), Level(1), Level(2)];
    v.sort(); // ordering → [1, 2, 3]

    let mut seen = HashMap::new(); // Hash → usable as a key
    seen.insert(Level(1), "low");
    println!("{}", seen.get(&Level(1))); // Some("low")
}
```

- **PartialEq** $\Rightarrow$ defines $=$ and $\neq$
- **PartialOrd** $\Rightarrow$ defines $< > \leq \geq$.sort()
- **Hash** $\Rightarrow$ reduces a value to a number so a HashMap / HashSet knows which bucket to use
 - For a hash: $a == b \Rightarrow \text{hash}(a) == \text{hash}(b)$
so Hash and equality Eq must agree
derive them together





Implement From and you get Into **for free** - the idiomatic way to convert between types.

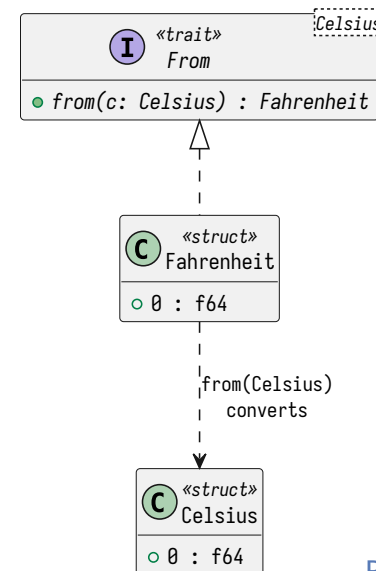
```
struct Celsius(f64);
struct Fahrenheit(f64);

impl From<Celsius> for Fahrenheit {
    fn from(c: Celsius) → Self {
        Fahrenheit(c.0 * 9.0 / 5.0 + 32.0)
    }
}

fn main() {
    // call From directly - explicit about the target type
    let a = Fahrenheit::from(Celsius(100.0));
    println!("{}", a); // 212

    // .into() - same conversion, target inferred from the type
    let b: Fahrenheit = Celsius(0.0).into();
    println!("{}", b); // 32
}
```

- Implement From<A> for B $\Rightarrow$ a.into() works automatically
- **Fallible** conversion $\Rightarrow$ TryFrom / TryInto (returns Result)
- The ? operator uses From to convert error types
- Convention: implement From, never Into directly





The Drop trait runs code when a value goes out of scope - Rust's destructor, the basis of C++ [RAII](#). Same object, three approaches:

 **Rust** - Drop, automatic:

```
struct Widget { id: u32 }
impl Drop for Widget {
    fn drop(&mut self) {
        println!("drop {}", self.id);
    }
}

fn main() {
    let _w = Widget { id: 1 };
    println!("using...");
} // _w dropped here
// → "drop 1"
```

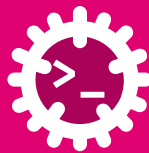
 **C++ RAI** - destructor, automatic:  **C++ classic** - manual delete:

```
struct Widget {
    int id;
    ~Widget() {
        std::cout << "drop " << id << "\n";
    }
};

int main() {
    Widget w{1};
    std::cout << "using...\n";
} // w destroyed here
// → "drop 1"
```

```
struct Widget {
    int id;
    ~Widget() {
        std::cout << "drop " << id << "\n";
    }
};

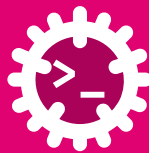
int main() {
    Widget* w = new Widget{1};
    std::cout << "using...\n";
    delete w; // × forget = leak
} // → "drop 1"
```



- **Rust & C++ RAI** - cleanup is **automatic** at end of scope, on **every** path
 - runs in reverse order of creation
 - no garbage collector
- **C++ classic** - **you** must delete
 - a missed branch **leaks**
 - a double delete is **undefined behaviour**
 - use-after-delete is a **dangling pointer**



Rust goes further - Drop is automatic **and** ownership makes double-free impossible; drop takes `&mut self` and can't be called by hand (drop early with `std::mem::drop(x)`)

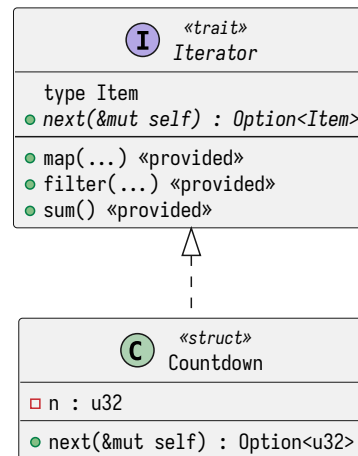


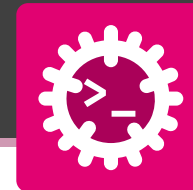
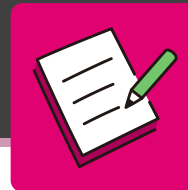
Implement one method - `next()` - and the whole iterator toolbox (`map`, `filter`, `sum`, `collect`, ...) comes for free.

```
struct Countdown { n: u32 }
impl Iterator for Countdown {
    type Item = u32;
    fn next(&mut self) → Option<u32> {
        if self.n == 0 { return None; } // None = done
        self.n -= 1;
        Some(self.n + 1)
    }
}

fn main() {
    let total: u32 = Countdown { n: 3 } // yields 3, 2, 1
        .map(|x| x * 2)                // 6, 4, 2
        .sum();
    println!("{}", total);             // 12
}
```

- Define type `Item` and `next()` returning `Option<Item>`
- `None` signals the end of iteration
- Every adapter (`map`, `filter`, `take`, ...) builds on `next()`
- Lazy: nothing runs until consumed by `sum` / `collect` / a loop



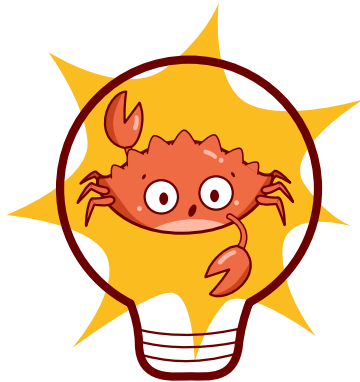


The program uses **four** standard traits, but the struct derives **none** of them. Add the **single** `#[derive(...)]` line that makes it compile.

```
struct Coord { x: i32, y: i32 }  
  
fn main() {  
    let a = Coord::default(); // ?  
    let b = a.clone();        // ?  
    println!("{a:?}");        // ?  
    println!("{}", a == b);    // ?  
}
```



Each call names one trait: `default()`, `.clone()`, `{:?}`, `==`.



Idiomatic Patterns

Patterns that feel native in Rust

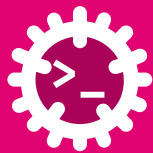


Patterns that fall naturally out of Rust's own features - not workarounds borrowed from class-based languages.

- The language often **is** the pattern: implement a std trait (Iterator, Drop) and you're done
- Others are conventions the type system rewards (Newtype, Builder)

Jump to any pattern:

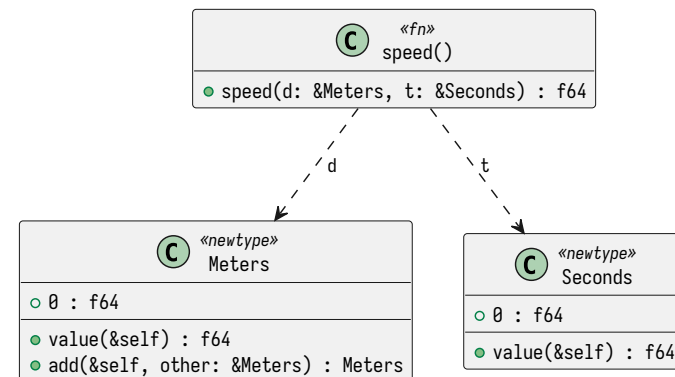
- **Newtype** - a type-safe wrapper
- **RAII / Drop** - acquire in new(), release automatically in Drop
- **Singleton** - one shared instance via OnceLock<T> / static
- **Builder** - step-by-step construction of complex structs
- **Strategy** - interchangeable behaviour via trait objects or generics

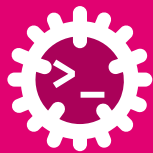


```
struct Meters(f64);
struct Seconds(f64);
impl Meters {
    fn value(&self) → f64 { self.0 }
    // distance + distance = distance
    fn add(&self, other: &Meters) → Meters {
        Meters(self.0 + other.0)
    }
}
impl Seconds { fn value(&self) → f64 { self.0 } }
fn speed(d: &Meters, t: &Seconds) → f64 { // calculate speed
    d.0 / t.0
}
fn main() {
    let d = Meters(100.0);
    let total = d.add(&Meters(5.0)); // Meters(105.0)
    let v = speed(&total, &Seconds(9.58)); // uses both types
    println!("{}", total.value()); // 105
    println!("{:.2}", v); // 10.97
    speed(&total, &Seconds(9.58)); // ✓ compiles - Meters, Seconds
    speed(&total, 9.87); // ✗ won't compile - Meters, f64
    speed(&Seconds(9.58), &total); // ✗ won't compile - Seconds, Meters
}
```

Wrap a value in a one-field struct to give it a **distinct type**

- A **tuple struct** with one field - zero runtime cost
- The compiler intervenes on misuse
- Meters and Seconds are **different types**, even though both hold an f64
- Encodes **units / intent** into the type system

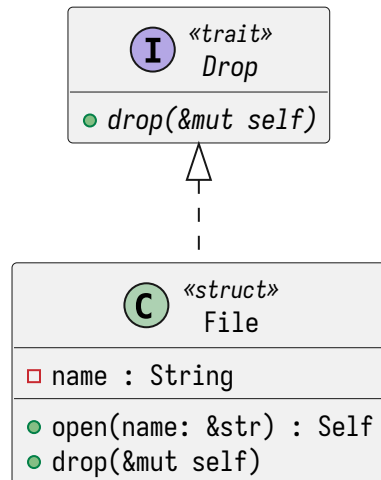




Tie a resource's lifetime to a value: acquire it in the constructor, release it automatically in Drop.

```
struct File { name: String }  
impl File {  
    fn open(name: &str) -> Self {  
        println!("opening {name}");  
        Self { name: name.into() }  
    }  
}  
  
impl Drop for File {  
    fn drop(&mut self) { // runs automatically  
        println!("closing {}", self.name);  
    }  
}  
  
fn main() {  
    let _f = File::open("data.txt");  
} // _f goes out of scope -> "closing data.txt"
```

- **Acquire** in open / new, **release** in drop
- Drop::drop runs **automatically** at end of scope
- Implement Drop::drop only if you require custom cleanup
- No manual close(), no forgotten cleanup, no leaks
- Similar to C++ destructors / **RAII** but guaranteed by ownership





Guarantee **exactly one** shared instance, reached through a **single access point** - e.g. one app config, one logger, one connection pool, one hardware resource.

```
use std::sync::OnceLock;

struct Config {
    host: String,
    port: u16,
}

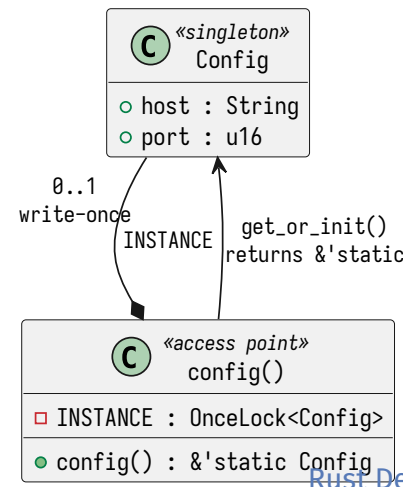
fn config() → &'static Config {
    // the single, write-once global slot
    static INSTANCE: OnceLock<Config> = OnceLock::new();

    INSTANCE.get_or_init(|| Config { // built on first call only
        host: "localhost".to_string(),
        port: 8080,
    })
}

fn main() {
    println!("{}", config().port); // first call → builds it
    println!("{}", config().host); // later call → same instance
}
```

Two promises: built **once**, then **shared** by all.

- **static INSTANCE:** OnceLock<Config> - the global slot, empty at start
- **get_or_init(...)** - **constructs** the Config once; later calls reuse it
- **returns** → &'static Config - callers get a **read-only** reference to the one instance





Construct a complex object **step by step**

```
#[derive(Default)]
struct Server { host: String, port: u16, tls: bool }

struct ServerBuilder { server: Server }    // the scaffold

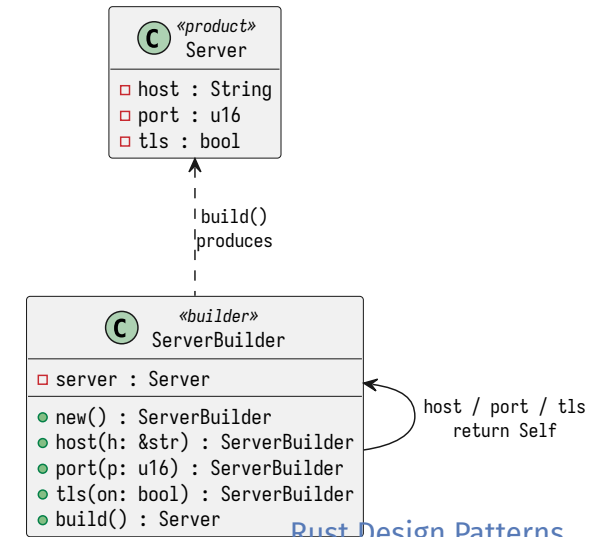
impl ServerBuilder {
    fn new() → Self { Self { server: Server::default() } }
    fn host(mut self, h: &str) → Self {
        self.server.host = h.into();
        self
    }
    fn port(mut self, p: u16) → Self { self.server.port = p; self }
    fn tls(mut self, on: bool) → Self { self.server.tls = on; self }
    fn build(self) → Server { self.server }    // hand back product
}

fn main() {
    let s = ServerBuilder::new()
        .host("localhost")    // set only what you need,
        .tls(true)            // in any order, by name
        .build();             // → Server { host, port: 0, tls:
true }
}
```

- each setting is optional, chosen by name
- `build()` produces the finished object



Avoids long positional constructors
`new("localhost", 0, true)` and a
maze of overloads for optional fields.





Make behaviour **interchangeable**: define a family of algorithms behind one trait, and let the caller pick or swap which one runs.

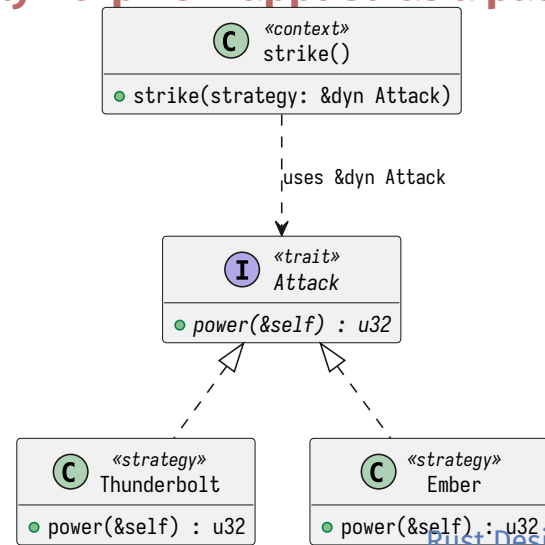
```
// the strategy interface
trait Attack {
    fn power(&self) → u32;
}

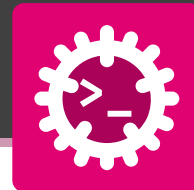
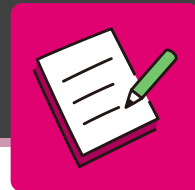
// concrete strategies
struct Thunderbolt;
impl Attack for Thunderbolt { fn power(&self) → u32 { 90 } }
struct Ember;
impl Attack for Ember { fn power(&self) → u32 { 40 } }

// the context: uses a strategy, doesn't know which one
// Accept any type implementing the Attack trait
fn strike(strategy: &dyn Attack) {
    println!("hit for {}", strategy.power());
}

fn main() {
    strike(&Thunderbolt); // hit for 90
    strike(&Ember);       // hit for 40 (swapped)
}
```

- Each algorithm is a type implementing a shared **trait** (Attack)
- Swap behaviour via:
 - &dyn Trait dynamic dispatch (run time)
 - <T: Trait> static dispatch (compile time)
- This is **polymorphism applied as a pattern**





Use the **Newtype** pattern so a user-id can never be confused with a product-id.

1. Define `UserId(u32)` and `ProductId(u32)` as tuple structs.
2. Write `fn load_user(id: UserId)` that prints the inner number.
3. Show that `load_user(ProductId(7))` and `load_user(7)` do **not** compile.



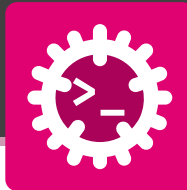
A one-field tuple struct is a **distinct type** at **zero runtime cost**.



Concurrency

Doing more than one thing at once

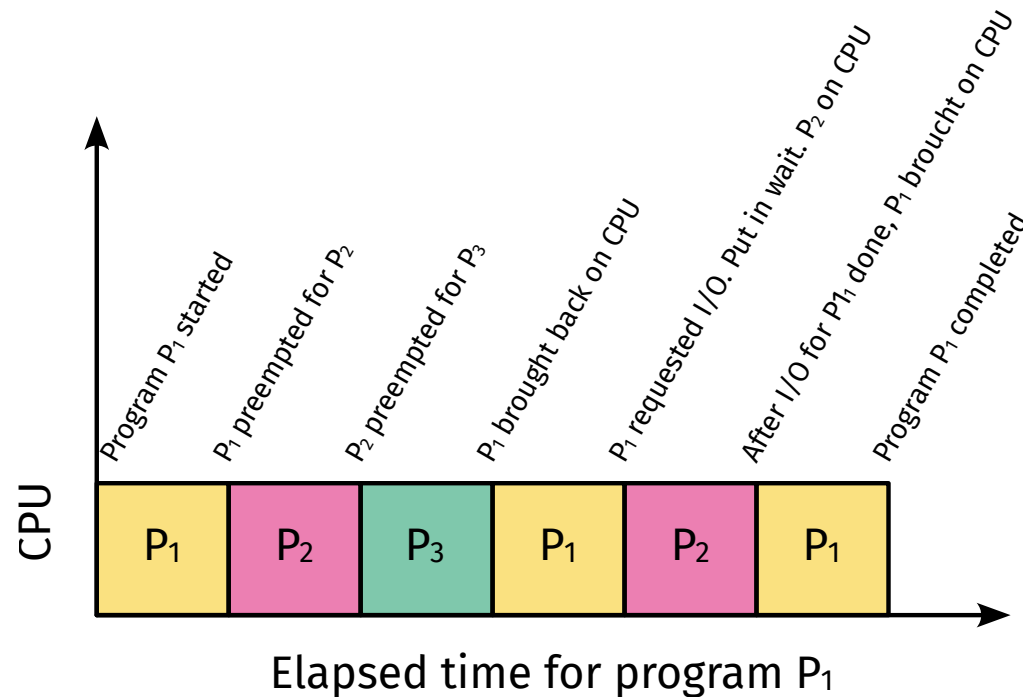
Why concurrency?



A program that does **one thing at a time** leaves the system underutilized.

- Programs spend a lot of time **waiting**
 - disk, network, the user
- That waiting time could be spent doing **other useful work**.

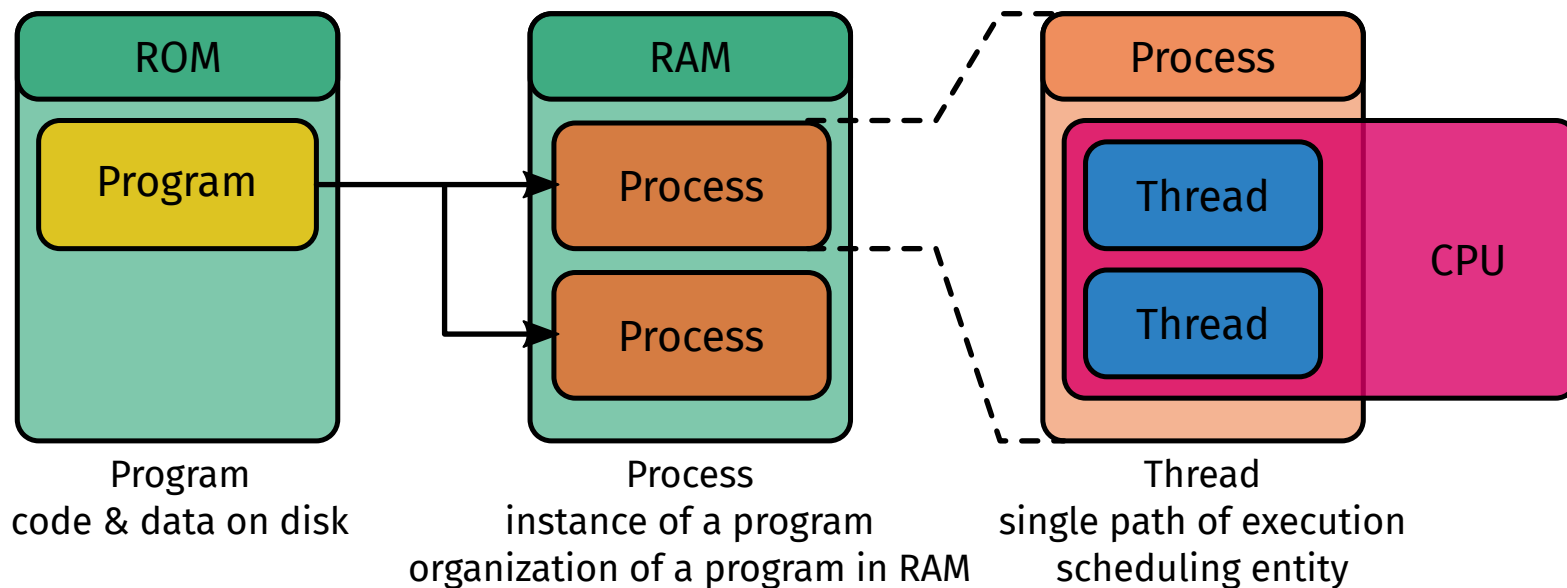
Goal: do more in the same wall-clock time, and stay **responsive**.

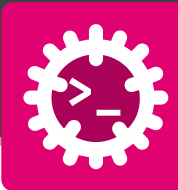




Three words we will use constantly - let's pin them down first.

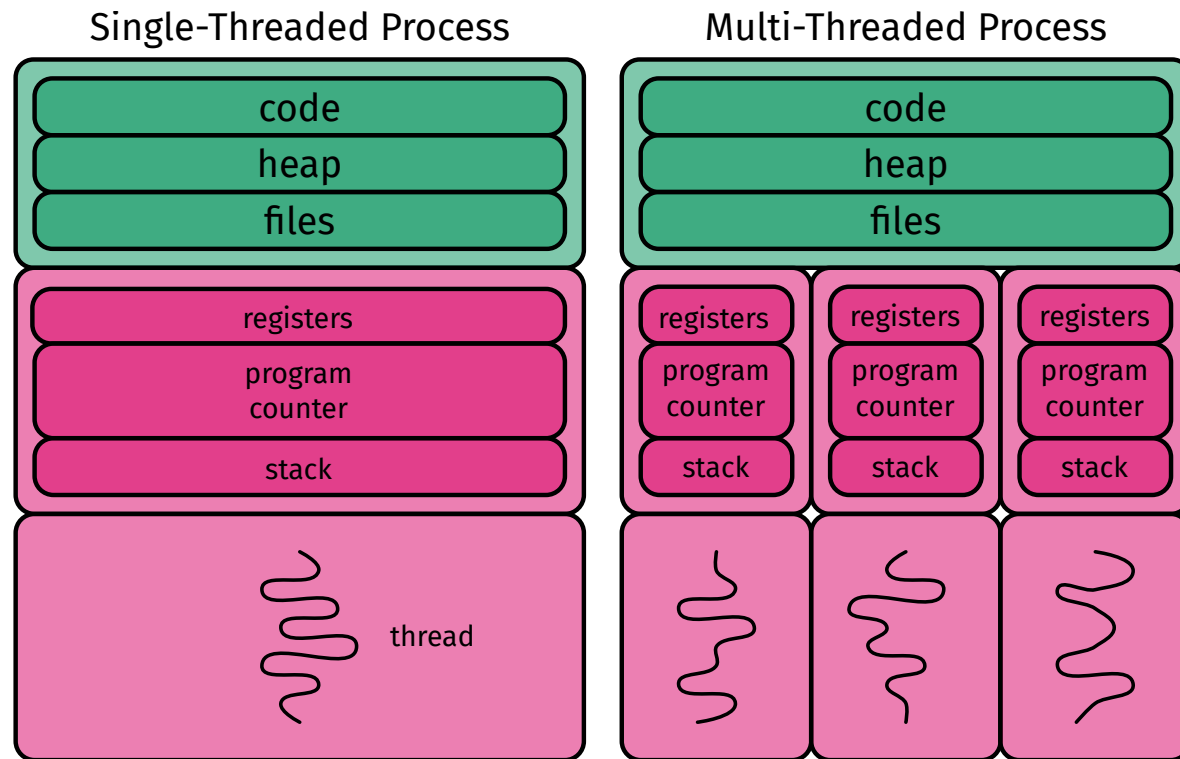
- **Program** - code & data sitting **on disk** (not running)
- **Process** - a **running** instance of a program, with its **own memory**
- **Thread** - a **single path of execution** inside a process (the unit the **Operating System (OS) schedules**)





A process can hold **several threads**. They **share** most of the process...

- **Shared** by all threads
 - **code**, **heap**, open **files**
- **Private** to each thread
 - its own **stack** and **registers**
- Sharing the heap is what makes threads **fast to communicate**...
- ...and exactly what makes them **dangerous**





A **data race** occurs when:

1. two or more threads access the **same** memory location,
2. at least one of these operation is a **write** access,
3. the operations are **unsynchronized** (no lock, no ordering guarantee between them)
4. the operations **overlap in time**, meaning they occur concurrently.



The result is **undefined** - corrupted data, random crashes.

Fearless concurrency

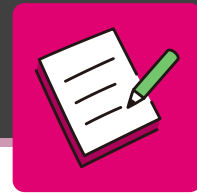
Rust rejects data races **at compile time**:

- **Ownership & borrowing**
 - one **&mut** **or** many **&**, never both
 - either **many readers**, or **one writer**, never both
- **Send** - a type is safe to **move** to another thread
- **Sync** - a type is safe to **share** (&T) between threads



If it **compiles**, it has **no data races**.

(Deadlocks & logic bugs 🐛 are still on you.)



Pick the **one** correct answer for each question.

1. What is private to each thread?

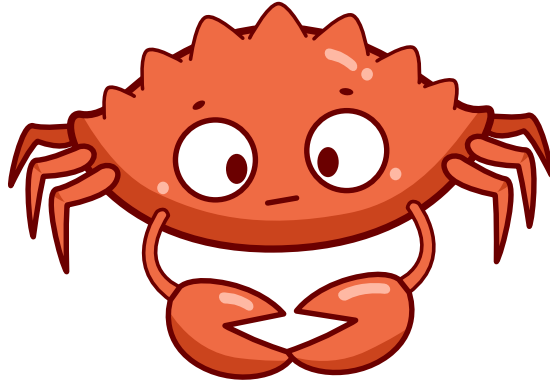
- a) the heap
- b) open files
- c) its stack and registers
- d) the program code

2. A data race needs all of these EXCEPT:

- a) two threads touch the same location
- b) at least one operation is a write
- c) the operations are guarded by a lock
- d) the operations overlap in time

3. “If it compiles, no data races” - thanks to:

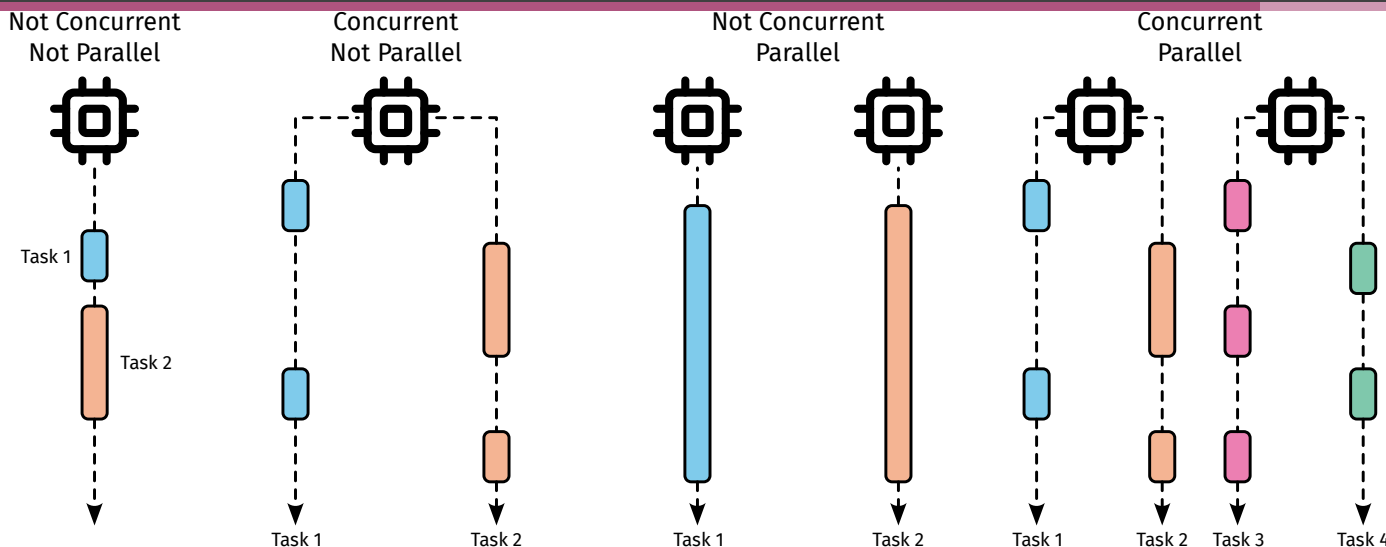
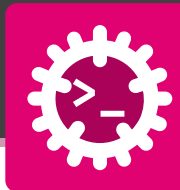
- a) a garbage collector
- b) ownership / borrowing + Send / Sync
- c) the OS scheduler
- d) running on a single core



Concurrency // Parallelism

Structure vs. simultaneous execution

Concurrency ≠ Parallelism



Concurrency - structure

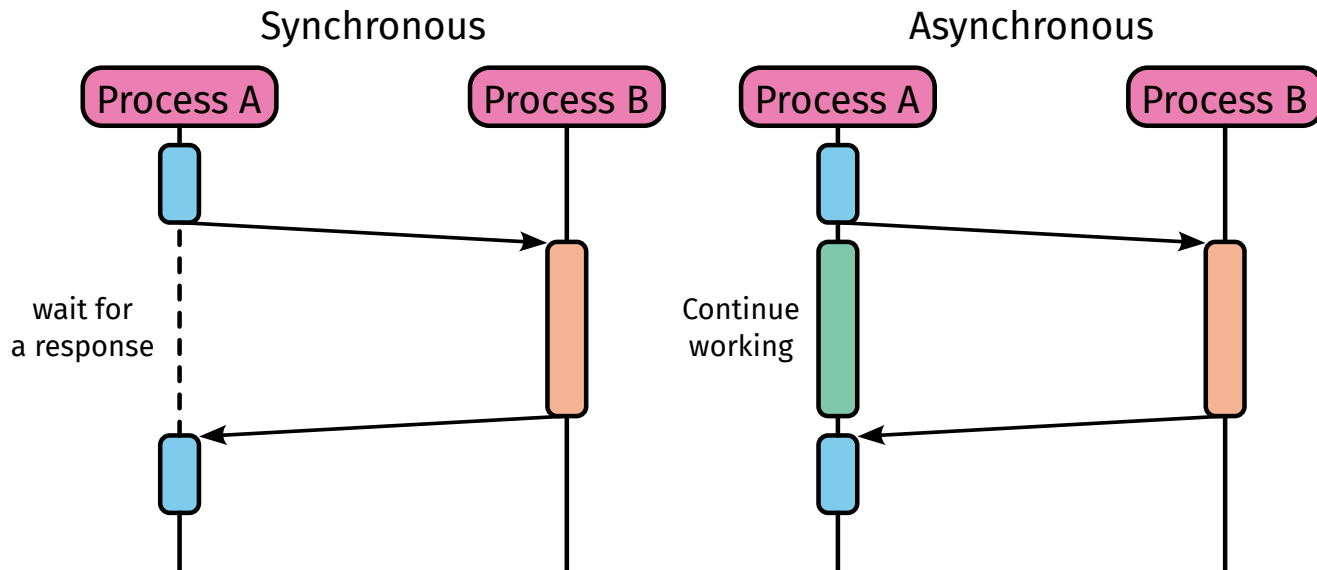
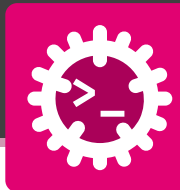
- many tasks that progress **independently**
- can **interleave** on a **single** core
- *“dealing with many things at once”*

Parallelism - execution

- tasks running at the **same instant**
- needs **multiple** cores
- *“doing many things at once”*



A program can be **concurrent** on one core, and becomes **parallel** when given more.



Synchronous

Process B must **wait** for Process A's response before continuing.

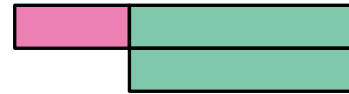
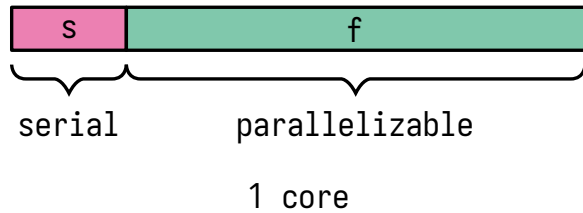
Asynchronous

Process A **fires off** the request and **keeps working**, it handles the response later.

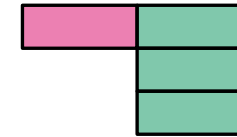


The same idea at every level: **don't block while you wait.**

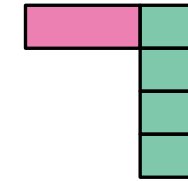
The limit: Amdahl's Law



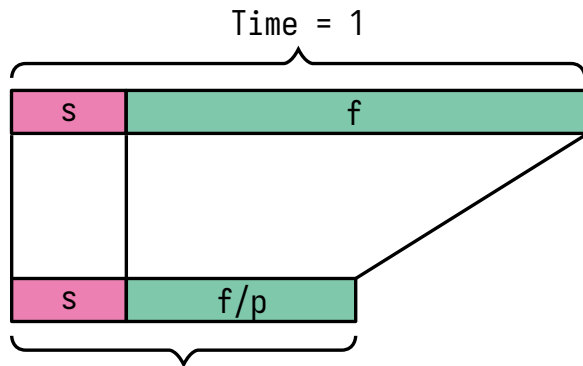
2 cores



3 cores



4 cores

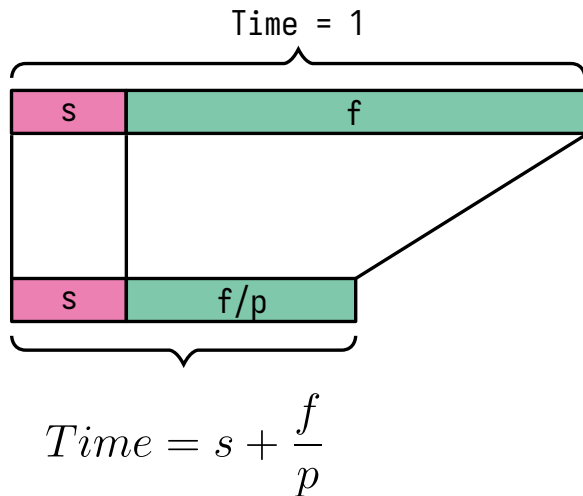
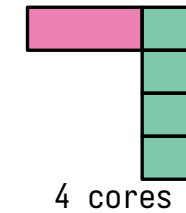
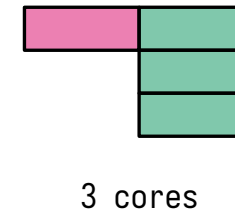
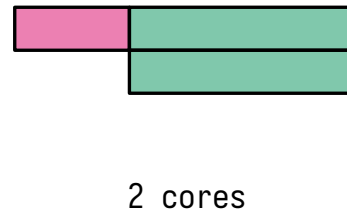
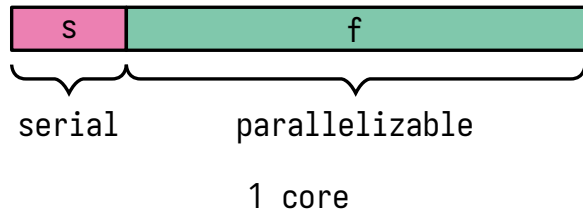


$$Time = s + \frac{f}{p}$$



If 50% of the execution time is sequential, the maximum speedup is 2x, no matter how many cores you throw at it.
- Gene Amdahl, 1967

The limit: Amdahl's Law



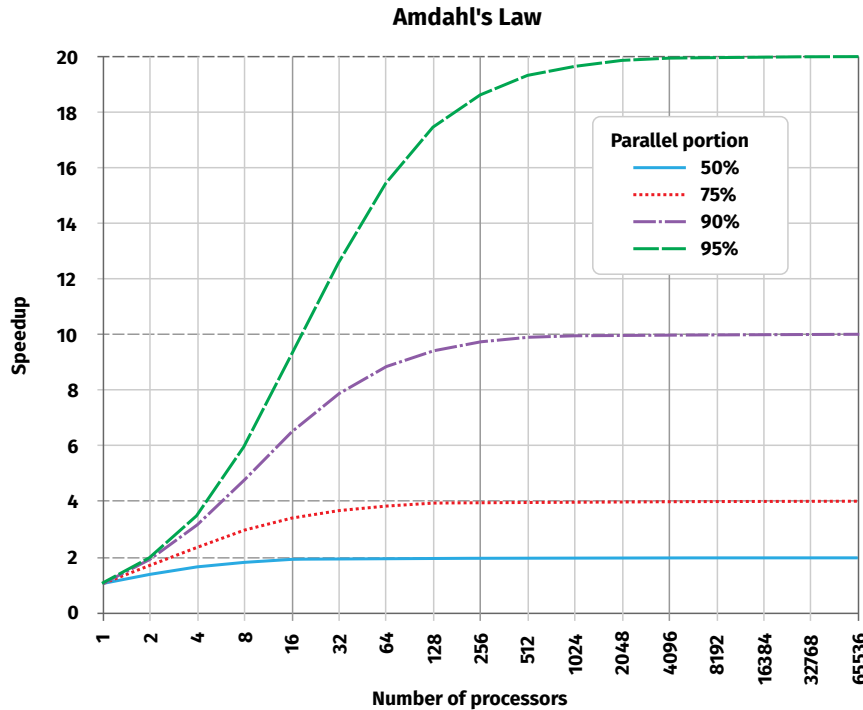
$$S = \frac{1}{s + \frac{f}{p}} = \frac{1}{1 - f + \frac{f}{p}}$$

S = Speedup

$s = 1 - f$ = Serial fraction

$f = 1 - s$ = Parallel fraction

p = Number of processors



- Every program has a **serial part** that **cannot** be parallelised.
- Only the **parallel part** benefits from more cores.
- Example: if **5%** is serial, the maximum speed-up is **20×**, ever.



Adding cores has **diminishing returns**.
Measure before you parallelise.



Floating point instructions improved to run 2x as fast, but only 10% of all executed instructions are Floating point. What will the overall speedup be?



Threads

Spawning and joining



```
use std::thread;
use std::time::Duration;

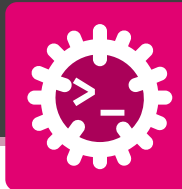
fn main() {
    // spawn a thread that runs ALONGSIDE main
    let handle = thread::spawn(|| {
        for i in 1..=4 {
            println!("  thread: step {i}");
            thread::sleep(Duration::from_millis(100));
        }
        42 // value returned by the thread
    });

    // main keeps working at the SAME time
    for i in 1..=4 {
        println!("main: step {i}");
        thread::sleep(Duration::from_millis(100));
    }

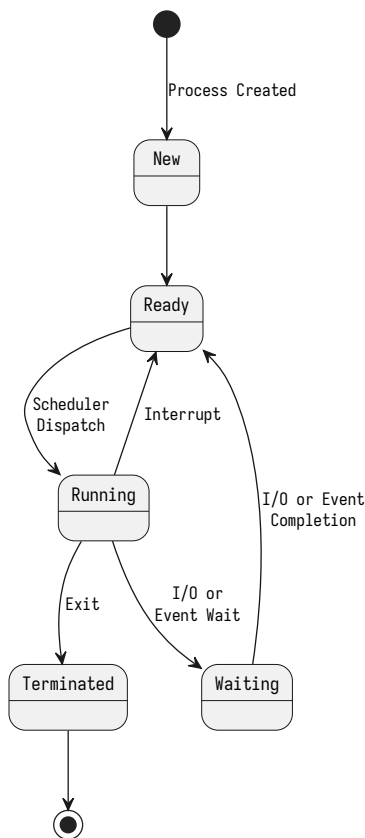
    // join: wait for the worker, collect its result
    let result = handle.join().unwrap();
    println!("thread returned {result}");
}
```

```
main: step 1
  thread: step 1
main: step 2
  thread: step 2
main: step 3
  thread: step 3
main: step 4
  thread: step 4
thread returned 42
```

- `thread::spawn` starts a **new OS thread** that runs **alongside** main
- Both loops print **interleaved** - proof they run at the **same time**
- `join()` **blocks** until the thread finishes and **returns its value** (42)
- Without `join`, main could **end first** and kill the thread mid-run



A thread moves through a small **state machine** - the OS decides who runs when.



The states a thread is in:

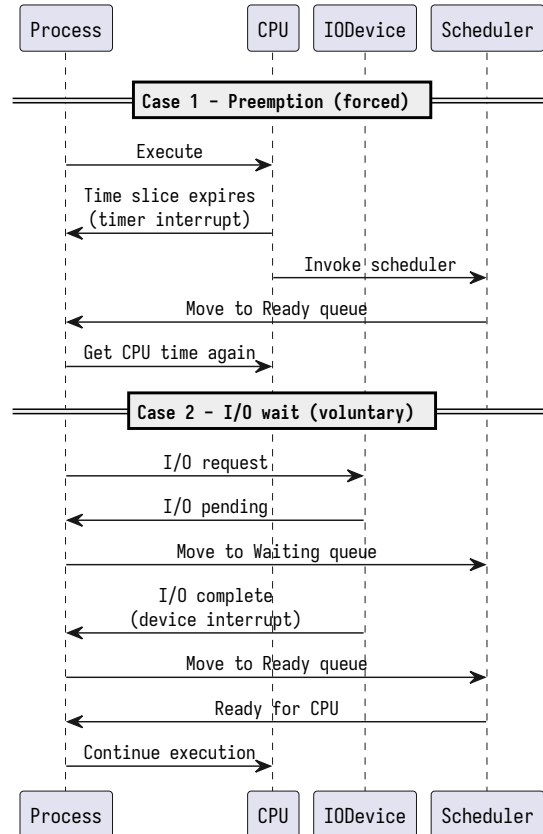
- **Ready** - requires the Central Processing Unit (CPU), waiting its turn
- **Running** - currently executing on a core
- **Waiting** - blocked on Input/Output (I/O) or an event, not competing for the CPU

How it moves between them:

- **Dispatch** - the scheduler picks a Ready thread and runs it
- **Preemption** - the OS can **interrupt** a Running thread at any time, sending it back to Ready so another can run
- **Block / wake** - a thread that waits on I/O leaves the CPU when the I/O completes it returns to Ready



A running thread gives up the core in **two** ways - the scheduler then parks it and runs another.



Case 1 - preemption (forced):

- The thread runs on the **CPU** until its **time slice** expires
- The **CPU invokes the scheduler**, which moves the thread to the **ready queue**
- It waits its turn, then gets the **CPU** back

Case 2 - I/O wait (voluntary):

- The thread issues an **I/O request** and can't continue
- It moves to the **waiting queue** - off the **CPU**, not competing for it
- When the device signals **completion** (an **interrupt**), the thread returns to the **ready queue**



Either way the **CPU** is never idle - while one thread waits, another runs.



```
use rayon::prelude::*;
use std::time::Instant;

fn cost(n: u64) → u64 {
    (1..=n).map(|i| i % 7).sum()
}

fn main() {
    let nums: Vec<u64> = (1..=20_000).collect();

    // — sequential —
    let start = Instant::now();
    let _seq: u64 = nums.iter().map(|&n| cost(n)).sum();
    let seq_time = start.elapsed();

    // — parallel (Rayon) —
    let start = Instant::now();
    let _par: u64 = nums.par_iter().map(|&n| cost(n)).sum();
    let par_time = start.elapsed();
    let speedup = seq_time.as_secs_f64() / par_time.as_secs_f64();
    println!("sequential: {seq_time:?}");
    println!("parallel:    {par_time:?}");
    println!("speedup:      {:.2}x", speedup);
}
```

Macbook Pro M4 Max (16cores)

sequential: 1.553656209s

parallel: 133.673667ms

speedup: 11.62x

- Compare `iter()` (sequential) vs `par_iter()` (parallel), timed with `Instant`
 - One line changes - `iter()` $\Rightarrow$ `par_iter()` - and Rayon splits the work over a **thread pool** ($\approx$ one thread per core)
 - **11.6× faster** here on a CPU-bound, **independent** workload
 - Rust's safety guarantees still hold - **no data races**, enforced at compile time

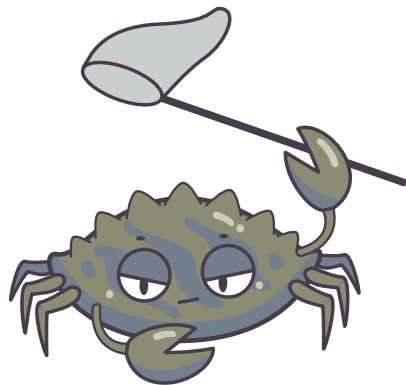


Read the program and give the value it prints. Does the **thread order** change the result?

```
use std::thread;

fn main() {
    let mut handles = vec![];
    for i in 1..=3 {
        handles.push(thread::spawn(move || i * 10));
    }

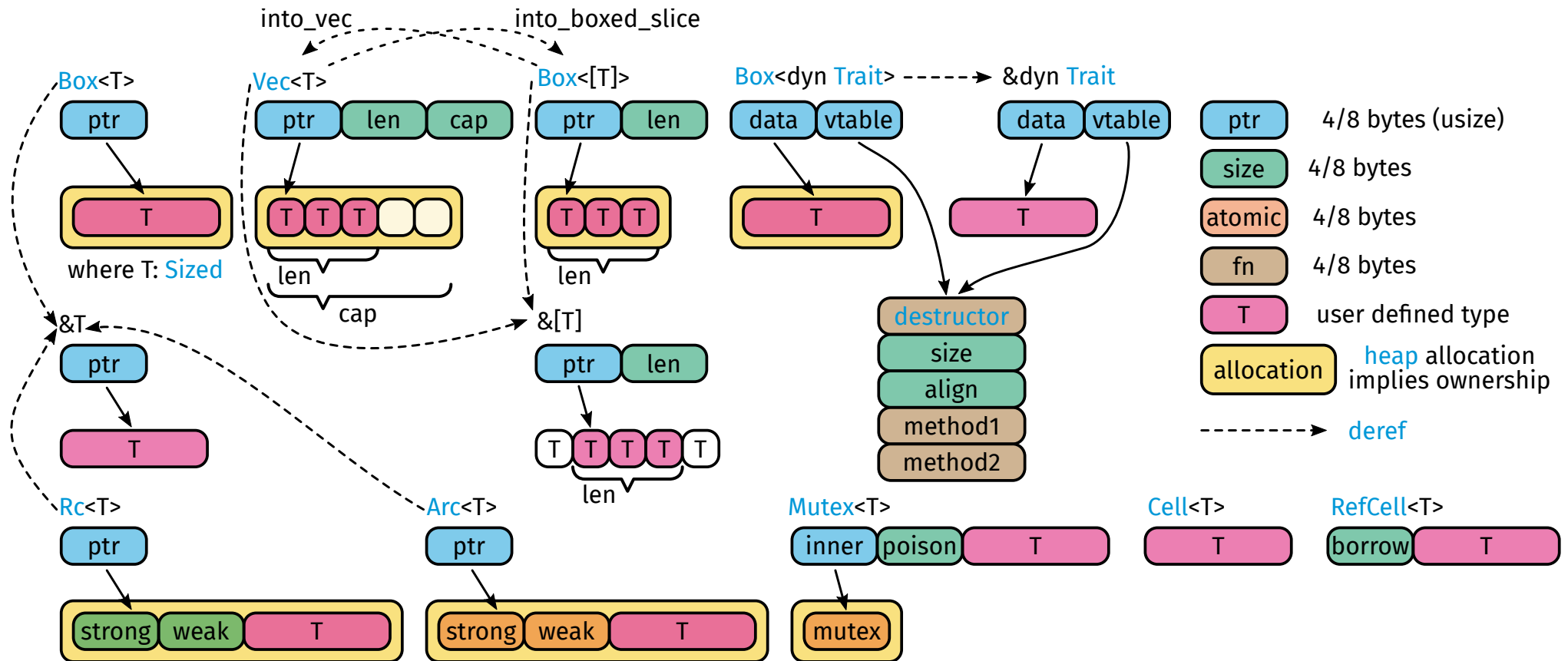
    let mut total = 0;
    for h in handles {
        total += h.join().unwrap();
    }
    println!("total = {total}"); // ?
}
```

Smart Pointers

Owning and sharing values safely

Smart Pointers Overview



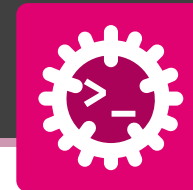


A **smart pointer** is a type that owns data **and adds behaviour** (heap allocation, reference counting, locking, runtime borrow checks). A plain `&T` is just a **borrow** - not a smart pointer.

Type	Group	Description
Single owner	<code>Box<T></code>	Puts a value on the heap instead of the stack, with one owner. Use it for data too big for the stack, or for recursive types (a struct that contains itself) where the size would otherwise be infinite.
	<code>Box<dyn Trait></code>	A Box holding a trait object : “some type implementing <code>Trait</code> , decided at run time.” Lets you own a value when you don’t know its concrete type - e.g. one element in a list of mixed shapes.
Shared owner	<code>Rc<T></code>	Reference-counted shared ownership: many parts of your program co-own one value. It counts the owners and frees the value when the last one is dropped. Single thread only (the count isn’t thread-safe).
	<code>Arc<T></code>	The same as <code>Rc</code> , but the owner count mutations are atomic , so it is safe to share across threads . Slightly slower than <code>Rc</code> ; use it only when threads are involved.



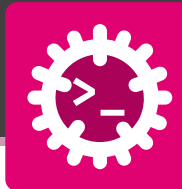
Type	Group	Description
Interior mutability	Cell<T>	Lets you change a value even through a shared & reference, by swapping the whole value in and out with <code>get</code> / <code>set</code> . For small Copy types (numbers, flags). No borrowing, never panics.
	RefCell<T>	Like <code>Cell</code> , but lets you borrow the inner value (<code>borrow</code> / <code>borrow_mut</code>). The borrow rules (one writer or many readers) are checked while the program runs instead of at compile time - and it panics if you break them.
	Mutex<T>	Guards data so one thread at a time can touch it. <code>lock()</code> blocks until it's your turn, hands you exclusive access, and unlocks automatically when you're done. The way to mutate shared data across threads.
	RwLock<T>	Like <code>Mutex</code> , but smarter about reading: it allows many readers at once OR one writer . Use it when reads are frequent and writes are rare.



Ask yourself two questions: **How many threads?** and **read or write?**

Need	Single thread	Multi-thread
heap, one owner	Box<T>	Box<T>
shared ownership (read)	Rc<T>	Arc<T>
shared + mutable	Rc<RefCell<T>>	Arc<Mutex<T>> Arc<RwLock<T>>
interior mutability mutate a Copy field via &self	Cell<T>	Atomic*

- The columns **mirror** each other: Rc ⇔ Arc, RefCell ⇔ Mutex, Cell ⇔ Atomic
- Same three ideas - **own, share, mutate-through-shared** - each with a **thread-safe** twin
- **Rule**: pick the **single-thread** tool unless you actually cross threads (it's cheaper)



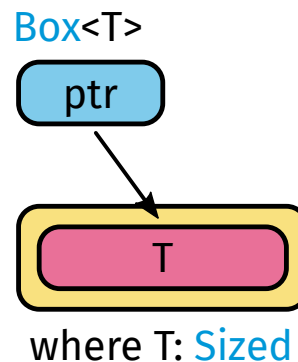
Concept: put **one** value on the **heap**.

- The **same** tool single- or multi-threaded (move it across with move).
- The value is freed when the Box drops (RAII).

```
use std::thread;
// 4 KB grid: too big for the stack arrow.stroked heap
let grid = Box::new([[0u8; 64]; 64]);
// `move` transfers the ONE owner to the worker
let h = thread::spawn(move || grid.len());
println!("rows: {}", h.join().unwrap()); // 64
```

Box<T>

- one value on the **heap**
(data too big for the stack, or a **recursive** type)
- move hands that **sole owner** to a thread





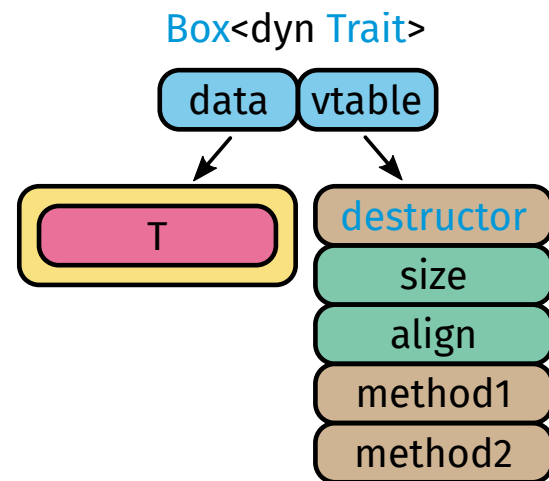
Concept: put **one** value on the **heap**.

- The **same** tool single- or multi-threaded (move it across with move).
- The value is freed when the Box drops (RAII).

```
trait Job: Send { fn run(&self); }
struct Email;
impl Job for Email {
    fn run(&self) { println!("sending"); }
}
// concrete type hidden behind the Box
let job: Box<dyn Job> = Box::new(Email);
// move the trait object onto a worker thread
thread::spawn(move || job.run()).join().unwrap();
```

Box<dyn Trait>

- owns **some** type implementing Trait, picked at **run time** (dynamic dispatch).
- A Send object can move across threads.





Concept: many owners of **one** read-only value

- a non-atomic **reference count** frees it when the **last** owner drops.

```
use std::rc::Rc;

struct Config { db_url: String, retries: u32 }
struct Service { cfg: Rc<Config> }

fn main() {
    // build the read-only config ONCE
    let cfg = Rc::new(Config {
        db_url: "postgres://db:5432".into(),
        retries: 3,
    });

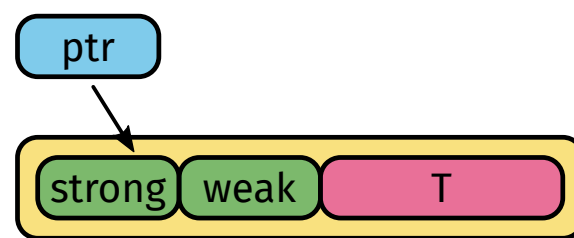
    // every service co-owns the SAME config, no data copy
    let api = Service { cfg: Rc::clone(&cfg) };
    let worker = Service { cfg: Rc::clone(&cfg) };

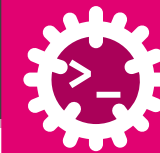
    println!("owners: {}", Rc::strong_count(&cfg)); // 3
    println!("{}", api.cfg.db_url, worker.cfg.retries);
}
```

Rc<T>

- many owners of one **immutable** value
- clone bumps a **non-atomic** count
⇒ **no deep copy**
 - last owner dropped ⇒ value **freed**
- **single thread only** (count isn't thread-safe)

Rc<T>





Concept: **many** owners of **one** read-only value, but the count is **atomic** $\Rightarrow$ safe to share **across threads**

- Slightly slower, use it only when threads are involved.

```
use std::sync::Arc;
use std::thread;

fn main() {
    // load a big read-only dataset ONCE
    let data = Arc::new(vec![7, 2, 9, 4, 8, 1, 5, 3]);
    let mut handles = vec![];
    for t in 0..4 {
        let data = Arc::clone(&data); // owner per thread
        handles.push(thread::spawn(move || {
            // each thread reads its own slice, no copy
            data[t * 2..t * 2 + 2].iter().sum::<i32>()
        }));
    }

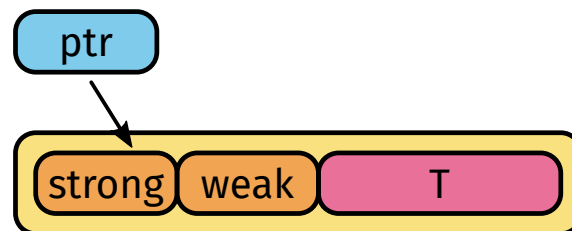
    let total: i32 = handles.into_iter()
        .map(|h| h.join().unwrap()).sum();
    println!("total = {total}"); // 39
}
```

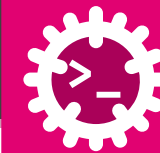
Arc<T>

- **atomic** reference count $\Rightarrow$ Send + Sync
- share **immutable** data across threads, **no copy**
- each thread gets its own **owner** via **clone**
- need to **mutate**? add a lock

$\Rightarrow$ Arc<Mutex<T>>

Arc<T>





Every Rc/Arc carries **two** counts. The value lives as long as **strong > 0**.

```
use std::rc::{Rc, Weak};

let a = Rc::new(5);
let b = Rc::clone(&a);           // strong owner
let w: Weak<i32> = Rc::downgrade(&a); // weak, non-owning

println!("{}", Rc::strong_count(&a)); // 2
println!("{}", Rc::weak_count(&a));  // 1

// a Weak must be upgraded before use
if let Some(rc) = w.upgrade() {    // Some while alive
    println!("still alive: {rc}"); // 5
}                                  // None once strong = 0
```

- **Strong** (strong_count) - the **owning** handles from new/clone. When it hits **0**, the value is **dropped**.
- **Weak** (weak_count) - **non-owning** handles from downgrade. They **do not** keep the value alive.
- upgrade() turns a Weak into an Option<Rc> - None if the value is already gone, so you can never read freed memory.



Use Weak to break **reference cycles** (e.g. child $\Rightarrow$ parent): a cycle of **strong** Rcs never reaches 0 and **leaks**. Arc works identically via Arc::downgrade.



Concept:

- Rc gives **many owners** but only shared **reading**.
- Put a RefCell inside to also **mutate** $\Rightarrow$ borrow rules then checked **at runtime**.

```
use std::rc::Rc;
use std::cell::RefCell;

fn main() {
    // one bank account, co-owned by two parts of the app
    let account = Rc::new(RefCell::new(0));
    let teller = Rc::clone(&account); // second owner

    *account.borrow_mut() += 100; // deposit
    *teller.borrow_mut() -= 30; // withdraw

    // both handles see the same balance
    println!("balance: {}", account.borrow()); // 70
}
```

Rc<RefCell<T>>

- **single thread**: shared **and** mutable
- Rc - **many owners** of one value
- RefCell - mutate through &, rules checked **at runtime** (**panics** if you alias &mut)

RefCell<T>





Concept: the same shape across threads

- Arc shares ownership atomically
- Mutex lets **one thread at a time** mutate the value safely.

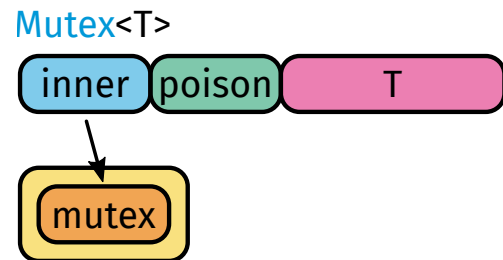
```
use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
    // same account, now shared across worker threads
    let account = Arc::new(Mutex::new(0));

    let mut handles = vec![];
    for _ in 0..3 {
        let account = Arc::clone(&account); // owner per thread
        handles.push(thread::spawn(move || {
            *account.lock().unwrap() += 100; // safe deposit
        }));
    }
    for h in handles { h.join().unwrap(); }
    println!("balance: {}", *account.lock().unwrap()); // 300
}
```

Arc<Mutex<T>>

- **across threads**: shared **and** mutable
- Arc - **atomic** many-owners (Send+Sync)
- Mutex - lock() waits its turn, grants **exclusive** access, guard **auto-unlocks**





Concept: mutate a value **through a shared &**

- no `&mut`, no `Rc`, no lock. The lightweight tool for a single `Copy` field (a counter, a flag).

```
use std::cell::Cell;
struct Button { clicks: Cell<u32> }
impl Button {
    // &self, yet we still mutate the counter
    fn click(&self) {
        self.clicks.set(self.clicks.get() + 1);
    }
    fn count(&self) -> u32 { self.clicks.get() }
}
fn main() {
    let b = Button { clicks: Cell::new(0) };
    b.click();
    b.click();
    println!("clicks: {}", b.count()); // 2
}
```

`Cell<T>`

- mutate through `&self` (no `&mut`)
- get/set **swap the whole value**
- for small **Copy** types, **never panics**
- **single thread only**

`Cell<T>`

`T`



A `Cell` holds one **Copy** value.

To mutate a **non-Copy** value (a `Vec`, a `String`) through `&`, reach for `RefCell`.



Concept: the **lock-free** twin of Cell for one Copy value

- a single hardware instruction does the update, safe to share across threads via Arc.

```
use std::sync::Arc;
use std::sync::atomic::{AtomicU32, Ordering};
use std::thread;

fn main() {
    // shared hit counter, no lock needed
    let hits = Arc::new(AtomicU32::new(0));

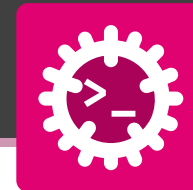
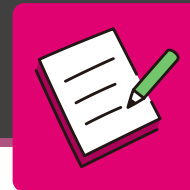
    let mut handles = vec![];
    for _ in 0..4 {
        let hits = Arc::clone(&hits);
        handles.push(thread::spawn(move || {
            for _ in 0..25 {
                hits.fetch_add(1, Ordering::Relaxed);
            }
        }));
    }
    for h in handles { h.join().unwrap(); }
    println!("hits: {}", hits.load(Ordering::Relaxed)); // 100
}
```

Atomic*

- AtomicU32, AtomicBool, AtomicUsize, ...
- **lock-free**: one atomic CPU instruction
- share across threads with Arc
- perfect for **counters** and **flags**



Atomics fit only **Copy** values. For a **non-Copy** shared value use Mutex/ RwLock. (Pin<P> and Cow<'a, T> exist too, but are out of scope here.)

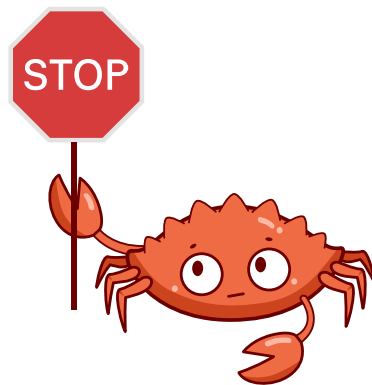


For each situation, name the **smart pointer** you would reach for.

1. A **single owner** of a large value that must live on the **heap**.
2. **Several owners** reading the same data, all on **one** thread.
3. **Several threads** that must **share and mutate** one value.
4. **Shared and mutable** data among many owners, on **one** thread.

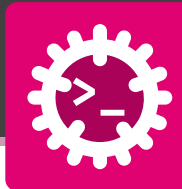


Two questions decide it: **how many threads?** and **read or write?**



Synchronization & Hazards

Locks, races and deadlocks



Critical section - code that must run **exclusively**, one thread inside at a time.

- Guard it with a `Mutex<T>` (or `RwLock<T>` for read-mostly data).
- Keep it **short** - it **serialises** your program (remember Amdahl!).
- Limit to **N** concurrent users? Use a **semaphore** (`tokio::sync::Semaphore`).

Deadlock - two threads wait **forever** on each other's locks:

- Thread A holds lock 1, wants lock 2
- Thread B holds lock 2, wants lock 1
- Neither can proceed $\Rightarrow$ **frozen**.

How to avoid it

- **Lock ordering** - always acquire locks in the **same order**
- **Timeouts** - `try_lock` and back off
- **Less shared state** - prefer **message passing**



Rust stops **data races**, **not** deadlocks
- those are a **design** problem.



Asynchronous Execution Model

Make progress while you wait



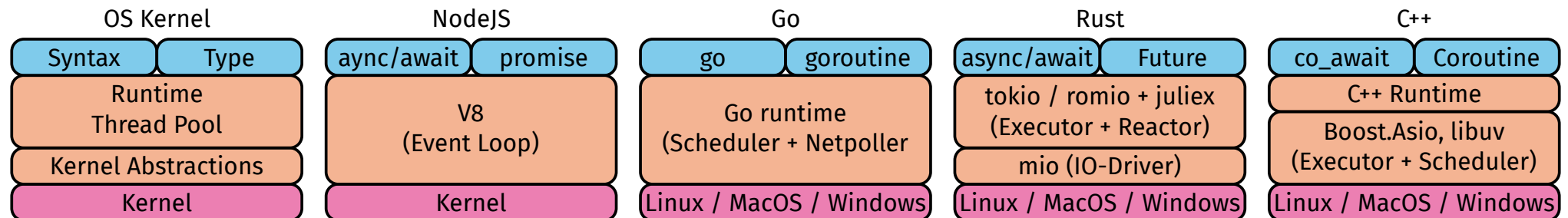
Asynchronous execution is not threads - it is a **separate model**: many **tasks** driven by a **runtime**, instead of **threads** scheduled by the **OS**.

Threads - the **OS** way

- one **stack** per task (~MB), **preemptively** scheduled
- **tens of thousands** of connections $\Rightarrow$ as many threads
- most sit **blocked on I/O**, burning memory

Async - the runtime way

- thousands of **tasks** on a **handful** of threads
- a task is a cheap **state machine**, **cooperatively** scheduled
- ideal for **I/O-bound** waiting at scale





Write code that **reads** sequentially but **doesn't block** the thread.

```
// `async fn` returns a Future instead of running now
async fn fetch(id: u32) → String {
    // .await yields the thread while the IO is pending
    let body = download(id).await;
    format!("got {} bytes", body.len())
}

#[tokio::main]
async fn main() {
    // both tasks share ONE thread, interleaving at .await
    let (a, b) = tokio::join!(fetch(1), fetch(2));
    println!("{a} / {b}");
}
```

- `async fn` - turns code into a **Future**
- `.await` - “**pause here** until ready, **let others run** meanwhile”
- `#[tokio::main]` - starts the **runtime** that drives it all



Idea: run **thousands** of tasks on a **handful** of threads - each one **yields** at `.await` instead of blocking.

```
async fn handler() {  
    // ✗ blocks the WHOLE thread - every other  
    //   task on it is frozen until this returns  
    std::thread::sleep(Duration::from_secs(1));  
  
    // ✔ yields the thread, others make progress  
    tokio::time::sleep(Duration::from_secs(1)).await;  
}
```

- **Preemptive** (OS threads) - the kernel can **interrupt** anytime.
- **Cooperative** (async) - a task runs until it **yields** (`.await`).
- **Never block** inside async: no `thread::sleep`, no heavy CPU loops.
- Offload CPU work to **threads / Rayon**, or use `spawn_blocking`.



The compiler **rewrites** every `async fn` into a **state machine** - one state per `.await`.

```
async fn fetch(id: u32) → String {
    let body = download(id).await; // suspension point
    format!("got {} bytes", body.len())
}

// the compiler turns `fetch` into a state machine:
enum Fetch {
    Start { id: u32 },           // before the .await
    Downloading { fut: Download }, // parked at the .await
    Done,                       // value ready
}

#[tokio::main]
async fn main() {
    // the runtime polls that machine to completion
    let s = fetch(1).await;
    println!("{s}");
}
```

- Each `.await` is a **suspension point** $\Rightarrow$ a **state**
- `poll` advances the machine **one state at a time**
- Locals alive **across** an `.await` are **stored in the future**
- Nothing is blocked - the task just **remembers where it was**



An async block **does nothing** until something **polls** it.

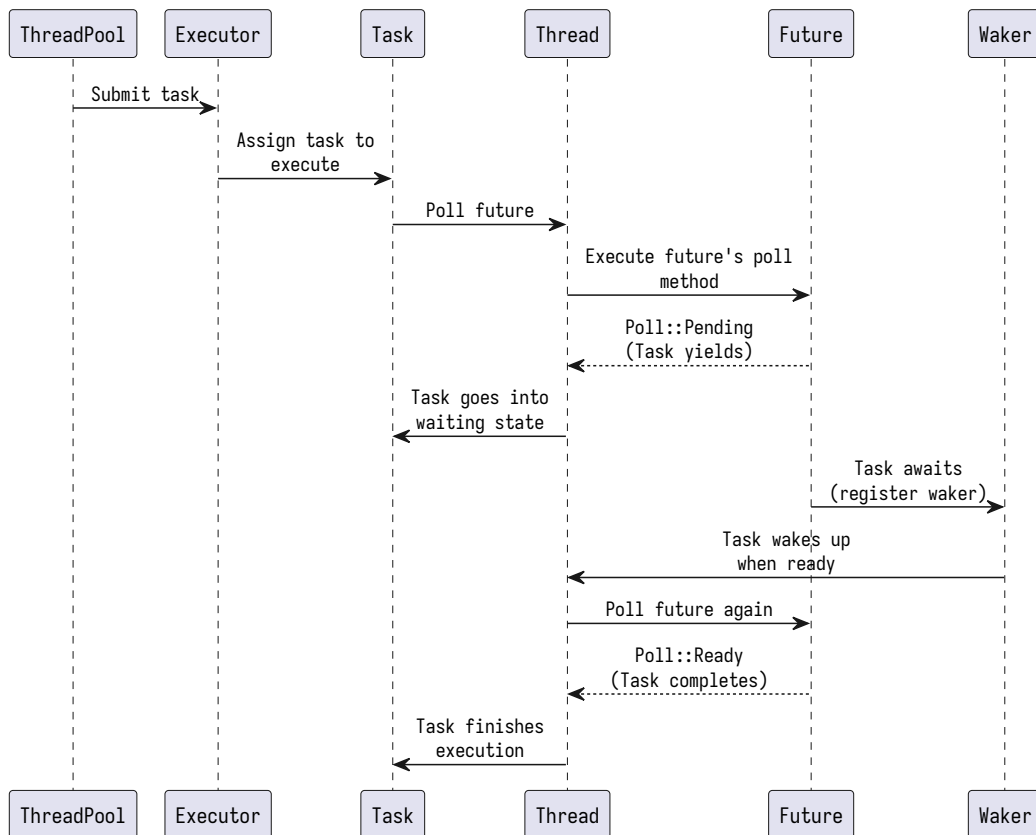
```
enum Poll<T> {
    Ready(T),    // done, here is the value
    Pending,     // not yet - poll me again later
}

// (simplified) a Future is polled to completion
trait Future {
    type Output;
    fn poll(&mut self, cx: &mut Context)
        → Poll<Self::Output>;
}
```

- A Future is a **state machine** that is **polled**.
- Pending $\Rightarrow$ not ready; Ready(v) $\Rightarrow$ finished.
- **Lazy**: nothing runs until an **executor** polls it.
- So who polls, and how often? $\Rightarrow$ the **runtime**.



The runtime is a handful of pieces that **poll your futures to completion**.



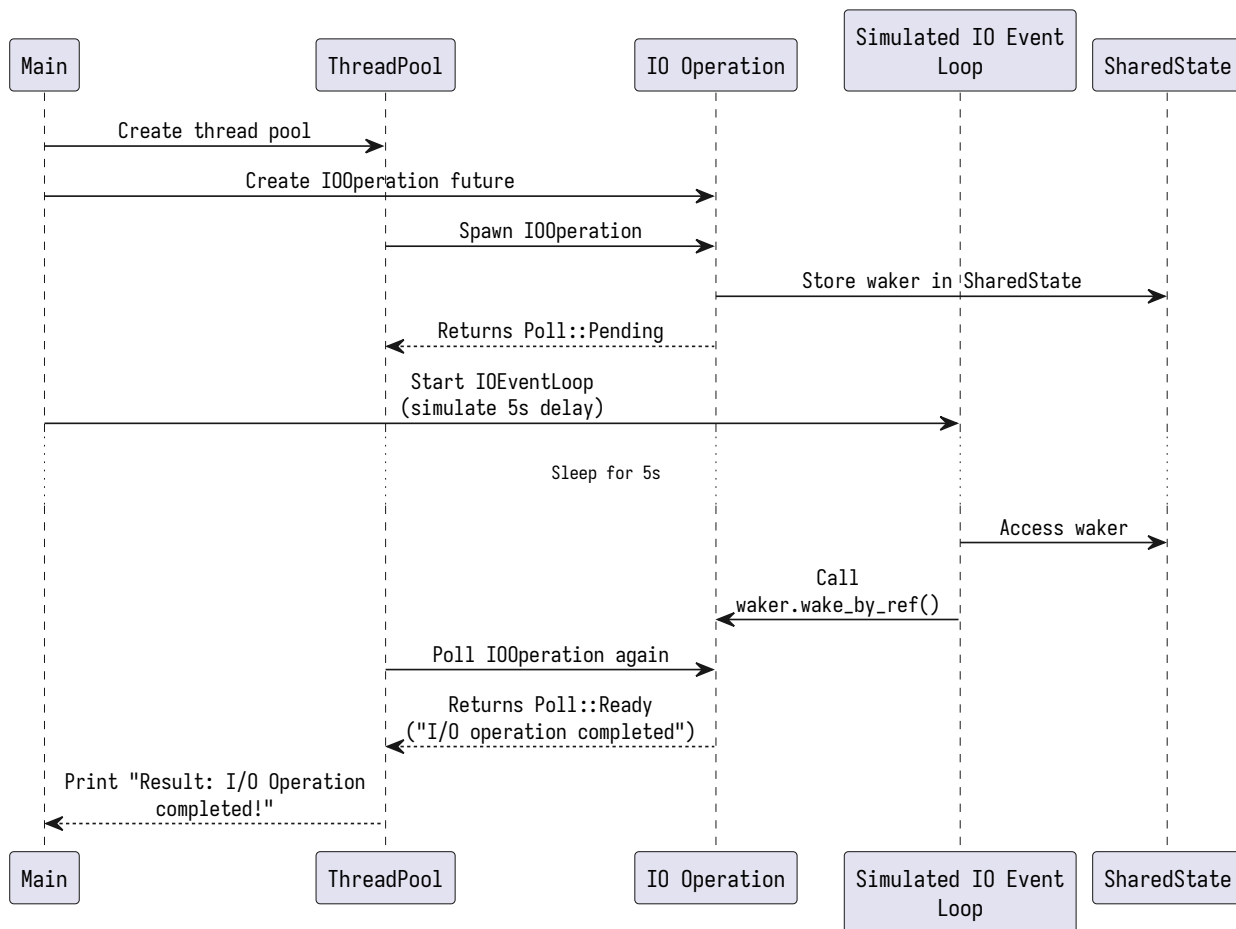
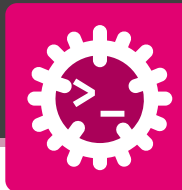
Elements of a runtime

- **Executor** - polls futures on a **thread pool**
- **Reactor** - watches the **OS** for **I/O** readiness
- **Waker** - the “call me back” handle of a parked task

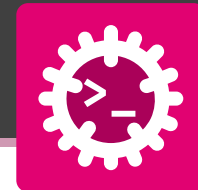
The poll cycle

1. poll $\Rightarrow$ Pending, **register a Waker**
2. task **parked** - uses **no CPU**
3. **I/O** ready $\Rightarrow$ the **Waker** wakes it
4. poll again $\Rightarrow$ Ready $\Rightarrow$ done

Under the hood: putting it together



- A parked task **stores its waker**
- when the **I/O** event fires, `wake()` schedules a **re-poll** $\Rightarrow$ Ready.

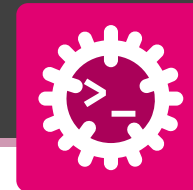
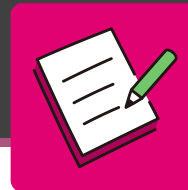


Threads / Rayon	Async / Tokio
best for CPU-bound work	best for I/O-bound work
true parallelism across cores	huge concurrency on few threads
heavy: MB stack per thread	cheap: a task is just a state machine
spawn + join, par_iter	async/.await, a runtime



Rule of thumb

- compute $\Rightarrow$ threads
- wait (network, disk) $\Rightarrow$ async.



The function should **not block** the thread while it waits. Add the **two** missing pieces and answer the question.

```
// (1) make this function asynchronous
fn fetch(id: u32) → String {
    // (2) wait for the future without blocking the thread
    let body = download(id);
    format!("got {} bytes", body.len())
}
```



Question: why use `.await` here instead of a blocking call?



Inter-Thread Communication

Talk, don't share



Threads need to **coordinate** and **hand data** to each other. There are **two ways**.

Share memory - with Smart Pointers, Mutexes, and RwLocks

- one value behind `Arc<Mutex<T>>`
- every thread **locks**, touches it, unlocks
- risk of **races** (without the lock) and **deadlocks**

Pass messages - with channels

- threads **send owned values** down a **channel**
- the receiver **owns** what it gets - no shared lock
- sidesteps data races **and** most deadlocks



“Don’t communicate by sharing memory; share memory by communicating.” - the Go proverb, equally true in Rust.



From **threads in one process** to **separate processes** - same idea, different reach.

Channels - between threads, one process

Mechanism	What it is
mpsc	multi-producer, single-consumer queue - many senders, one receiver (in std)
oneshot	a single value sent once - perfect for a reply / result (Tokio)
broadcast	many consumers each receive every message - fan-out (Tokio)

OS-level Inter-Process Communication (IPC) - between processes

Mechanism	What it is
pipes	a byte stream from one process into another
sockets	a two-way channel, even across machines (the network)
files / shared mem	a region both processes read and write



The **most common** channel: many threads **send**, one thread **receives**.

```
use std::sync::mpsc;
use std::thread;

fn main() {
    let (tx, rx) = mpsc::channel(); // sender, receiver
    thread::spawn(move || {
        tx.send("hello from worker").unwrap(); // value MOVES in
    });
    let msg = rx.recv().unwrap(); // blocks until a message
    println!("{msg}"); // hello from worker
}
```

- `channel()` hands back a **sender** tx and **receiver** rx
- `send` **moves** the value into the channel - the sender gives it up
- `recv` **blocks** until a message arrives (or every sender is dropped)
- **clone tx** to get **multiple producers**



Channels turn **shared-state** problems into simple **producer / consumer** flows.



Let **3 worker threads** report their result to `main` through an `mpsc` channel - **no shared lock**.

1. Create a channel with `mpsc::channel()`.
2. Spawn **3** threads; each **clones** the sender and sends its `id * 10`.
3. In `main`, **receive** the 3 values and print their **sum**.



Clone tx for multiple producers; send **moves** the value into the channel; recv **blocks** until a message arrives.



Debug / Testing / Documentation

Ship code that works - and prove it



The safety net of a healthy Rust project - each layer catches a different class of mistake.

- **Unit tests** - `#[test]` functions, run by `cargo test`
- **Integration & doc tests** - the `tests/` folder and runnable examples in docs
- **cargo check / clippy** - fast type-check and lints
- **cargo fmt** - one canonical style
- **Error handling** - `Result` / `Option`, no surprise crashes
- **Logging & debugging** - see what the program actually did
- **Profiling** - measure before you optimise
- **CI/CD** - run **all** of the above automatically on every push

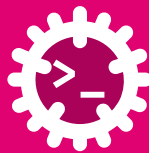


The goal: catch a mistake **as early and as cheaply** as possible - ideally at compile time.



Testing

Prove the code does what you claim



A **unit test** checks **one** piece of code in **isolation** - including the **edge cases**.

```
// the code under test
pub fn parse_temperature(s: &str) → Option<i32> {
    s.trim().trim_end_matches("°C").parse().ok()
}

#[cfg(test)]                // compiled only for `cargo test`
mod tests {                 // a *submodule* for the tests
    use super::*;           // bring the parent module into scope

    #[test]                 // a *unit test* function
    fn plain_number() {
        assert_eq!(parse_temperature("20"), Some(20));
    }

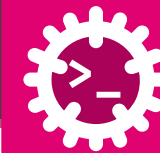
    #[test]
    fn strips_unit_and_spaces() {
        assert_eq!(parse_temperature(" 20°C "), Some(20));
    }

    #[test]
    fn rejects_garbage() {
        assert_eq!(parse_temperature("hot"), None);
    }
}
```

- lives **next to the code**, in `#[cfg(test)]` mod tests
- `#[test]` marks each test function
- the rhythm: **arrange** ⇒ **act** ⇒ **assert**
- test the **happy path**, the **edge**, and the **failure**
- run them all with `cargo test`



A failure prints the **file, line** and **both values** - it points right at the bug.



The **same** function, tested two more ways - both run under cargo test.

Integration Tests

files in `tests/`, exercise the crate **from outside**

- only public API is visible

```
// tests/temperature.rs
use weather::parse_temperature;

#[test]
fn reads_celsius() {
    assert_eq!(parse_temperature("20°C"), Some(20));
    assert_eq!(parse_temperature("nope"), None);
}
```

Doc tests

Code in `///` examples is **compiled and run** ⇒
documentation that **can't go stale**

```
/// Parses a temperature like `20°C`. DOC test
///
/// # Examples
/// ```
/// use weather::parse_temperature;
/// assert_eq!(parse_temperature("20°C"), Some(20));
/// ```
pub fn parse_temperature(s: &str) → Option<i32> {
    // ...
}
```



An assertion says what **must** be true - and sometimes the **correct** behaviour is to **panic**.

```
// °F → °C, rejecting impossible input
fn f_to_c(f: f64) → f64 {
    assert!(f ≥ -459.67, "below absolute zero");
    (f - 32.0) * 5.0 / 9.0
}

#[test]
fn freezing_and_boiling() {
    assert_eq!(f_to_c(32.0), 0.0); // = (both shown on fail)
    assert_ne!(f_to_c(212.0), 0.0); // ≠ (it is 100.0)
}

#[test]
#[should_panic(expected = "below absolute zero")]
fn rejects_impossible() { f_to_c(-500.0); } // pass if it panics

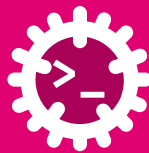
#[test]
fn parses_ok() → Result<(), String> {
    let t = parse_temperature("20°C").ok_or("bad")?;
    assert_eq!(t, 20); // a test may return Result
    Ok(())
}
```

- `assert_eq!` / `assert_ne!` print **both values** on failure
- `assert!(cond, "msg")` - assert a bool, with a **message**
- `#[should_panic]` - the test **passes only if it panics**
- a test may **return Result** - ? inside, an `Err` fails it
- filter: `cargo test freezing` runs all function with the word `freezing` in the function name.
⇒ Here only `freezing_and_boiling()`



Code Quality & CI

Automate the boring checks



Three commands that catch **most** mistakes before a single test runs.

```
cargo fmt      # one canonical style
cargo check    # type-check only, no binary
cargo clippy    # ~750 lints, catches foot-guns
# auto-fix where possible
cargo fmt --all
cargo clippy --fix
```

Rust **fmt** can be configured with `rustfmt.toml` - but the **default** is good enough for most projects. Just run it **on save** in your editor and forget about it.

```
edition = "2024"
tab_spaces = 2
max_width = 160
reorder_imports = true
```

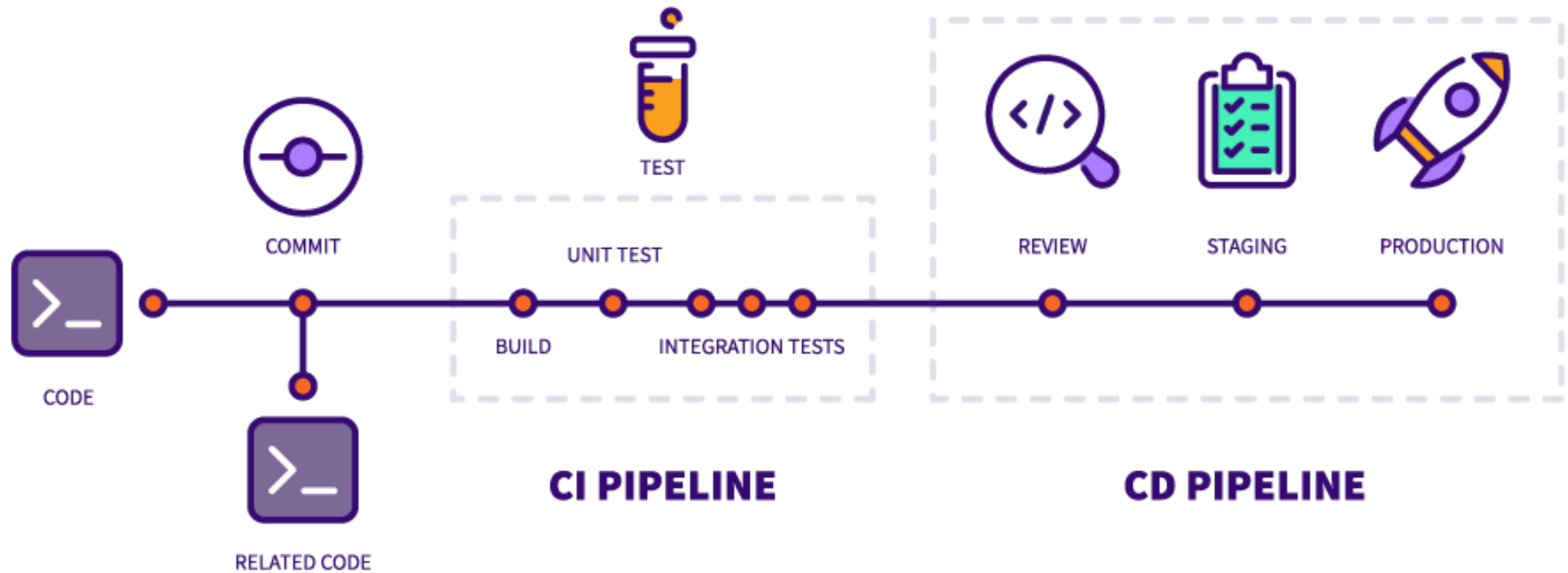
- **fmt** - rustfmt; run it **on save** and end style arguments
- **check** - compiler **front-end** only → fast feedback loop
- **clippy** - idiomatic-Rust **linter**; explains **why** and suggests a fix

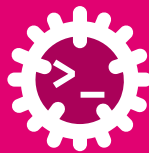


In **CI**, make warnings fatal: `cargo clippy -- -D warnings`.



Run every check **automatically** on each **push / pull-request**





```
# .github/workflows/ci.yml
on: [push, pull_request]
jobs:
  fmt:
    - run: cargo fmt --all -- --check
  clippy:
    - run: cargo clippy -- -D warnings
  test:
    - run: cargo test --verbose
  build:
    - run: cargo build --verbose
```

- the **same** commands you run locally - now enforced for **everyone**
- four parallel jobs: **fmt** · **clippy** · **test** · **build**
- on **every push & PR** to main



Green check = formatted, lint-clean, builds, **all tests pass.**

The screenshot shows the GitHub Actions interface for a workflow named 'ci.yml' triggered by a push to the 'main' branch. The status is 'Success'. The summary card shows a total duration of 1m 38s. A list of jobs on the left includes Test Suite, Rustfmt, Clippy, Build Check, and three cross-platform build jobs (ubuntu-latest, windows-latest, macos-latest), all with green checkmarks. The 'Run details' section shows a 'Matrix: Cross-platform build' with 3 jobs completed. The 'Test Suite' job details show durations: Test Suite (50s), Rustfmt (8s), Clippy (24s), and Build Check (50s).

`unwrap()` was a mistake



Handling Failures

When things go wrong, gracefully



`Result<T, E>` and `Option<T>` make failure **part of the type** (see **Imperative Programming**).

Method	What it does	Use when
Propagate / handle		
<code>?</code>	hands the <code>Err/None</code> to the caller	the caller should decide
<code>match / if let</code>	branch on each case explicitly	you handle both paths here
Fallback - never panics		
<code>unwrap_or(x)</code>	the value, or <code>x</code>	you have a cheap default
<code>unwrap_or_else(f)</code>	the value, or <code>f()</code>	the default is expensive to build
<code>unwrap_or_default()</code>	the value, or <code>T::default()</code>	<code>0</code> / <code>""</code> / empty is fine
Panic - give up		
<code>unwrap()</code>	the value, or panic	tests / “can’t happen”
<code>expect("msg")</code>	the value, or panic w/ msg	document the invariant
<code>panic!()</code>	abort the thread	unrecoverable bug



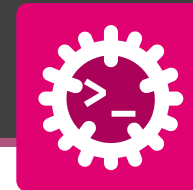
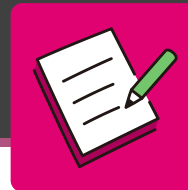
```
use std::fs;
fn load_port(path: &str) → Option<u16> {
    // ? propagates None upward if read fails
    let text: String = fs::read_to_string(path).ok()?;
    // ? propagates None upward if parsing fails
    let port: u16 = text.trim().parse().ok()?;
    Some(port)
}

fn main() {
    // 1. Fallback - never panics: use 8080 if loading fails
    let port: u16 = load_port("app.toml").unwrap_or(8080);
    // 2. Handle both arms explicitly: success and failure
    if let Some(p) = load_port("app.toml") {
        println!("listening on {p}");
    } else {
        eprintln!("config problem, using defaults");
    }
    // 3. Fallback with a computed/expensive default
    let port: u16 = load_port("app.toml").unwrap_or_else(|| {
        eprintln!("falling back to default port");
        8080
    });
}
```

```
// 4. Fallback to the type's default value
let port: u16 = load_port("app.toml").unwrap_or_default();
// 5. Last resort in a prototype: panic with a message if
// config is missing/invalid
let port: u16 = load_port("app.toml").expect("config
required");
// 6. Unrecoverable state: abort explicitly
if port == 0 {
    panic!("port 0 is never valid");
}
}
```

In the example

- ? - bubble the error **up**
- unwrap_or - safe default 8080
- if let / match - handle **both** arms
- expect - **panic** w/ message
- panic! - **abort** the thread



The code is given. On each `// TODO` line, add the **right method** from the failure toolbox - each line uses a **different** one (`?`, `unwrap_or`, `unwrap_or_else`, `unwrap_or_default`, `expect`).

```
use std::num::ParseIntError;

// parse_port returns Result<u16, ParseIntError>
fn parse_port(s: &str) → Result<u16, ParseIntError> {
    s.trim().parse()
}

// TODO 1: propagate the Err to the caller (one operator)
fn sum_ports(a: &str, b: &str)
    → Result<u16, ParseIntError> {
    let x = parse_port(a)/* ? */; // → u16
    let y = parse_port(b)/* ? */; // → u16
    Ok(x + y)
}
```

```
fn main() {
    // TODO 2: use 8080 when parsing fails (cheap default)
    let p1: u16 = parse_port("9000")/* ? */;

    // TODO 3: build the default lazily (only if needed)
    let p2: u16 = parse_port("oops")/* ? */;

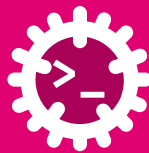
    // TODO 4: fall back to u16::default() (= 0)
    let p3: u16 = parse_port("oops")/* ? */;

    // TODO 5: panic with a message if it must succeed
    let p4: u16 = parse_port("oops")/* ? */;
}
```



Debugging

Find the bug



The compiler is your **first** debugger - then reach for a print.

```
fn main() {  
    let xs = vec![1, 2, 3];  
    println!("xs = {xs:?}"); // you choose what to show  
    let _n = dbg!(xs.len()) * 2; // prints AND returns the value  
}
```

```
Compiling playground v0.0.1 (/playground)  
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.49s  
Running `target/debug/playground`
```

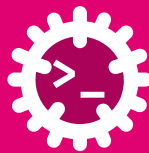
```
[src/main.rs:6:12] xs.len() = 3 ← dbg! prints *file:line*  
                             the *expression* and its *value*
```

```
xs = [1, 2, 3]                ← println! prints what you choose
```

- **println!** - quick, but you **format it yourself**
- **dbg!(expr)** - prints **file:line**, the **expression** and its **value**, then **returns it**
- **rustc --explain E0382** - the **full story** behind an error code



Read the borrow-checker hint first - it usually names the fix.



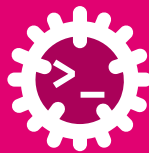
Beyond a quick print: **levels** you can switch on/off **without recompiling**.

```
use log::{info, warn, error};

fn main() {
    env_logger::init();
    info!("starting on port {}", 8080);
    warn!("connection slow, retrying");
    error!("giving up");
}
```

```
RUST_LOG=info cargo run    # show info and above, everywhere
RUST_LOG=debug cargo run   # show debug and above
RUST_LOG=trace cargo run   # everything
```

- levels: error! > warn! > info! > debug! > trace!
- **log** is a **facade** - pick a backend: **env_logger**, **tracing**
- **tracing** - structured spans, great for **async** / production
- filter via RUST_LOG, **no code change**



```
===== RUST_LOG=info =====  
[2026-06-23T06:41:25Z INFO logging_levels] application starting  
[2026-06-23T06:41:25Z WARN logging_levels::net] attempt 1 failed, retrying  
[2026-06-23T06:41:25Z WARN logging_levels::net] attempt 2 failed, retrying  
[2026-06-23T06:41:25Z INFO logging_levels::net] connected on attempt 3  
[2026-06-23T06:41:25Z WARN logging_levels] disk almost full: 92% used  
[2026-06-23T06:41:25Z ERROR logging_levels] database connection lost  
[2026-06-23T06:41:25Z INFO logging_levels] application stopped
```

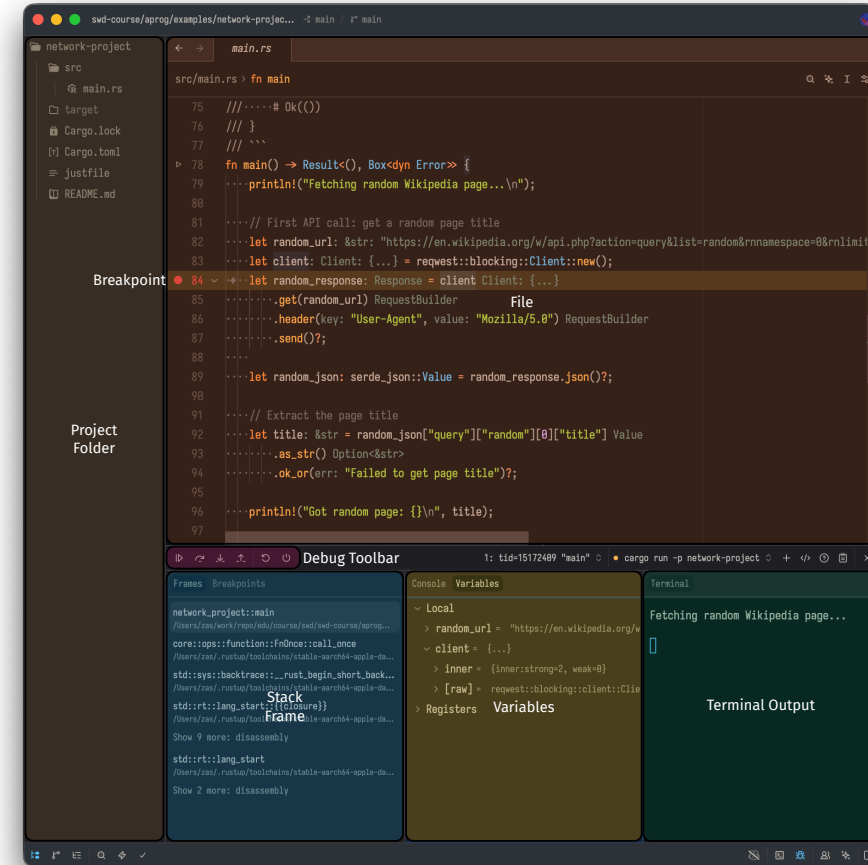
```
===== RUST_LOG=debug =====  
[2026-06-23T06:41:25Z INFO logging_levels] application starting  
[2026-06-23T06:41:25Z DEBUG logging_levels::net] attempt 1/3  
[2026-06-23T06:41:25Z WARN logging_levels::net] attempt 1 failed, retrying  
[2026-06-23T06:41:25Z DEBUG logging_levels::net] attempt 2/3  
[2026-06-23T06:41:25Z WARN logging_levels::net] attempt 2 failed, retrying  
[2026-06-23T06:41:25Z DEBUG logging_levels::net] attempt 3/3  
[2026-06-23T06:41:25Z INFO logging_levels::net] connected on attempt 3  
[2026-06-23T06:41:25Z WARN logging_levels] disk almost full: 92% used  
[2026-06-23T06:41:25Z ERROR logging_levels] database connection lost  
[2026-06-23T06:41:25Z INFO logging_levels] application stopped
```

```
===== RUST_LOG=trace =====  
[2026-06-23T06:41:25Z INFO logging_levels] application starting  
[2026-06-23T06:41:25Z TRACE logging_levels::net] entering connect()  
[2026-06-23T06:41:25Z DEBUG logging_levels::net] attempt 1/3  
[2026-06-23T06:41:25Z WARN logging_levels::net] attempt 1 failed, retrying  
[2026-06-23T06:41:25Z DEBUG logging_levels::net] attempt 2/3  
[2026-06-23T06:41:25Z WARN logging_levels::net] attempt 2 failed, retrying  
[2026-06-23T06:41:25Z DEBUG logging_levels::net] attempt 3/3  
[2026-06-23T06:41:25Z INFO logging_levels::net] connected on attempt 3  
[2026-06-23T06:41:25Z WARN logging_levels] disk almost full: 92% used  
[2026-06-23T06:41:25Z ERROR logging_levels] database connection lost  
[2026-06-23T06:41:25Z INFO logging_levels] application stopped
```

Debugging with Breakpoints (in Zed / VS Code)



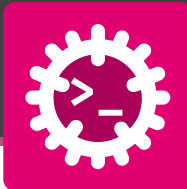
1. Open the *main.rs* file and click the *gutter* to set a *breakpoint*
2. run the program in *debug mode* F4
3. Go through the program with the following actions:
 - *continue* - F5 - run until the next breakpoint
 - *step over* - F7 - run the current line
stop at the next line
 - *step into* - Ctrl+F11 - run the current line
stop at the first line of the called function
 - *step out* - Shift+F11 - run until the current
function returns
4. **Inspect** live the:
 - *stack frame* - see the **call stack**
 - *variables* - see the **current values** of variables in scope
 - *terminal* - see the **program output**





Profiling

Make it fast



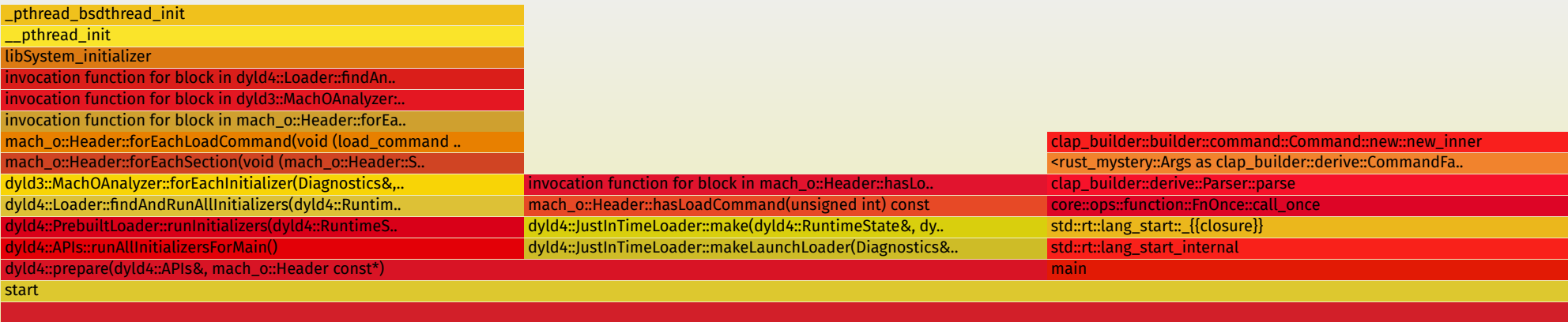
```
hyperfine --warmup 3 --show-output --min-runs 10 "release/rust_mystery_v1_0_0_Mac_AARCH64 -m 1"
```

Execution Mystery Program 1

```
...
Time (mean ± σ):      2.013 s ±  0.002 s   [User: 0.003 s, System: 0.004 s]
Range (min ... max):  2.009 s ...  2.014 s   10 runs
```

Flame Graph

Search





```
...
Time (mean ± σ):      2.213 s ± 0.009 s    [User: 2.192 s, System: 0.016 s]
Range (min ... max):  2.201 s ... 2.227 s    10 runs
```

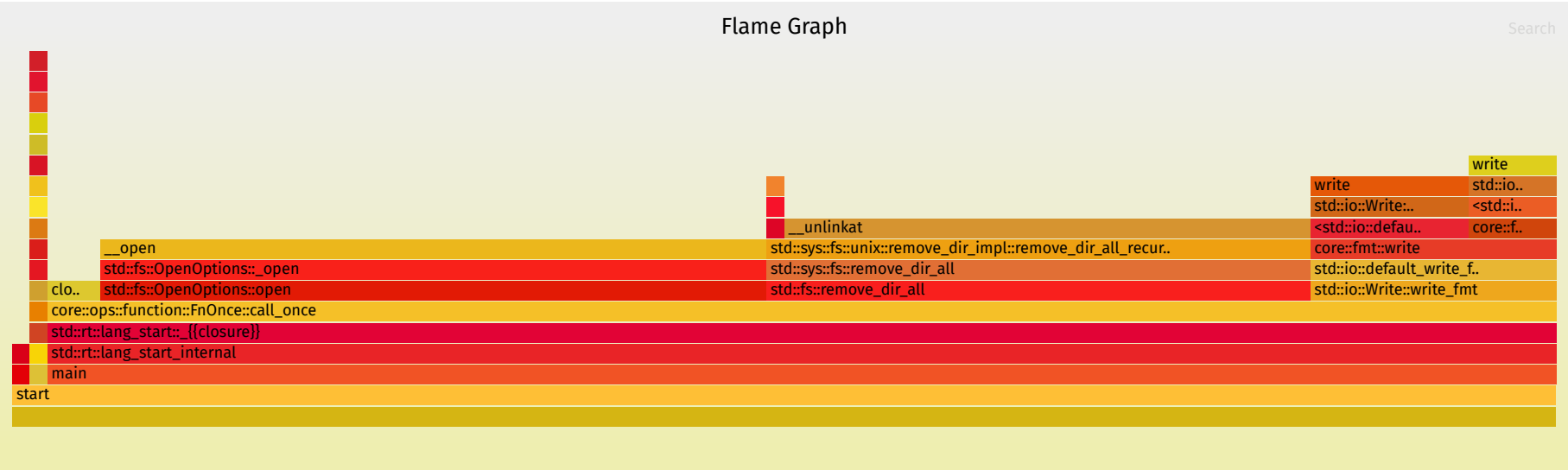




```
hyperfine --warmup 3 --show-output --min-runs 10 "release/rust_mystery_v1_0_0_Mac_AARCH64 -m 3"
```

Execution Mystery Program 3

```
...
Time (mean ± σ):    115.0 ms ± 18.9 ms   [User: 2.5 ms, System: 107.6 ms]
Range (min ... max): 83.0 ms ... 143.0 ms   24 runs
```





```
hyperfine --warmup 3 --show-output --min-runs 10 "release/rust_mystery_v1_0_0_Mac_AARCH64 -m 4"
```

Execution Mystery Program 4

```
...
Time (mean ± σ):      760.5 ms ± 45.9 ms   [User: 10451.8 ms, System: 41.7 ms]
Range (min ... max):  703.5 ms ... 852.2 ms   10 runs
```

Flame Graph

Search

```
<core::num::<impl i32>::overflowing_add rust_mystery::mystery_parallel_cpu::_{{closure}}
std::thread::Builder::spawn_unchecked::_{{closure}}::_{{closure}}
std::sys::pal::unix::thread::Thread::new::thread_start
_pthread_start
thread_start
```

Mystery Program 5



```
hyperfine --warmup 3 --show-output --min-runs 10 "release/rust_mystery_v1_0_0_Mac_AARCH64 -m 5"
```

```
Execution Mystery Program 5
results front: 301
results back: 1300
kernel execution duration (ns): 204
...
Time (mean ± σ):      47.4 ms ±   5.6 ms   [User: 11.0 ms, System: 13.5 ms]
Range (min ... max):  41.0 ms ... 79.5 ms   62 runs
```





Documentation

Code others (and future-you) can read



Doc comments become a **browsable website** - and the examples are **compiled and tested**.

```
//! # mycrate
//!
//! Temperature conversions for the AProg course.
//! Crate-level docs (`//!`) sit at the top of `lib.rs`
//! and become the front page of the generated site.
//!
//! # Examples
//! ```
//! use mycrate::temperature::Celsius;
//! assert_eq!(Celsius::new(0.0).to_fahrenheit(), 32.0);
//! ```
/// Temperature handling.
pub mod temperature {
    /// A temperature in degrees Celsius.
    ///
    /// # Examples
    /// ```
    /// use mycrate::temperature::Celsius;
    /// let t = Celsius::new(25.0);
    /// assert_eq!(t.to_fahrenheit(), 77.0);
    /// ```
```

```
pub struct Celsius(f64);
impl Celsius {
    /// Creates a new [`Celsius`] value.
    pub fn new(deg: f64) → Self {
        Self(deg)
    }
    /// Converts to Fahrenheit.
    ///
    /// # Panics
    /// Never - this conversion is total.
    pub fn to_fahrenheit(&self) → f64 {
        self.0 * 9.0 / 5.0 + 32.0
    }
}
```



Documentation lives **next to the code** - the same source is both the comments and the published site.

Writing the comments

- `///` documents the **next item** - a struct, method, or function
- `//!` documents the **enclosing** item - the module or whole crate
- **Markdown** inside: headings, lists, code blocks
- convention sections: `# Examples`, `# Panics`, `# Errors`
- `[Celsius]` - an **intra-doc link** to another item

Building & testing

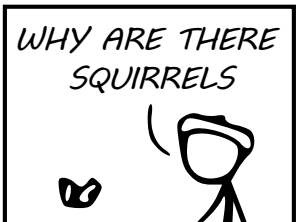
- the `# Examples` block is **real code**, compiled and run
- `cargo test` executes every doc example → docs **can't drift**
- public crates publish automatically to **docs.rs**

```
# build the HTML docs, open in browser
cargo doc --open

# also compiles & runs the doc examples
cargo test
```

WHY DO WHALES JUMP
WHY ARE THERE MIRRORS ABOVE BEDS
WHY DO I SAY UH
WHY IS SEA SALT BETTER
WHY ARE THERE TREES IN THE MIDDLE OF FIELDS
WHY IS THERE NOT A POKEMON MMO
WHY IS THERE LAUGHING IN TV SHOWS
WHY ARE THERE DOORS ON THE FREEWAY
WHY ARE THERE SO MANY SVCHOST.EXE RUNNING
WHY AREN'T ANY COUNTRIES IN ANTARCTICA
WHY ARE THERE SCARY SOUNDS IN MINECRAFT
WHY IS THERE KICKING IN MY STOMACH
WHY ARE THERE TWO SLASHES AFTER HTTP
WHY ARE THERE CELEBRITIES
WHY DO SNAKES EXIST
WHY DO OYSTERS HAVE PEARLS
WHY ARE DUCKS CALLED DUCKS
WHY DO THEY CALL IT THE CLAP
WHY ARE KYLE AND CARTMAN FRIENDS
WHY IS THERE AN ARROW ON AANG'S HEAD
WHY ARE TEXT MESSAGES BLUE
WHY ARE THERE MUSTACHES ON CLOTHES
WHY WUBA LUBBA DUB DUB MEANING
WHY IS THERE A WHALE AND A POT FALLING
WHY ARE THERE SO MANY BIRDS IN SWISS
WHY IS THERE SO LITTLE RAIN IN WALLIS
WHY IS WALLIS WEATHER FORECAST ALWAYS WRONG
WHY ARE THERE MALE AND FEMALE BIKES

WHY ARE THERE BRIDESMAIDS
WHY DO DYING PEOPLE REACH UP
HOW FAST IS LIGHTSPEED
WHY ARE OLD KLINGONS DIFFERENT

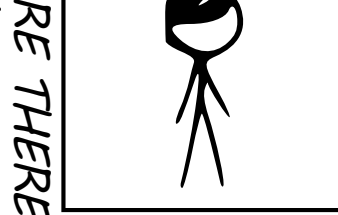


WHY ARE THERE TINY SPIDERS IN MY HOUSE
WHY DO SPIDERS COME INSIDE
WHY ARE THERE HUGE SPIDERS IN MY HOUSE
WHY ARE THERE LOTS OF SPIDERS IN MY HOUSE
WHY ARE THERE SPIDERS IN MY ROOM
WHY ARE THERE SO MANY SPIDERS IN MY ROOM
WHY DO SPYDER BITES ITCH

WHY ARE HATS SO EXPENSIVE
WHY IS THERE CAFFEINE IN MY SHAMPOO
WHY HAVE DINOSAURS NO FUR
WHY DO IGUANAS DIE
WHY AREN'T ECONOMISTS RICH
WHY DO AMERICANS CALL IT SOCCER
WHY ARE MY EARS RINGING
WHY IS 42 THE ANSWER TO EVERYTHING
WHY CAN'T NOBODY ELSE LIFT THORS HAMMER
WHY IS MARVIN ALWAYS SO SAD
WHY IS EARTH TILTED
WHY IS SPACE BLACK
WHY IS OUTER SPACE SO COLD
WHY ARE THERE PYRAMIDS ON THE MOON
WHY IS NASA SHUTTING DOWN
WHY ARE THERE FEMALE

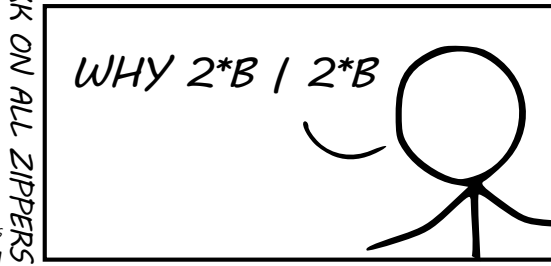
WHY ARE THERE TINY SPIDERS IN MY HOUSE
WHY DO SPIDERS COME INSIDE
WHY ARE THERE HUGE SPIDERS IN MY HOUSE
WHY ARE THERE LOTS OF SPIDERS IN MY HOUSE
WHY ARE THERE SPIDERS IN MY ROOM
WHY ARE THERE SO MANY SPIDERS IN MY ROOM
WHY DO SPYDER BITES ITCH

WHY ARE TWINS HAVE DIFFERENT FINGERPRINTS
WHY ARE SWISS AFRAID OF DRAGONS
WHY IS THERE A SWARM OF ANTS
WHY IS THERE PILGRIM
WHY IS THERE LAVA
WHY AREN'T THERE GUNS IN HARRY POTTER
WHAT IS <https://xkcd.com/1256/>
WHY DO THEY SAY T-MINUS
WHY ARE OCEANS BECOMMING MORE ACIDIC



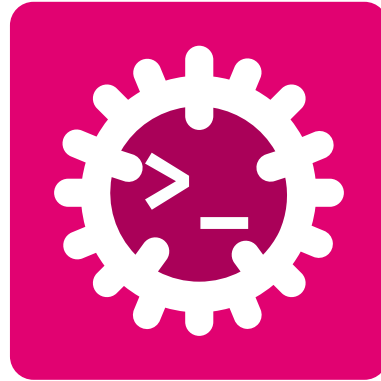
WHY IS HTTPS CROSSED OUT
WHY IS THERE A LINE THROUGH
WHY IS THERE A RED LINE THROUGH HTTPS
WHY IS HTTPS IMPOR
WHY ARE THERE SO MANY CROWS IN ROCHES
WHY IS TO BE OR NOT TO BE FL
WHY DO CHILDREN GET CANCER
WHY IS POSEIDON ANGRY WITH ODYSSEUS
WHY IS THERE ICE IN SPACE
WHY IS THERE AN OWL IN MY BACKYARD
WHY IS THERE AN OWL OUTSIDE MY WINDOW
WHY IS THERE AN OWL ON THE DOLLAR BILL
WHY DO OWLS ATTACK PEOPLE
WHY ARE FPGA's EVERYWHERE
WHY ARE THERE HELICOPTERS CIRCLING MY HOUSE
WHY ARE THERE GODS
WHY ARE THERE TWO SPOCKS
WHY IS LI

WHY ARE MY BOOBS ITCHY
WHY ARE CIGARETTES LEGAL
WHY ARE THERE DUCKS IN MY POOL
WHY IS JESUS WHITE
WHY IS THERE LIQUID IN MY EAR
WHY DO Q TIPS FEEL GOOD
WHY DO PEOPLE DIE



QUESTIONS

CAN BE ASKED BY ANYONE ANYTIME



Glossary & Bibliography



Computer Science

ASCII – American Standard Code for Information Interchange: Character encoding standard for representing text. [63](#)

CPU – Central Processing Unit: The main processing component of a computer. [232](#), [232](#), [232](#), [233](#), [233](#), [233](#), [233](#), [256](#), [256](#), [259](#), [261](#), [288](#)

OS – Operating System: System software that manages hardware resources and applications. [219](#), [222](#), [231](#), [232](#), [232](#), [232](#), [254](#), [254](#), [256](#), [259](#), [265](#)

RAM – Random Access Memory: Volatile memory used to store data and programs during execution. [23](#), [24](#), [24](#), [24](#)

RGB – Red Green Blue: Color model representing colors as combinations of red, green, and blue. [63](#)

Concurrency

I/O – Input/Output: Operations that involve reading from or writing to external devices or resources. [232](#), [232](#), [232](#), [233](#), [233](#), [254](#), [254](#), [259](#), [259](#), [260](#), [261](#), [288](#)

IPC – Inter-Process Communication: Mechanisms allowing processes to exchange data and synchronize execution. [265](#)

Course

AProg – Advanced Programming: Course covering advanced programming concepts using Rust. [5](#), [10](#)

ProgA – Applied Programming: Project-oriented course focused on applying programming concepts in practice. [5](#)

Data Structure

FIFO – First In First Out: Queue discipline where the oldest element is processed first. [139](#), [150](#)

Programming

API – Application Programming Interface: A set of rules and interfaces that allows software components to communicate. [40](#)

Heap – Heap: Memory area used for dynamic memory allocation. [23](#), [23](#), [24](#), [24](#), [82](#)



HTML – HyperText Markup Language: Markup language used to create web pages and generated documentation. [41](#)

OOP – Object-Oriented Programming: Programming paradigm based on abstraction, encapsulation, modularity and hierarchy. [158](#), [159](#), [159](#), [189](#), [191](#), [192](#), [192](#), [192](#), [193](#), [195](#)

RAII – Resource Acquisition Is Initialization: Resource management technique where acquisition and release are tied to object lifetime. [23](#), [205](#), [205](#), [210](#), [212](#), [212](#)

Stack – Stack: Memory area used for static memory allocation and function call management. [23](#), [23](#), [24](#), [24](#), [47](#)

USV – Unicode Scalar Value: A Unicode code point that is not a surrogate and represents a single character. [80](#)

UTF – Unicode Transformation Format: Encoding standard for representing Unicode characters. [82](#)

Technology

AWS – Amazon Web Services: Cloud computing platform provided by Amazon. [16](#)



Bibliography

- [1] S. Klabnik, C. Nichols, and C. Krycho, “The Rust Programming Language.”
- [2] J. Blandy, J. Orendorff, and L. F. s Tindall, *Programming Rust: Fast, Safe Systems Development*. O'Reilly Media, Incorporated, 2021.
- [3] H. Collard, *Black Hat Rust*, vol. 1. Amazon, 2025.
- [4] A. Shvets, “Refactoring Guru.” 2026.
- [5] R. C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship*. Pearson Education, 2008.
- [6] “SVG Repo - Free SVG Vectors and Icons.”
- [7] HEI-VS, “Module Description SWD1.”
- [8] HEI-VS, “Module Description SWD2.”
- [9] R. Pereira *et al.*, “Energy Efficiency across Programming Languages: How Do Energy, Time, and Memory Relate?,” in *Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering*, in SLE 2017. New York, NY, USA: Association for Computing Machinery, Oct. 2017, pp. 256–267. doi: [10.1145/3136014.3136031](https://doi.org/10.1145/3136014.3136031).
- [10] Amazing Info TV, “Ariane 5 Rocket Launch Explosion.” Feb. 2015.